

The Project Companion Guide

A learning companion for the 300-Project Space-Edition Curriculum

*“Confusion means you are at the edge of understanding.
Stay there a little longer than feels comfortable,
and the edge becomes the centre.”*

Read the chapter for a project, then build the project on your own.

For Stephane, Kigali.

Contents

Opening — How to Use This Guide	1
How this guide works alongside the curriculum	1
The learning loop: read → understand → build → reflect	1
How to handle being stuck (the most important skill)	1
Using spaced repetition with this guide	1
Setting up your Python environment	2
How to read code (actively, with a pencil)	2
The right mindset	2
Hello World CLI	4
① What you are building	4
② The concepts you need	4
③ Concept deep dives	4
④ How the concepts connect	8
⑤ The build walkthrough	9
⑥ Understanding the design	9
⑦ The exercise twist	9
⑧ What to remember from this chapter	9
⑨ Coming up next	10
Temperature Converter	11
① What you are building	11
② The concepts you need	11
③ Concept deep dives	11
④ How the concepts connect	15
⑤ The build walkthrough	15
⑥ Understanding the design	15
⑦ The exercise twist	15
⑧ What to remember from this chapter	16
⑨ Coming up next	16
Number Guessing Game	17
① What you are building	17
② The concepts you need	17
③ Concept deep dives	17
④ How the concepts connect	21
⑤ The build walkthrough	21
⑥ Understanding the design	21
⑦ The exercise twist	21
⑧ What to remember from this chapter	21
⑨ Coming up next	22
Calculator with History	23
① What you are building	23
② The concepts you need	23
③ Concept deep dives	23
④ How the concepts connect	26
⑤ The build walkthrough	26
⑥ Understanding the design	27
⑦ The exercise twist	27
⑧ What to remember from this chapter	27
⑨ Coming up next	27
File Word Counter	28

① What you are building	28
② The concepts you need	28
③ Concept deep dives	28
④ How the concepts connect	31
⑤ The build walkthrough	31
⑥ Understanding the design	31
⑦ The exercise twist	32
⑧ What to remember from this chapter	32
⑨ Coming up next	32
Contact Book	33
① What you are building	33
② The concepts you need	33
③ Concept deep dives	33
④ How the concepts connect	36
⑤ The build walkthrough	36
⑥ Understanding the design	37
⑦ The exercise twist	37
⑧ What to remember from this chapter	37
⑨ Coming up next	37
Todo List	38
① What you are building	38
② The concepts you need	38
③ Concept deep dives	38
④ How the concepts connect	41
⑤ The build walkthrough	41
⑥ Understanding the design	41
⑦ The exercise twist	42
⑧ What to remember from this chapter	42
⑨ Coming up next	42
Password Generator	43
① What you are building	43
② The concepts you need	43
③ Concept deep dives	43
④ How the concepts connect	46
⑤ The build walkthrough	46
⑥ Understanding the design	47
⑦ The exercise twist	47
⑧ What to remember from this chapter	47
⑨ Coming up next	47
Expense Tracker	48
① What you are building	48
② The concepts you need	48
③ Concept deep dives	48
④ How the concepts connect	51
⑤ The build walkthrough	51
⑥ Understanding the design	52
⑦ The exercise twist	52
⑧ What to remember from this chapter	52
⑨ Coming up next	52
Quiz Game	53
① What you are building	53
② The concepts you need	53
③ Concept deep dives	53
④ How the concepts connect	56
⑤ The build walkthrough	56

⑥ Understanding the design	56
⑦ The exercise twist	56
⑧ What to remember from this chapter	56
⑨ Coming up next	57
Hangman	58
① What you are building	58
② The concepts you need	58
③ Concept deep dives	58
④ How the concepts connect	61
⑤ The build walkthrough	61
⑥ Understanding the design	61
⑦ The exercise twist	61
⑧ What to remember from this chapter	61
⑨ Coming up next	61
Rock Paper Scissors	62
① What you are building	62
② The concepts you need	62
③ Concept deep dives	62
④ How the concepts connect	64
⑤ The build walkthrough	65
⑥ Understanding the design	65
⑦ The exercise twist	65
⑧ What to remember from this chapter	65
⑨ Coming up next	65
Student Grade Manager	66
① What you are building	66
② The concepts you need	66
③ Concept deep dives	66
④ How the concepts connect	69
⑤ The build walkthrough	70
⑥ Understanding the design	70
⑦ The exercise twist	70
⑧ What to remember from this chapter	70
⑨ Coming up next	70
Library Book System	71
① What you are building	71
② The concepts you need	71
③ Concept deep dives	71
④ How the concepts connect	74
⑤ The build walkthrough	75
⑥ Understanding the design	75
⑦ The exercise twist	75
⑧ What to remember from this chapter	75
⑨ Coming up next	75
Personal Diary	76
① What you are building	76
② The concepts you need	76
③ Concept deep dives	76
④ How the concepts connect	79
⑤ The build walkthrough	79
⑥ Understanding the design	79
⑦ The exercise twist	79
⑧ What to remember from this chapter	80
⑨ Coming up next	80

CSV Data Analyzer	81
① What you are building	81
② The concepts you need	81
③ Concept deep dives	81
④ How the concepts connect	84
⑤ The build walkthrough	84
⑥ Understanding the design	84
⑦ The exercise twist	84
⑧ What to remember from this chapter	85
⑨ Coming up next	85
Markdown Note-Taking App	86
① What you are building	86
② The concepts you need	86
③ Concept deep dives	86
④ How the concepts connect	89
⑤ The build walkthrough	89
⑥ Understanding the design	89
⑦ The exercise twist	89
⑧ What to remember from this chapter	90
⑨ Coming up next	90
Password Manager	91
① What you are building	91
② The concepts you need	91
③ Concept deep dives	91
④ How the concepts connect	94
⑤ The build walkthrough	94
⑥ Understanding the design	94
⑦ The exercise twist	94
⑧ What to remember from this chapter	95
⑨ Coming up next	95
Stock Price Fetcher	96
① What you are building	96
② The concepts you need	96
③ Concept deep dives	96
④ How the concepts connect	99
⑤ The build walkthrough	99
⑥ Understanding the design	99
⑦ The exercise twist	99
⑧ What to remember from this chapter	100
⑨ Coming up next	100
Web Scraper	101
① What you are building	101
② The concepts you need	101
③ Concept deep dives	101
④ How the concepts connect	104
⑤ The build walkthrough	104
⑥ Understanding the design	104
⑦ The exercise twist	104
⑧ What to remember from this chapter	105
⑨ Coming up next	105
Unit Converter Suite	106
① What you are building	106
② The concepts you need	106
③ Concept deep dives	106
④ How the concepts connect	109

⑤ The build walkthrough	109
⑥ Understanding the design	109
⑦ The exercise twist	109
⑧ What to remember from this chapter	109
⑨ Coming up next	110
Flashcard App with Spaced Repetition	111
① What you are building	111
② The concepts you need	111
③ Concept deep dives	111
④ How the concepts connect	113
⑤ The build walkthrough	114
⑥ Understanding the design	114
⑦ The exercise twist	114
⑧ What to remember from this chapter	114
⑨ Coming up next	114
Pomodoro Timer	115
① What you are building	115
② The concepts you need	115
③ Concept deep dives	115
④ How the concepts connect	117
⑤ The build walkthrough	117
⑥ Understanding the design	117
⑦ The exercise twist	118
⑧ What to remember from this chapter	118
⑨ Coming up next	118
Personal Budget Tracker	119
① What you are building	119
② The concepts you need	119
③ Concept deep dives	119
④ How the concepts connect	121
⑤ The build walkthrough	121
⑥ Understanding the design	122
⑦ The exercise twist	122
⑧ What to remember from this chapter	122
⑨ Coming up next	122
Bash Script Collection	123
① What you are building	123
② The concepts you need	123
③ Concept deep dives	123
④ How the concepts connect	126
⑤ The build walkthrough	126
⑥ Understanding the design	126
⑦ The exercise twist	126
⑧ What to remember from this chapter	127
⑨ Coming up next — and Part 1 closing	127

How to Use This Guide

This is not a reference manual. It is a teaching companion. It exists to do one thing: when you sit down to build a project from your 300-Project curriculum, you read the chapter for that project, you close the book, and you build the project on your own — without looking things up.

Every chapter teaches exactly what one project needs. No more, no less. Each chapter stands completely alone, even if the same concept appears in an earlier chapter, because you will return to chapters out of order.

How this guide works alongside the curriculum

Open your curriculum tracker. Pick the next project. Find that chapter here.

Read the chapter once, slowly. Then close it. Then build. When you get stuck — and you will — come back and read the relevant deep dive again. That round trip is the whole point.

Do not read this guide cover to cover. Read it project by project. That is how it is designed.

The learning loop: read → understand → build → reflect

Every project follows the same four-beat rhythm.

- **Read.** Take in the chapter once, end to end. Do not type yet. Just understand.
- **Understand.** Re-read the deep dives for any concept that felt slippery. Try to predict what the code will do before reading the explanation.
- **Build.** Close the book. Open your editor. Write the project. Refer back only when fully stuck.
- **Reflect.** When the project runs, ask: which concept did I really understand, and which one did I just copy? Mark the latter for tomorrow.

This loop is more powerful than reading three chapters in a row. Trust it.

How to handle being stuck (the most important skill)

Being stuck is not a failure. It is a signal. It tells you the exact place where your understanding is not yet complete. The whole craft of programming is, in a sense, the craft of being productively stuck.

Here is a simple ladder. Climb it in order. Do not skip rungs.

1. **Read the error message.** Out loud. Every word. Most beginner errors announce themselves clearly.
2. **Print things.** Add `print()` statements to show the values of variables at each step. Half of all bugs become obvious the moment you see the data.
3. **Walk through by hand.** Take a small example and trace the program with a pencil. Write the value of each variable line by line.
4. **Re-read the deep dive.** Go back to the concept in this guide. Often the gap is in the very first paragraph.
5. **Sleep on it.** Genuinely. A bug that defeats you for two hours often dissolves in the first ten minutes of the next morning. This is real, and well-documented.
6. **Then, and only then, look it up.** Searching too early teaches you to outsource your thinking. Searching after honest effort teaches you to learn.

Using spaced repetition with this guide

You will forget things. This is normal and required. Memory consolidates by being lost and re-found, not by being held constantly.

Keep a small notebook (paper is best) with one line per concept that gave you trouble. Once a week, re-open three random chapters you finished more than seven days ago. Re-read just the **What to Remember** section. If you cannot recall how it works, re-read the deep dive.

That five-minute weekly habit doubles long-term retention. It is the single highest-leverage thing you can do as a learner.

Setting up your Python environment

Before Project 1, you need three things: Python installed, a code editor, and a terminal.

- 1. Install Python.** Go to python.org and download the latest 3.x version. On install, check the box that says *Add Python to PATH*.
- 2. Install an editor.** Visual Studio Code is the easiest. After installing, open it and install the Python extension from the Extensions tab.
- 3. Use the terminal.** On Windows use Git Bash or PowerShell. On macOS or Linux, the built-in Terminal is fine. Open it and type:

```
python3 --version
```

If you see something like *Python 3.12.0*, you are ready.

- 4. Make a workspace folder.** Pick a folder on your computer, for example `~/curriculum`. Inside it, you will create one subfolder per project: `project_01/`, `project_02/`, and so on.

- 5. Run your first file.** Inside `project_01/` create a file called `hello.py` with one line: `print("hi")`. Then in the terminal, from that folder, type `python3 hello.py`. If you see *hi*, your environment works.

How to read code (actively, with a pencil)

Code is not prose. You cannot just let your eyes drift over it. You must *trace* it — pretend you are the computer, and write down what happens at each step.

Here is a tiny example. Trace it before reading the explanation.

```
x = 5
y = 3
x = x + y
y = x - y
x = x - y
print(x, y)
```

Line 1: `x` becomes 5. Line 2: `y` becomes 3. Line 3: `x` becomes $5 + 3 = 8$. Line 4: `y` becomes $8 - 3 = 5$. Line 5: `x` becomes $8 - 5 = 3$. Output: `3 5`. The two variables swapped — without a third helper variable.

If you guessed correctly before reading: good. If you did not: even better, because you just found a real gap and you are about to learn from it. *Tracing on paper is the single most underrated skill in learning to code.*

The right mindset

Two short truths, taped above your desk:

- **Confusion is the signal that you are learning.** When you understand everything, you are not at your edge. The edge is uncomfortable. Stay there.
- **The brilliant programmers you admire all started where you are.** They simply put in more hours of being confused. That is the entire trick.

You picked an ambitious curriculum: 300 projects, from a CLI greeter all the way to your own AI mission companion. This guide is the friend who sits next to you for that whole journey. When you get stuck on any of those 300, you come here, we work through it together, and you go back to your editor stronger.

Let us begin.

PART 1

Python Foundations

Covers Projects 1–25.

Part 1 is the bedrock. Everything else — the physics simulators, the AI models, the satellite trackers — will be written in this language and built out of the patterns you learn here.

Twenty-five projects, climbing carefully from a single `print` statement to your first object-oriented system, your first API call, and your first web scraper. By the end of Part 1 you are a working Python programmer.

Read one chapter, build the matching project, then come back for the next.

Hello World CLI

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A small command-line program that asks the user for four pieces of information about themselves — their name, a favourite colour, what they do, and a dream they are chasing — and then displays those facts back as a clean, multi-line personal greeting. Running it feels like talking to a friendly little robot that takes a genuine interest in the human at the keyboard. It is the smallest possible program with a real conversation in it, and once it works you will have already touched almost every fundamental thing Python does.

② The concepts you need

- **print().** The command that puts text on the screen.
- **input().** The command that pauses, waits for the user to type something, and returns what they typed.
- **Variables.** Named boxes that hold a value so you can reuse it.
- **Strings.** Pieces of text. The data type of everything that comes back from `input()`.
- **f-strings.** A way to drop the value of a variable directly inside a piece of text.
- **.strip() and .title().** Two small surgical tools that clean up messy text.
- **The or trick for empty input.** A tiny pattern for defaulting an empty string to something sensible.
- **Multi-line output.** Putting a newline inside text so one `print()` shows multiple lines.

③ Concept deep dives

print() — speaking to the screen

The big idea. `print()` is how your program shows anything to a human. You hand it some text in parentheses, and the text appears in the terminal.

This is like when... you press a key on a piano and a sound comes out. The key is the function call. The sound is what appears on screen.

The explanation.

Everything inside the parentheses is called an **argument**. The text itself must be wrapped in quotes — either single (`'hi'`) or double (`"hi"`). Both work; just be consistent.

After it prints, `print()` automatically jumps to the next line. That is why two `print()` calls show up on two separate rows.

You can hand `print()` more than one thing by separating items with commas. It will put a single space between them by default.

The code example.

```
print("Hello!")           # writes one line, then a newline
print("My name is", "SOCA") # comma -> a space joins the items
print()                  # an empty call prints a blank line
print("That blank line was on purpose.")
```

Output:

```
Hello!
My name is SOCA

That blank line was on purpose.
```

Common mistakes.

- Many beginners forget the quotes and write `print(Hello)`. Python then looks for a *variable* called `Hello` and crashes when it does not find one. Always quote text.
- Many beginners write `print 'hi'` (no parentheses). That was Python 2 syntax; in modern Python the parentheses are required.

Quick Check. What is printed by `print("a"); print("b")`?

Answer: Two lines: `a` on the first, `b` on the second. Each call ends with its own newline.

`input()` — listening for an answer

The big idea. `input()` shows a prompt to the user, waits patiently until they type something and press Enter, then hands back whatever they typed as a piece of text.

This is like when... you call a friend's name and wait. Nothing happens until they answer. The function call is the shout; the value you get back is their reply.

The explanation.

The text you pass *inside* the parentheses is shown to the user as a hint of what to type. It is called the **prompt**.

Whatever the user types comes back as a **string** — a piece of text — even if they typed digits. `input("Age: ")` for the user typing `17` gives you back the text `"17"`, not the number `17`.

Your program halts and does nothing else until the user presses Enter. This is normal.

The code example.

```
name = input("What is your name? ") # show prompt, wait, capture reply
print("Hi,", name)                 # use what we captured
```

Output:

```
What is your name? Stephane
Hi, Stephane
```

Common mistakes.

- Many beginners forget to capture the result. Writing just `input("Name? ")` with no `name =` in front means the answer is collected and immediately thrown away.
- Many beginners assume the result is a number. If you need a number for arithmetic, you must convert it explicitly with `int(...)` or `float(...)`.

Quick Check. Why does `age = input("Age: "); print(age + 1)` crash?

Answer: Because `age` is a string (e.g. `"17"`), and you cannot add a number to a string. You would need `int(age) + 1`.

Variables — named boxes

The big idea. A variable is a name you stick on a value so you can refer to that value later by name instead of by its content.

This is like when... you label a jar in your kitchen *sugar*. From then on, when you want sugar, you reach for the jar with that label. You do not have to remember which shelf or what colour the lid is.

The explanation.

You create a variable by writing `name = value`. The left side is the label; the right side is the thing being labelled. The equals sign here means *assign*, not *compare*.

You can overwrite a variable by assigning to the same name again. The old value is replaced.

Variable names should be short but meaningful. `n` is fine for a tiny example; `name` or `favourite_colour` is better in real code. Use lowercase with underscores between words — that is the Python convention.

The code example.

```
favourite_colour = "blue"      # create a variable
print(favourite_colour)        # use it
favourite_colour = "green"     # overwrite it
print(favourite_colour)        # the new value
```

Output:

```
blue
green
```

The diagram.

```
Step 1: favourite_colour ---> [ "blue" ]
Step 3: favourite_colour ---> [ "green" ] (old value is replaced)
```

A variable is a label pointing at a value in memory. Reassign, and the label points at a new value.

Common mistakes.

- Many beginners write `5 = x`. The variable always goes on the LEFT.
- Many beginners use a name and forget to set it first. Python crashes with *NameError*. You must *create* the variable (assign to it) before you can *use* it.

Quick Check. After `a = 1`; `b = 2`; `a = b`, what does `print(a, b)` show?

Answer: 2 2. The last assignment made `a` point at the same value as `b`.

f-strings — text with variables baked in

The big idea. An **f-string** is a normal string with an **f** in front of the opening quote, which lets you drop a variable straight inside the text by wrapping it in curly braces.

This is like when... you write a personalised letter that says “Dear {name}, ...”. The template stays the same; only the bit inside the braces changes for each reader.

The explanation.

Without f-strings, mixing variables into text is clunky — you have to glue pieces together with commas or plus signs. f-strings make it readable and short.

Anything Python understands can go inside the braces — a variable, an expression, even a small calculation. `f"Total: {a + b}"` works.

You can put a newline character (`\n`) inside an f-string to break the text onto a new line.

The code example.

```
name = "Stephane"
city = "Kigali"
greeting = f"Hello, {name}! Welcome from {city}."
print(greeting)
print(f"In 2 years you will be {17 + 2} years old.")
```

Output:

```
Hello, Stephane! Welcome from Kigali.
In 2 years you will be 19 years old.
```

Common mistakes.

- Many beginners forget the **f** at the start. Without it, `"Hello, {name}"` prints the literal text including the braces.
- Many beginners put quotes inside the braces (`{"name"}`). Inside the braces you write the variable, not text. No quotes.

Quick Check. What does `x = 3; print(f"x is {x*x}")` print?

Answer: `x is 9`. The expression inside the braces is evaluated and the result is inserted into the string.

`.strip()` and `.title()` — cleaning up messy input

The big idea. `.strip()` removes spaces from the front and back of a string. `.title()` capitalises the first letter of each word.

This is like when... you receive a handwritten form and tidy it up before filing — trimming the messy margins, fixing the capitalisation of the names — so every record in the cabinet looks neat.

The explanation.

When a user types " stephane ", you really want "Stephane". `.strip()` handles the spaces; `.title()` handles the case.

These are called **string methods**. You call them by writing the variable, a dot, the method name, and parentheses: `name.strip()`.

They return a *new* string — the original is not changed. To keep the cleaned version, you must assign it back: `name = name.strip()`.

The code example.

```
raw = "   stephane MBAH   "
clean = raw.strip()           # "stephane MBAH"
pretty = clean.title()        # "Stephane Mbah"
# Often chained in one line:
pretty = raw.strip().title()
print(repr(pretty))           # show with quotes to prove it
```

Output:

```
'Stephane Mbah'
```

Common mistakes.

- Many beginners call `raw.strip()` and expect `raw` itself to change. It does not. The method returns a cleaned copy; you must catch it.
- Many beginners reach for `.title()` on text where it does not belong, like email addresses, and end up with `Stephane@Example.Com`. Use it only on names and titles.

Quick Check. What does `" hello world ".strip().title()` evaluate to?

Answer: `'Hello World'`. The strip removes the spaces; the title capitalises both words.

Handling empty input with the `or` trick

The big idea. If the user just hits Enter without typing anything, `input()` returns the empty string `""`. You should not let that empty string spread through your program.

This is like when... a form has a default value already typed in light grey. If the user does not overwrite it, you use the default. If they do, you use what they wrote.

The explanation.

In Python, an empty string is considered **falsy** — in a boolean context it counts as `False`. A non-empty string is **truthy**.

The expression `a or b` evaluates to `a` if `a` is **truthy**, otherwise to `b`. So `name or "Anonymous"` means “use `name`, or fall back to `Anonymous` if `name` is empty”.

This is a one-line, idiomatic way to give a default. You will see it in real Python code constantly.

The code example.

```
name = input("Name: ").strip() # clean first
name = name or "Anonymous"     # default if empty
print(f"Hi, {name}!")
```

Output:

```
Name:
Hi, Anonymous!
```

Common mistakes.

- Many beginners write a long `if name == "": name = "Anonymous"`. That works, but the `or` form is shorter and idiomatic.
- Many beginners forget to `.strip()` first. A user who types only spaces is not really providing input — `.strip()` turns those spaces into the empty string so the default kicks in.

Quick Check. What does `x = ""; print(x or "default")` print?

Answer: `default`. The empty string is falsy, so the `or` falls through to the right side.

Multi-line output — one `print`, several lines

The big idea. Inside a string, the two characters `\n` are a special signal that means “break here and start a new line”. You can use them inside a single `print()` call to produce several lines of output at once.

This is like when... an old typewriter, where you press the *return* lever to roll the paper down one line. `\n` is the same lever, written in code.

The explanation.

`\n` is called the **newline character**. It is one character, even though you typed two symbols. The backslash is a signal to Python that the next character has a special meaning.

Inside an f-string, `\n` still means newline. So you can build a multi-line greeting with variables baked in, in a single f-string.

If you want a really clean multi-line block, triple-quoted strings (`"""..."""`) let you press Enter inside the string itself. Each press of Enter becomes a `\n` automatically.

The code example.

```
name = "Stephane"
print(f"Hello, {name}!\nWelcome to the curriculum.\nLet us begin.")
```

Output:

```
Hello, Stephane!
Welcome to the curriculum.
Let us begin.
```

Common mistakes.

- Many beginners write `/n` (forward slash) instead of `\n` (backslash). Only the backslash version works.
- Many beginners try to use `\n` outside a string — for example, on its own as a separator. It is meaningful only inside text.

Quick Check. How many lines does `print("a\nb\nc")` output?

Answer: Three. The two `\n` characters are line breaks, and `print()` adds one more at the end.

④ How the concepts connect

Here is the whole program in one breath, told as a story. Four `input()` calls collect the user’s answers and store each one in a variable. After each call we apply `.strip()` to clean up stray spaces, and we use the `or` trick to substitute a sensible default if the user pressed Enter without typing. Then a single `print()` with a multi-line f-string — with `\n` as line breaks and curly braces for the variables — assembles all four facts into a single, formatted greeting. That is the whole machine.

⑤ The build walkthrough

This is where you put the book down and write. The chapter has shown you every concept you need; what follows is the *plan*, in prose, so you can translate it into code yourself. If you get stuck on syntax, the deep dives above are your reference. Do not look at someone else's solution before you have honestly tried.

Step 1 · Collect the four facts. Use four `input()` calls in sequence, one for the name, one for the favourite colour, one for the job, one for the dream. Store each answer in its own variable. The prompts should be friendly, one-line questions.

Step 2 · Clean each input. For each variable, chain `.strip()` to remove stray spaces. For the name, also chain `.title()` so capitalisation looks correct. Then apply the `or` trick to substitute a sensible default if the user pressed Enter without typing — something like "Anonymous" for an empty name, "a beautiful colour" for an empty colour, and so on. Each cleaning happens in a single line, by reassigning the variable to its cleaned form.

Step 3 · Build the multi-line greeting. Make a `border` variable using string multiplication — for example, an equals sign repeated 32 times. Then use one `print()` call with a multi-line f-string. The f-string should include `\n` to break onto new lines, and it should drop each of the four cleaned variables in with curly braces. The visual idea is a “card” with the border at the top and bottom and the personalised text in the middle.

When the user types in proper answers, the output is a clean four-line greeting framed by two bars. When the user just hits Enter through every question, the output still looks correct — because every variable was given a fallback. Run it both ways before you call it done.

⑥ Understanding the design

Three habits to internalise from this project. First, *clean and default in the same line*: `name.strip().title() or "Anonymous"` is the smallest amount of code that handles both messy and missing input. Second, *use a single multi-line f-string instead of many print calls* — it keeps the visual shape of the output close to the code that produces it. Third, *store repeated values in a variable*: the border is created once and reused at the top and bottom of the card, so changing the card's width is a one-character edit. These are habits, not rules. You will reuse them in every project from here on.

⑦ The exercise twist

Build a multilingual greeter (English, French, Kinyarwanda) — ask the user for a language first.

What new concept does this need? A simple branch: an `if / elif / else` chain that picks which greeting template to use, based on the language the user typed. You have not formally seen `if` yet (it arrives in Chapter 2), but the shape is intuitive: *if* the language is French, use the French template; *else if* it is Kinyarwanda, use that one; otherwise default to English.

How does it extend what you built? The four inputs stay the same. Only the final f-string changes — you have three versions of it now, one per language. The cleanest approach is to store the three templates in a `dict` keyed by language code, then look up the right one. (Dictionaries arrive properly in Chapter 6, but a sneak peek hurts no one.)

Hint for the hardest part. Normalise the language input before checking it: `lang = input(...).strip().lower()`. Otherwise "French", "french" and "FRENCH" will all be treated as different answers.

⑧ What to remember from this chapter

- `print()` writes to the screen and always ends with a newline.
- `input()` shows a prompt, waits for Enter, and gives back a string — always a string.
- Variables are labels: `name = value`. The label is on the left.
- f-strings let you drop variables straight into text with curly braces, and `\n` breaks the text onto a new line.
- `.strip()` cleans whitespace, `.title()` fixes case, and `name or "default"` gives a fallback when the input is empty.

⑨ Coming up next

In the next chapter you will turn the patterns you just used into reusable **functions**, so the same six temperature conversions do not require six copies of nearly-identical code.

Temperature Converter

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line tool that converts a temperature value from one unit to another — Celsius, Fahrenheit, or Kelvin — using the physics formulas you may know from school. The user types a number and two unit letters, and the program reports the converted value rounded to two decimals. It keeps running in a loop so the user can do a long session of conversions, and it refuses to crash even if they type “hello” instead of a number. When it works, it feels like a quiet, dependable lab assistant: ask, answer, repeat.

② The concepts you need

- **Functions.** A reusable mini-program with a name. Defined with `def`.
- **Parameters and return values.** The inputs a function takes and the value it hands back.
- **Type conversion: `float()`.** Turning a piece of text like “100” into the actual number 100.0.
- **The `.2f` format spec.** Rounding a float to exactly two decimal places when displaying it.
- **`if` / `elif` / `else`.** Choosing one branch out of several based on a condition.
- **`while` loops.** Repeating a block until a condition becomes False.
- **`try` / `except`.** Trying something risky and catching the explosion if it fails.

③ Concept deep dives

Functions — named, reusable mini-programs

The big idea. A **function** is a block of code with a name. You define it once, and after that you can run it as many times as you want by “calling” its name.

This is like when... you teach a friend a recipe once. After that, you just say “make me your special pancakes,” and they execute the whole recipe.

The explanation.

You define a function with the keyword `def`, followed by a name, parentheses (for any inputs), and a colon. The body is the indented block underneath. Functions are the main tool for organising bigger programs — instead of one giant flat script, you break the work into named pieces.

Defining a function does *not* run it. You must call it — write its name with parentheses — for the code inside to execute.

The code example.

```
def greet():           # define a function named greet
    print("Hi!")       # body: anything indented under the def

greet()                # CALL it – now it actually runs
greet()                # call it again – runs again
```

Common mistakes.

- Many beginners define a function and never call it — then wonder why nothing happens. Calling is a separate step.
- Many beginners forget the colon at the end of the `def` line, or forget to indent the body. Both are required.

Quick Check. What does `def f(): print("a")` print, on its own, if nothing else runs?

Answer: Nothing. Defining a function does not call it.

Parameters and return values

The big idea. A function can take inputs (**parameters**) and hand back an output (a **return value**). That is how you make functions reusable for many different inputs.

This is like when... you put coins in a coffee machine and get coffee out. Same machine, different inputs, different outputs.

The explanation.

Parameters go inside the parentheses of the **def** line. When you call the function, you pass actual values into those slots. Inside the function, use **return** to send a value back — whoever called the function receives that value.

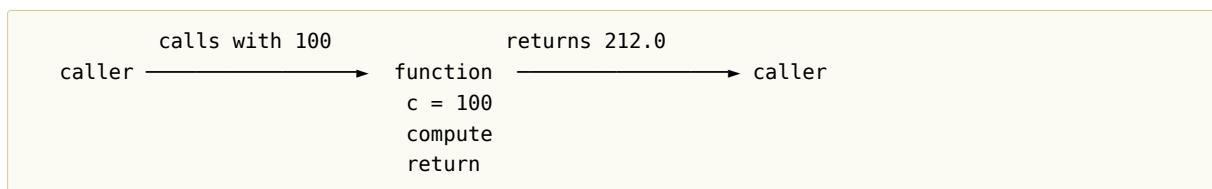
A function without an explicit **return** still returns something — the special value **None**. So a function that only prints does its work but returns nothing useful for further computation.

The code example.

```
def celsius_to_fahrenheit(c):    # c is the parameter
    f = (c * 9 / 5) + 32        # do the work
    return f                    # hand back the result

result = celsius_to_fahrenheit(100) # pass 100 in, catch the answer
print(result)                       # 212.0
```

The diagram.



A call passes values IN; a return passes one value BACK.

Common mistakes.

- Many beginners use **print** when they mean **return**. Printing shows on screen but does not give anything back to the caller.
- Many beginners forget to capture the return: `celsius_to_fahrenheit(100)` with no `result =` in front discards the answer.

Quick Check. What is `x` after `x = celsius_to_fahrenheit(0)`?

Answer: `32.0`. The formula gives $(0 \times 9/5) + 32 = 32$.

Type conversion with `float()`

The big idea. Text from `input()` cannot be added or multiplied as a number. `float()` converts a numeric string into an actual decimal number.

This is like when... you receive an address on paper and transcribe it into your GPS app. Same information, different format.

The explanation.

There are three main number-conversion functions: `int()` for whole numbers, `float()` for decimals, and `str()` to go the other way. If the string does not look like a number — the user typed "hello" — the conversion raises a `ValueError`. Always convert input as soon as you receive it, so bugs surface near the source.

The code example.

```
raw = input("Enter value: ")    # raw is always a string
value = float(raw)              # convert to a real number
print(value + 1)                # works because value is a float
```

Common mistakes.

- Many beginners write `float(input(...))` without thinking about what happens if the user types text. That crashes. Wrap it in `try/except`.
- Many beginners use `int()` on text with a decimal point (`int("3.5")` crashes). Use `float()` when decimals are possible.

Quick Check. What is `float("3.14") + float("2.86")`?

Answer: 6.0. Both strings convert to floats; the addition is real arithmetic.

The .2f format spec — rounded display

The big idea. Inside an f-string, a colon plus format spec controls how a number is displayed. `:.2f` means “show this as a float with exactly two decimal places.”

This is like when... you write “1.25 cups” on a shopping list, not “1.24999999 cups.”

The explanation.

The colon inside the curly braces separates the value from the format spec. `{x:.2f}` means “the value x, formatted with 2 decimal places.” This is a display tool — the variable itself still holds full precision. Other handy specs: `:`, adds thousands separators, and `>10` right-aligns in a width-10 column.

The code example.

```
pi = 3.141592653589793
print(f"pi rounded      : {pi:.2f}")      # 3.14
print(f"pi to four dp   : {pi:.4f}")      # 3.1416
print(f"big number      : {1234567:,}")    # 1,234,567
```

Common mistakes.

- Many beginners forget the colon and write `{pi.2f}`. Python then looks for an attribute called `2f` on `pi` and crashes.
- Many beginners assume `:.2f` changes the variable. It does not — only the display.

Quick Check. What does `print(f"{1/3:.3f}")` print?

Answer: 0.333. Three decimal places, rounded.

if / elif / else — picking one branch

The big idea. An `if` statement chooses what to do based on a condition. `elif` (short for “else if”) tests additional conditions; `else` handles “none of the above.”

This is like when... a fork in a road. Based on a sign, you take one path. If two more forks appear, you check each in turn. If none apply, you take the default path home.

The explanation.

A condition is anything that evaluates to True or False. Each branch ends with a colon and an indented body. Only **one** branch runs — the first whose condition is True. After that, Python skips the rest.

Be careful: `=` is for assignment, `==` is for comparison. They look almost identical and trip everyone up at first.

The code example.

```
score = 87
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(grade)                                # B
```

Common mistakes.

- Many beginners write `if score = 90`. Python rejects it — a single equals sign is not allowed in a condition.
- Many beginners write `else condition:`. The keyword for “another condition” is `elif`, not `else`.

Quick Check. If `x = 5`, what runs from `if x > 10: ... elif x > 0: ... else: ...`?

Answer: The `elif` branch, because 5 is not greater than 10 but is greater than 0.

while loops — repeat until False

The big idea. A `while` loop runs its body over and over for as long as a condition stays `True`. The moment the condition becomes `False`, the loop stops.

This is like when... a child on a swing keeps asking “higher? higher?” — you keep pushing as long as the answer is yes. The first time they say no, you stop.

The explanation.

The header is `while condition:` followed by an indented body. Python checks the condition before each iteration. Something inside the body **must** eventually make the condition become `False` — otherwise you have an infinite loop, and the program runs forever.

A classic pattern: ask the user something, do the work, then ask “again?” If they say no, set the loop variable so the condition becomes `False` and the loop ends naturally.

The code example.

```
again = "y"                                # start truthy so the loop enters
while again == "y":
    print("Doing work...")
    again = input("Again? (y/n) ").strip().lower()
print("Done.")
```

Common mistakes.

- Many beginners forget to update the loop variable inside the body — the condition stays `True` forever.
- Many beginners initialise the loop variable to something the condition does not accept — then the body never runs even once.

Quick Check. What stops an infinite loop while it is running?

Answer: Pressing `Ctrl+C` in the terminal.

try / except — catching the explosion

The big idea. Some operations can fail. `try / except` lets you attempt the risky thing and gracefully recover if it blows up, instead of crashing the whole program.

This is like when... a safety net under a trapeze artist. If the artist falls, the net catches them and the show continues.

The explanation.

Put the risky line(s) inside a `try:` block. Put the recovery code inside an `except SomeError:` block underneath. Python jumps to `except` only if something in the `try` actually went wrong. Always name the specific exception you expect — `except ValueError:` is good; a bare `except:` swallows every kind of error including real bugs.

The code example.

```
raw = input("Enter a number: ")
try:
    value = float(raw)                # this can fail
    print(f"You entered {value}.")
except ValueError:
    print("That was not a valid number.")
```

Common mistakes.

- Many beginners use `except:` without naming a specific error. This hides bugs.

- Many beginners wrap too much code in one `try:`. Keep the block small so the `except` deals with one cause.

Quick Check. If the user types 42, what does the example print?

Answer: You entered 42.0. The conversion succeeds, so the `except` block is skipped.

④ How the concepts connect

A `while` loop keeps the program asking until the user says quit. Inside the loop, `input()` collects the value and the two unit letters. A `try/except` wraps the `float()` call so non-numeric input does not crash the program. A small chain of `if/elif/else` picks the correct conversion **function** based on the unit pair, and that function does the formula and returns the answer. Finally, an f-string with `:.2f` displays the result to two decimals. Every concept in section ② shows up exactly once in this flow.

⑤ The build walkthrough

Step 1 · Write the six conversion functions. Define one function per direction: Celsius to Fahrenheit, Fahrenheit to Celsius, Celsius to Kelvin, Kelvin to Celsius, Fahrenheit to Kelvin, Kelvin to Fahrenheit. Each takes one parameter (the value), applies the formula, and returns the result. The formulas come from physics: $(c \times 9/5) + 32$ for $C \rightarrow F$, $(f - 32) \times 5/9$ for $F \rightarrow C$, $c + 273.15$ for $C \rightarrow K$, and so on. Keep each function to a single line where possible — they are pure formulas.

Step 2 · Build a dispatch table. Create a dictionary that maps a *pair* of unit letters (the source and destination, as a tuple like `("C", "F")`) to the function that performs that conversion. You will look up the right function by indexing the dictionary with the pair, rather than writing six `if` branches. Dictionaries arrive properly in Chapter 6, but the shape is intuitive.

Step 3 · Write the interactive loop. Use a `while True:` outer loop. Each iteration: read a line for the value, then check for a quit keyword and `break` if matched. Wrap the `float()` conversion in `try/except ValueError`, and use `continue` to ask again on bad input. Read two more lines for the source and destination units, normalising with `.strip().upper()`. If they are equal, no conversion is needed — print the value back. Otherwise, look up the pair in your dispatch table. If the lookup returns `None`, the pair was unknown — tell the user and `continue`. Otherwise, call the function with the value, capture the result, and print both the input and the result formatted with `:.2f`.

Step 4 · Defensive touches. Reject negative Kelvin values with a friendly message (absolute zero is the floor). Trim a trailing prompt newline for cleanliness. Print a brief welcome banner above the loop and a goodbye after it. These are small polish steps that turn a script into a tool.

⑥ Understanding the design

Three deliberate choices shape this program. First, *each formula is its own function* — small, named pieces are easier to read, easier to test, and easier to reuse later when, say, a chemistry project wants `celsius_to_kelvin`. Second, the *dispatch table replaces an if/elif ladder*; adding new units later (Rankine, Réaumur) is a matter of adding two functions and two dictionary entries, not rewriting conditional logic. Third, the `try/except` with `continue` makes the loop *survive bad input gracefully*. The program should never feel fragile to the human using it.

⑦ The exercise twist

Add Rankine and Réaumur scales, plus unit auto-detection (e.g. typing 100C in one go).

What new concept does this need? A small piece of string parsing: splitting "100C" into the number part and the unit part. Look at the last character — if it is a letter, that is the unit and the rest is the value. The rest is more of the same: define the new conversion functions and add their dictionary entries.

How does it extend what you built? The dispatch-table architecture pays off. You do not change the loop logic at all — only add new functions and new dictionary keys.

Hint for the hardest part. Rather than every-pair-to-every-pair (which grows quadratically), convert everything through a single base unit (Celsius is convenient). `any_to_any(value, src, dst)` then becomes: convert $src \rightarrow C$, then convert $C \rightarrow dst$. You only need two functions per new unit (to-C and from-C), not $2 \times N$.

⑧ What to remember from this chapter

- `def name(params): body` defines a function; `name(args)` calls it; `return` sends a value back.
- Input is always a string — convert with `float()` or `int()` before doing arithmetic.
- `{value:.2f}` inside an f-string rounds to two decimal places for display only.
- `if / elif / else` picks exactly one branch. Use `==` for comparison, not `=`.
- `try / except ValueError` wraps a risky conversion so bad input does not crash the program.

⑨ Coming up next

In the next chapter you use a `while` loop with `break` and a fresh tool — the `random` module — to build a guessing game where the program thinks of a number and you have to find it.

Number Guessing Game

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A little detective game. The computer secretly picks a whole number between 1 and 100, and you have to find it. After each guess the program tells you whether the secret is higher or lower than your guess. When you finally land on it, the program reports how many guesses you took and offers a new round. It refuses to crash when you type letters by mistake, and it gently corrects you when you guess outside the range. By the end of this chapter you will have your first real game loop — a pattern you will reuse in dozens of later projects.

② The concepts you need

- **Importing modules.** Bringing extra tools into your program with `import`.
- **`random.randint()`.** Asking Python to pick a random whole number from a range.
- **`while True` with `break`.** A loop that runs until something inside it decides to stop.
- **Counting with a variable.** Adding 1 to a counter each time something happens.
- **Comparison operators.** The symbols (`<`, `>`, `==`) that let an `if` compare values.
- **`try` / `except` for input validation.** Catching `ValueError` when a player types junk.
- **The play-again loop pattern.** An outer loop that wraps the whole game so the player can replay.

③ Concept deep dives

Importing modules

The big idea. Python comes with hundreds of bundled tools called **modules** — pre-written collections of functions for tasks like maths, random numbers, dates, files, networking. The keyword `import` says “I want to use the tools from this collection.”

This is like when... you walk into a workshop and open a specific drawer of tools. Before you open the drawer, you cannot reach what’s inside.

The explanation.

Place the `import` line at the very top of your file — by convention all imports go there so a reader can see what your program depends on. After importing, you reach into the module with dot notation: `random.randint(1, 100)` means “the `randint` function inside the `random` module.”

The code example.

```
import random          # open the 'random' drawer
n = random.randint(1, 6) # roll a die
print(n)               # e.g. 4
```

Common mistakes.

- Many beginners forget the import line and get `NameError: name 'random' is not defined`.
- Many beginners write `import random.randint` and expect to call it as `randint(...)`. Either `import random` (then `random.randint`) or `from random import randint` (then `randint`). The two forms do not mix.

Quick Check. If you write `from random import randint`, how do you call it?

Answer: Just `randint(1, 6)` — no `random.` prefix, because you imported the function directly.

random.randint(a, b) — a fair roll

The big idea. `random.randint(a, b)` picks a whole number uniformly at random from `a` up to and including `b`. Every value in that range is equally likely.

This is like when... you roll a fair die — each face equally likely, no predicting it, values spread evenly over many rolls.

The explanation.

Both endpoints are *inclusive*. `randint(1, 100)` can return 1 or 100. This is unlike `range(1, 100)`, where 100 is excluded — an inconsistency in Python that catches everyone out at least once.

Each call is independent. If `randint` just returned 47, the next call has the same chance of returning 47 again. Random does not mean “avoids repeats.”

The code example.

```
import random
for _ in range(5):                # do five rolls
    print(random.randint(1, 10), end=' ')
print()                          # newline at the end
```

Common mistakes.

- Many beginners reach for `random.random()` — which returns a float between 0 and 1 — and try to scale it themselves. `randint` already gives whole numbers.
- Many beginners think `randint(1, 100)` excludes 100. It does not.

Quick Check. What is the smallest and largest possible result of `random.randint(5, 10)`?

Answer: 5 and 10, both inclusive. Six possible values: 5, 6, 7, 8, 9, 10.

while True with break — loop until something inside stops it

The big idea. `while True:` is a loop whose condition is always true — so on its own it would run forever. `break` is the escape hatch: when reached, it immediately exits the surrounding loop.

This is like when... you sit at a table playing rounds of a card game. You keep playing “forever” — until the moment somebody says “I quit,” at which point everyone stands up.

The explanation.

This shape is useful when the condition for stopping is more naturally checked *in the middle* of the loop body rather than at the top. A guessing game stops when the player guesses correctly — and you do not know that until you have read the guess.

There is also `continue` — it skips the rest of the current iteration and jumps back to the top of the loop. Useful for “bad input, ask again” patterns.

The code example.

```
count = 0
while True:                # infinite shell
    count += 1              # count = count + 1
    if count == 3:
        break              # escape hatch
    print(f"count is {count}")
print("after the loop")
```

Common mistakes.

- Many beginners place `break` outside a loop and get a `SyntaxError`.
- Many beginners use `while True` without ever putting a `break` inside — the program runs forever; you must kill it with Ctrl+C.

Quick Check. What is the difference between `break` and `continue`?

Answer: `break` exits the loop immediately. `continue` skips the rest of just this iteration and goes back to the loop’s top.

Counting with a variable

The big idea. A counter is a variable that starts at 0 (or 1) and gets incremented each time something specific happens.

This is like when... you tally votes on a chalkboard — each new vote, add a tick.

The explanation.

The classic shape: declare `count = 0` *before* the loop. Inside the loop, write `count += 1` at the moment the thing you are counting happens. `count += 1` is shorthand for `count = count + 1`. Counters are independent of any loop variable — `for i in range(10):` already has an `i`, but if you only want to count *some* iterations, you need your own counter.

The code example.

```
attempts = 0
for guess in [50, 75, 87, 87]:      # imagine four guesses
    attempts += 1
    print(f"attempt {attempts}: guessed {guess}")
```

Common mistakes.

- Many beginners declare the counter *inside* the loop — it gets reset to 0 every iteration and never goes above 1.
- Many beginners forget to declare it at all and get `NameError` on the first `+=`.

Quick Check. What does `x = 5; x += 3` leave `x` equal to?

Answer: 8. `+=` adds the right side and stores the result back.

Comparison operators

The big idea. Comparison operators take two values and produce a boolean. They are the language an `if` speaks.

This is like when... you compare two stones on a scale — the result is one of three: heavier, lighter, or balanced.

The explanation.

The six operators: `==` (equal), `!=` (not equal), `<` (less), `>` (greater), `<=` (less or equal), `>=` (greater or equal). `==` is two equals signs because one is already taken by assignment. Mixing them up is the single most common syntax bug among beginners.

Comparisons combine with `and`, `or`, and `not`. Python also allows chaining: `1 <= x <= 100` reads naturally and works exactly as it looks.

The code example.

```
guess = 47
secret = 50
print(guess == secret)      # False
print(guess < secret)       # True
print(1 <= guess <= 100)    # True – chained
```

Common mistakes.

- Many beginners write `if guess = secret:` and get a syntax error. Use `==` for comparison.
- Many beginners write `if 1 <= guess and guess <= 100:` when the chained `1 <= guess <= 100` is shorter and clearer.

Quick Check. Evaluate `3 != 4` and `5 > 4`.

Answer: True. Both sides are True, so the `and` is True.

try / except for input validation

The big idea. When the user types text where you expect a number, calling `int(text)` blows up with a `ValueError`. Wrap it in `try / except ValueError` and the program survives.

This is like when... you ask a friend for a coin. If they hand you a button, you politely give it back and ask again. You do not stop the whole party.

The explanation.

You met `try / except` in Chapter 2 with `float`. Here it is the same shape with `int`. Pair `except` with `continue` inside a loop to ask again on bad input — one of the most reused patterns in interactive Python.

The code example.

```
while True:
    raw = input("Number: ")
    try:
        n = int(raw)
    except ValueError:
        print("That is not a whole number.")
        continue          # back to top of loop
    print(f"OK, got {n}")
    break
```

Common mistakes.

- Many beginners catch `except Exception:` — which hides bugs alongside user mistakes. Be specific.
- Many beginners forget `continue` after the friendly message, and the loop body runs with a value that was never assigned.

Quick Check. Why is `except ValueError:` better than `except:`?

Answer: A bare `except` catches every kind of error, including bugs in your own code. The narrow version only catches the one you expected.

The play-again loop pattern

The big idea. When a game ends naturally and you want to offer a restart, the cleanest shape is *two* loops — an outer one for “again?” and an inner one for the round itself.

This is like when... a board game ends in a winner, then everyone is asked “shall we play again?” before resetting the board.

The explanation.

The inner loop runs one full game and exits cleanly when a winning condition is met. The outer loop wraps the whole thing and only exits when the user says “no, I’m done.” Reset every per-round variable (the secret number, the attempt counter) at the top of the outer loop. Forgetting to reset is a common source of subtle bugs.

The code example.

```
while True:                                # outer: replay?
    print("--- new round ---")
    # ... play one round here ...
    again = input("Play again? (y/n) ").strip().lower()
    if again not in ("y", "yes"):
        break
```

Common mistakes.

- Many beginners forget to reset per-round variables — the new game uses old state.
- Many beginners check `if again == "y":` with no normalisation, then are baffled when “Y ” (with a space, or capital) does not work.

Quick Check. Why use two loops instead of one big loop?

Answer: The two stopping conditions are different. The inner ends when the round is won; the outer ends when the player declines to replay. Splitting them keeps each loop’s job clear.

④ How the concepts connect

An outer `while True` is the play-again loop. At the top of each round, `random.randint(1, 100)` picks a fresh secret and a counter resets to zero. An inner `while True` reads a guess each iteration. `try / except ValueError` filters non-numeric junk; a range check catches out-of-bounds guesses; the counter increments only on valid guesses. An `if / elif / else` chain compares the guess to the secret and prints “too high,” “too low,” or — on a match — reports the attempt count and `breaks` out of the inner loop. Then the outer loop asks “again?”

⑤ The build walkthrough

Step 1 · The outer loop. Open a `while True`: that wraps the entire game. At its top, pick a secret with `random.randint(1, 100)` and initialise an attempt counter to zero. After all the game logic, ask “play again?” and `break` out of this outer loop if the answer is not yes.

Step 2 · The inner loop. Inside the outer loop, open a second `while True`:. Each iteration: prompt for a guess and clean the input with `.strip()`.

Step 3 · Validate the input. Wrap an `int()` conversion in `try / except ValueError`. If conversion fails, print a friendly message and `continue` so the user can try again. If it succeeds, check the range with a chained comparison `1 <= guess <= 100` — if the guess is outside, print a warning and `continue` (do not count this guess).

Step 4 · Count and compare. Increment the attempt counter. Then an `if / elif / else` chain: if the guess is less than the secret, print “Too low”; if greater, print “Too high”; otherwise it is correct — print a congratulatory message including the attempt count, then `break` out of the inner loop.

Step 5 · Polish. Use the small word/words trick to print “1 attempt” vs “2 attempts” correctly. Add a welcome banner and a goodbye line outside the outer loop.

⑥ Understanding the design

Three design choices repay close reading. First, the input validation is a *two-stage filter* with `continue`: the program decides whether the input is even a number, then whether it is in range, before it counts as a real guess. Second, `attempts += 1` sits *after* the validation but *before* the comparison — so a player who types 200 four times does not get charged for four guesses. Third, two loops with completely different jobs — one round versus many rounds — are written as two separate `while True` blocks rather than trying to express both stopping conditions in one. When two ideas are this different, two loops is the honest shape.

⑦ The exercise twist

AI mode: the computer guesses YOUR number using binary search.

What new concept does this need? **Binary search** — if you know the answer is in a range, you can find it in at most about $\log_2$ of the range size guesses by always picking the midpoint, then halving the range based on the feedback. For 1–100, that is at most 7 guesses, ever.

How does it extend what you built? Swap roles. You pick a secret in your head; the computer guesses. Maintain two variables `low = 1` and `high = 100`. Each turn the computer guesses `(low + high) // 2`, asks you whether that is too low, too high, or correct, and updates `low` or `high` accordingly.

Hint for the hardest part. Defend against impossibility. If `low > high`, the user has contradicted themselves (maybe said “too high” when they meant “too low”). Detect that case and call them out politely. You have just written your first piece of self-aware code.

⑧ What to remember from this chapter

- `import random` at the top, then `random.randint(1, 100)` for a uniform whole number in 1..100 inclusive.
- `while True`: with a `break` inside is the natural shape when the stopping condition is in the middle of the body.
- Initialise counters *before* the loop. Inside the loop, `count += 1` only at the moment you want to count.
- Use `try / except ValueError + continue` to validate numeric input without crashing.

- For replayable interactions, wrap everything in an outer loop and break only on a clear “no.”

⑨ Coming up next

In the next chapter you will use a **list** — a growing collection of values — to remember every calculation a user performs, and you will meet special commands that dispatch to different actions inside the same loop.

Calculator with History

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line calculator that evaluates whatever maths expression you type — $5 + 3$, $(100 - 1) / 3$, $2 ** 10$ — and prints the result. It *remembers* every calculation in a list, and a few command words let you peek at the history, wipe it, or quit. Division by zero, typos, and other expression errors no longer crash it — they get caught and reported. By the end you will have a tool that feels surprisingly like the calculator app on a phone, plus your first taste of how a “command-driven” program is shaped.

② The concepts you need

- **`eval()`** — and its risks. A built-in function that takes a string of Python and runs it as code.
- **Lists.** An ordered, growable collection of items.
- **`list.append()`**. The standard way to add one more item to the end of a list.
- **`enumerate()`**. Walking through items while also having a counter, for numbered display.
- **Catching multiple exceptions.** Reacting differently to `ZeroDivisionError`, `SyntaxError`, and others.
- **Command dispatch.** Inspecting input and routing to different actions depending on a keyword.

③ Concept deep dives

`eval()` — running text as Python code

The big idea. `eval(s)` takes a string `s`, treats it as a Python expression, runs it, and returns the result.

This is like when... you read out a recipe to a chef and they immediately cook it. The text turns directly into action.

The explanation.

For a calculator this is wonderfully convenient: a user types `"2 + 3 * 4"`, you call `eval`, and Python applies its real operator-precedence rules and returns 14. No expression parser to write.

`eval` is dangerous. Whatever the user types, Python will run. If they paste `__import__('os').system('rm -rf /')`, `eval` cheerfully executes it. That is fine in a learning project that only you run; it is catastrophic in any program exposed to other people. Production calculators do not use `eval` — they write a small expression parser or use libraries that only allow safe maths. For now, treat `eval` as a knife you handle carefully and never expose to outside input.

The code example.

```
expr = "2 + 3 * 4"           # text
print(eval(expr))           # 14
print(eval("(5 + 3) ** 2")) # 64
```

Common mistakes.

- Many beginners use `eval` on input from outside their machine (a web form, a chat message). That is a remote-code-execution vulnerability — do not.
- Many beginners use `eval` when they really want `int()` or `float()` — much narrower tools that just turn a string into a number.

Quick Check. What does `eval("10 % 3")` return?

Answer: 1. The `%` operator is the remainder operator; $10 \div 3$ leaves remainder 1.

Lists — ordered, growable collections

The big idea. A **list** is a single variable that holds many values in order. You write it with square brackets and commas: `[10, 20, 30]`.

This is like when... a numbered shopping list on a notepad. Items have an order. You can add to the end, remove items, look at the third item, count them, or read through them top to bottom.

The explanation.

Lists are **indexed from zero**: `nums[0]` is the first item, `nums[1]` the second, and so on. Negative indices count from the end: `nums[-1]` is the last item. Lists are **mutable** — they can be modified in place. This contrasts with strings, which are immutable.

The code example.

```
history = []                # an empty list
history.append("5 + 3 = 8") # add to the end
history.append("10 * 2 = 20")
print(history[0])           # first item
print(history[-1])          # last item
print(len(history))         # how many items
```

The diagram.

index:	0	1
history:	"5+3=8"	"10*2=20"

len(history) == 2
history[-1] is the same as history[len(history)-1]

A list is laid out in order; each slot has a positive index from the front and a negative index from the end.

Common mistakes.

- Many beginners write `nums[1]` expecting the first item. The first item is at index 0.
- Many beginners ask for `nums[5]` on a list with three items and get `IndexError`. Check `len(nums)` first when in doubt.

Quick Check. What does `[10, 20, 30][-2]` evaluate to?

Answer: 20. The second-from-last item is the middle element.

list.append() — growing a list by one

The big idea. `list.append(item)` adds `item` to the end of the list. The list grows by one element. It returns nothing — the change is in place.

This is like when... you add a new entry at the bottom of a written log. The log itself is updated; you do not get a new log.

The explanation.

The classic shape: create an empty list *before* a loop, then `.append()` something on each iteration. After the loop, the list contains all the items collected. Because `append` returns `None`, the line `history = history.append(x)` is a bug — `history` becomes `None` and you have just deleted your list. Always write `history.append(x)` alone.

The code example.

```
nums = []                # start empty
for i in range(5):       # loop 5 times
    nums.append(i * i)    # add a square
print(nums)              # [0, 1, 4, 9, 16]
```

Common mistakes.

- Many beginners write `nums = nums.append(x)` and silently destroy their list.
- Many beginners try `nums.append(1, 2, 3)` expecting three items added. `Append` takes ONE argument. For many, use `nums.extend([1, 2, 3])`.

Quick Check. After `a = [1, 2]`; `a.append([3, 4])`, what is `a`?

Answer: `[1, 2, [3, 4]]`. The list `[3, 4]` was added as a single element, making a list of three items where the last one is itself a list.

`enumerate()` — loop with item and counter together

The big idea. `for index, item in enumerate(my_list):` walks through a list and gives you the position AND the value on each step. Perfect for numbered display.

This is like when... you read out a numbered list to someone: “one, milk; two, eggs; three, bread.” You need both the number and the item on every line.

The explanation.

Without `enumerate`, the usual pattern is the clumsy `for i in range(len(my_list)):` followed by `item = my_list[i]` inside. `Enumerate` gives you both in one line. By default it counts from 0. To count from 1 (more natural for display), pass `start=1`.

The code example.

```
items = ["5 + 3 = 8", "10 * 2 = 20", "100 / 4 = 25"]
for i, calc in enumerate(items, start=1):
    print(f"{i}. {calc}")
```

Common mistakes.

- Many beginners write `for item, i in enumerate(...)` — index and item swapped. The index comes first.
- Many beginners forget the `start=1` and confuse their users with zero-based numbering. Humans count from 1.

Quick Check. How would you print only items at odd human-positions (1st, 3rd, 5th)?

Answer: `for i, x in enumerate(my_list, start=1): if i % 2 == 1: print(x)`. `i % 2 == 1` is true for odd indices.

Catching multiple exceptions

The big idea. A single `try` block can have many `except` branches, each reacting to a specific type of failure with its own message.

This is like when... a hospital triage room. A nurse looks at the patient and routes them differently — broken arm one way, fever another, serious bleed a third.

The explanation.

Each `except SomeError:` handles exactly the type named. Python tries them top to bottom and runs the first one that matches. You can also group several types in one branch with a tuple: `except (ValueError, TypeError):`.

For a calculator, the common explosions from `eval` are: `SyntaxError` (malformed expression), `ZeroDivisionError` (`5 / 0`), `NameError` (using an undefined identifier), and the catch-all `Exception` for anything else.

The code example.

```
expr = input("calc> ")
try:
    print(eval(expr))
except ZeroDivisionError:
    print("Error: division by zero.")
except SyntaxError:
    print("Error: malformed expression.")
except Exception as e:
    # catch-all, last
    print(f"Error: {e}")
```

Common mistakes.

- Many beginners put the catch-all `except Exception:` ABOVE the specific ones. Python uses the first match, so the specific branches become dead code. Specific first, general last.
- Many beginners catch every error silently with no message — then their program quietly does the wrong thing.

Quick Check. In what order should you list `ZeroDivisionError`, `Exception`, and `SyntaxError`?

Answer: Specific first: `ZeroDivisionError` and `SyntaxError` in any order, then `Exception` last as the catch-all.

Command dispatch — same loop, different actions

The big idea. Often a program reads one line of input and must choose between many possible actions: do the maths, show history, clear history, quit. **Command dispatch** means inspecting the input, recognising a special keyword, and branching to the matching action.

This is like when... a waiter listens to what you say. “Menu” gets one response, “water” another, “bill” another, anything else gets “I’ll bring it to the chef.”

The explanation.

The shape is: normalise the input (`.strip().lower()`), check known commands with `if/elif`, and fall through to the default action (here, evaluating the maths) only if no command matched. Keep keywords short and obvious: `history`, `clear`, `quit`. As commands grow you can replace the if-chain with a dictionary of keyword → function — a dispatch table, like in Chapter 2.

The code example.

```
raw = input("calc> ").strip()
cmd = raw.lower()
if cmd == "quit":
    print("bye")
elif cmd == "history":
    print("(show history here)")
else:
    # fall through: it was not a command, treat as maths
    print(eval(raw))
```

Common mistakes.

- Many beginners compare the raw input without lowercasing — then `Quit` with a capital Q is not recognised.
- Many beginners forget `continue` after a command branch, and the body falls through into the calculator code with empty input.

Quick Check. Why lowercase the input before comparing it to a command string?

Answer: So that `QUIT`, `Quit`, and `quit` all match the same command.

④ How the concepts connect

A list called `history`, declared before the loop, accumulates one string per successful calculation. Each iteration of `while True` reads a line of input, strips and lowercases it, and checks for the commands `quit`, `history`, and `clear`. `continue` is used after each command branch so the maths code does not run on a keyword. If no command matched, the input is passed to `eval` inside a `try` block with several specific `except` branches (`ZeroDivisionError`, `SyntaxError`, then a catch-all `Exception`). When `eval` succeeds, the calculation is appended to `history` and the result is printed. When the user asks for `history`, `enumerate(history, start=1)` gives numbered output.

⑤ The build walkthrough

Step 1 · The shell. Print a small banner (“Calculator — type maths, commands: `history` / `clear` / `quit`”). Declare an empty list `history = []`. Open a `while True:` loop. Inside it, read a line of input and `.strip()` it. If the line is empty, `continue` so blank Enters do nothing.

Step 2 · The command branches. Take a `.lower()` copy of the cleaned input for command comparison. Compare it to `"quit"` and `break` if matched. Compare it to `"history"` — if matched, walk through the list with `enumerate(..., start=1)` and print each numbered line; if the list is empty, print a friendly placeholder; then `continue`. Compare it to `"clear"` — if matched, call `history.clear()`, print `"History cleared,"` and `continue`.

Step 3 · The maths branch. Wrap an `eval(raw)` call in a `try`. Use three `except` branches in order: `ZeroDivisionError` (`"division by zero"`), `SyntaxError` (`"malformed expression"`), and a catch-all `Exception` as `e` that reports `e`. Each branch should `continue` so the success-only code below does not run.

Step 4 · Display and record. When the eval succeeded, format the result for display: if it is a float whose value is a whole number, convert it to `int` so you show `8` not `8.0`. Print the formatted result. Append a string like `"5 + 3 = 8"` to `history` so the entry recorded is the user's actual input alongside the formatted result.

⑥ Understanding the design

Three design choices worth pausing on. First, *commands are checked before the eval* — otherwise typing the word `history` would crash on eval (it is not valid Python). The command-dispatch ladder must come first. Second, the history is updated *after* a successful eval, never inside the `except` branches — we only want a record of calculations that worked. Third, we converted `2.0` to `2` for display when the result is whole, because users do not want to see `5 + 3 = 8.0`. Tiny touches of polish like this are the difference between a script and a tool.

⑦ The exercise twist

Add parentheses support and an undo command.

What new concept does this need? Parentheses already work for free with `eval` — Python's parser handles them. The real twist is **undo**, which means removing the last entry from the history list and reporting which calculation got rolled back. The list method you need is `history.pop()` — it removes and returns the last item.

How does it extend what you built? Add another command branch, parallel to `history` and `clear`: when the user types `undo`, pop the last entry. Print which calculation you removed; print a friendly message if there was nothing to undo.

Hint for the hardest part. Guard against the empty case. `history.pop()` on an empty list raises `IndexError`. Check `if not history:` first — an empty list is falsy in Python.

⑧ What to remember from this chapter

- `eval(s)` runs `s` as Python; never use it on input from outside your machine.
- Lists are ordered, indexed from 0, growable, and mutable. `my_list.append(x)` adds `x` to the end.
- Use `enumerate(list, start=1)` when you want both the human-friendly number AND the value while looping.
- A single `try` can have many `except` branches; list specific errors first, the catch-all last.
- Command dispatch: normalise the input, check known keywords with `if/elif`, and put the "everything else" logic after.

⑨ Coming up next

In the next chapter you stop using just the keyboard and start reading from **files** — counting every word in any text the user names — and meet the `Counter` class from the standard library.

File Word Counter

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line tool that opens any text file the user names and reports a small analysis: total word count, number of unique words, the longest word, and a top-five list of the most frequent words. Punctuation does not confuse it; capitalisation does not split the same word in two; a missing file does not crash it. By the end of this chapter your programs stop talking only to the keyboard and start reading from disk — which means they can suddenly handle real-world data: books, log files, transcripts.

② The concepts you need

- **Opening a file with `with open(...)`.** The standard, safe way to read a file.
- **`.read()` vs `.readlines()`.** Two ways to pull text in.
- **`.split()`.** Breaking a string into a list of words.
- **`.lower()`.** Reused from earlier — case-insensitive comparison.
- **`string.punctuation` and `.strip()` on punctuation.** Stripping commas, full stops, etc. from word edges.
- **`collections.Counter`.** A purpose-built tool for counting how often things appear.
- **`FileNotFoundError`.** Catching the specific error when a file does not exist.

③ Concept deep dives

Opening a file with `with open(...)`

The big idea. `open(path)` gives you a file object that you can read from. Wrapping it in a `with` block guarantees the file is closed properly when you are done, even if an error happens in the middle.

This is like when... you open a fridge to grab milk. The `with` block is the polite reflex to close the door behind you when you walk away, so cold air does not leak out forever.

The explanation.

The shape is `with open(path) as f:` followed by an indented block that uses `f`. The variable `f` is the open file; the moment the block ends, Python closes it for you. Without `with`, you would have to remember `f.close()` at every exit point — and forgetting it on Windows can lock the file so nothing else can open it.

By default the file opens in *read text mode*, which is what you want for `.txt` files. Pass `encoding="utf-8"` for safety when the file might contain non-ASCII characters.

The code example.

```
with open("notes.txt", encoding="utf-8") as f:    # open for reading
    text = f.read()                             # one big string
print(len(text), "characters")                  # the block has ended
```

Common mistakes.

- Many beginners write `f = open(path)` without the `with` and forget `f.close()`. Use `with` and you cannot forget.
- Many beginners forget `encoding="utf-8"` and crash on files containing accented characters or emoji.

Quick Check. What is the value of `f` after the `with` block ends?

Answer: The variable `f` still exists, but the file behind it is closed — any further read or write on it raises an error.

.read() vs .readlines()

The big idea. A file object can be read either as one giant string (`.read()`) or as a list of lines (`.readlines()`). Choose based on whether you care about line structure.

This is like when... you can read a letter all at once, or read it one line at a time. The text is the same; the unit you process differs.

The explanation.

For word counting, line structure does not matter — you want every word regardless of which line it sits on. So `.read()` is the right choice: give me the whole text, I will deal with whitespace myself. `.readlines()` shines when each line *means* something (a log entry, a CSV row, a Markdown heading).

The code example.

```
with open("notes.txt", encoding="utf-8") as f:
    text = f.read()                                # whole file as one string

with open("notes.txt", encoding="utf-8") as f:
    lines = f.readlines()                          # list of strings, one per line
```

Common mistakes.

- Many beginners call `.read()` on huge files and run out of memory. For very large files, iterate line by line (`for line in f:`) instead.

Quick Check. If `notes.txt` contains three lines, what is `len(lines)` after `lines = f.readlines()`?

Answer: 3. Each line, including its trailing newline character, is one element.

.split() — string to list of words

The big idea. `text.split()` with no argument breaks the string at *any* whitespace — spaces, tabs, newlines — and returns a list of pieces. It collapses runs of multiple spaces into a single split.

This is like when... you tear a long paragraph along the spaces to get a pile of individual words.

The explanation.

For a simple word count, `text.split()` is exactly what you want. If you pass a specific separator (`text.split(",")`), it splits only on that character — useful for CSV-like data, but not for prose. The returned list never contains empty strings as long as you use the no-argument form.

The code example.

```
text = "Hello world!\nThis is a test."
words = text.split()                                # default: any whitespace
print(words)
# ['Hello', 'world!', 'This', 'is', 'a', 'test.']
```

Common mistakes.

- Many beginners write `text.split(" ")` (with a space argument) and get empty strings when the text has double spaces. Use `text.split()` with no argument for whitespace splitting.

Quick Check. What does `"a, b, c".split(",")` produce?

Answer: `['a', ' b', ' c']`. The commas are removed, but the leading spaces remain. You would `.strip()` each piece next.

Cleaning words: .lower() and string.punctuation

The big idea. Raw words from `.split()` are messy — "Hello," and "hello" look different to Python even though they are the same word. Lowercasing and stripping punctuation turn them into one form.

This is like when... you reduce all the variant spellings of someone's name to a single canonical version before alphabetising a guest list.

The explanation.

`string.punctuation` (after `import string`) is a ready-made string of every punctuation character: `!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~`. Passing it to `.strip()` removes any of those characters from the front and back of a word. Combine with `.lower()` and “Hello,” becomes “hello”.

The code example.

```
import string

raw = "Hello, world!"
clean = raw.lower().strip(string.punctuation)
print(repr(clean))    # 'hello, world!' - only the END was stripped!
```

The example above shows a subtle trap: `.strip()` only acts on the front and back. To clean each word, you split first, then strip each word individually.

Common mistakes.

- Many beginners call `.strip(string.punctuation)` on the whole text and wonder why interior punctuation survives. Split first, strip each word.
- Many beginners forget `import string` and get `NameError`.

Quick Check. What does `"hello!!".strip("!")` return?

Answer: `"hello"`. `.strip()` removes any of the characters you pass it, from both ends, until it hits a non-match.

`collections.Counter` — purpose-built counting

The big idea. `Counter` is a special kind of dictionary built for counting. You give it an iterable, and it returns a mapping of `item -> how many times it appeared`.

This is like when... you walk through a crowd with a clipboard, ticking off each person by name. At the end, the clipboard tells you the totals.

The explanation.

`Counter(my_list)` builds a counter in one call. The result has dictionary access (`counts["the"]` returns its count, or `0` if missing) and a special method `most_common(n)` that returns the top `n` items as a list of `(item, count)` tuples. This is dramatically tidier than writing a loop that increments a regular dictionary.

The code example.

```
from collections import Counter

words = ["the", "fox", "the", "dog", "the", "fox"]
counts = Counter(words)
print(counts)           # Counter({'the': 3, 'fox': 2, 'dog': 1})
print(counts.most_common(2)) # [('the', 3), ('fox', 2)]
```

Common mistakes.

- Many beginners write a manual for word in words: `counts[word] = counts.get(word, 0) + 1`. That works, but `Counter` is one line.
- Many beginners import it wrong: `from collections import Counter` (capital C). Lowercase `counter` does not exist.

Quick Check. What does `Counter("mississippi").most_common(1)` return?

Answer: `[('i', 4)]`. `Counter` works on any iterable, including strings, where each character is an item.

Handling `FileNotFoundError`

The big idea. If the user types a path that does not exist, `open()` raises `FileNotFoundError`. Catch it and print a friendly message instead of crashing.

This is like when... you ask a librarian for a book that is not on the shelf. They politely say “we do not have that one,” not throw the catalogue at you.

The explanation.

You met `try / except` in Chapters 2 and 3. `FileNotFoundError` is just another specific exception type — same shape. Wrap the `open()` call (or the whole with block) in a `try`, and put your recovery in `except FileNotFoundError:`. You can also catch `PermissionError` for files you cannot read.

The code example.

```
try:
    with open("missing.txt", encoding="utf-8") as f:
        text = f.read()
except FileNotFoundError:
    print("That file does not exist. Check the path.")
```

Common mistakes.

- Many beginners check `if os.path.exists(path):` before opening — but the file could be deleted in the microsecond between check and open. Just try to open it and catch the error.

Quick Check. What error does `open("does_not_exist.txt")` raise?

Answer: `FileNotFoundError`. The specific subclass of `OSError` that means “no such file.”

④ How the concepts connect

The user types a filename. Inside a `try / except FileNotFoundError`, a `with open(...)` block reads the file into one string. `.split()` breaks the string into a list of raw words. A list comprehension (you will meet these properly in Chapter 8 — for now think of it as a compact `for` loop) walks every word, applying `.lower()` and `.strip(string.punctuation)` to produce a clean list. `Counter(clean_words)` builds the frequency map in a single call. From the counter you read off total, unique count, and top five with `.most_common(5)`. A separate pass through the cleaned words finds the longest with `max(words, key=len)`.

⑤ The build walkthrough

Step 1 · Get the filename safely. Use `input()` to prompt for a filename, `.strip()` the answer, and bail out gracefully on empty input. Open the file inside a `try` block; in `except FileNotFoundError`, print a clear message and return early.

Step 2 · Read and split. Inside the `with` block, call `.read()` to get the whole text as one string. Then `.split()` it (no argument) to break into a list of raw words. If the list is empty, the file was effectively blank — print “no words found” and stop.

Step 3 · Clean every word. Import `string`. Build a new list of cleaned words by walking the raw list and, for each one, lowercasing it and stripping `string.punctuation` from both ends. Skip any word that becomes empty after stripping (a string of pure punctuation collapses to “”).

Step 4 · Compute the metrics. Import `Counter`. Build it from the cleaned list. The total word count is `len(cleaned)`. The unique count is `len(counter)`. The longest word is `max(cleaned, key=len)`. The top-five is `counter.most_common(5)`.

Step 5 · Display. Print a header. Print the three metrics on three lines with f-strings using thousands separators (`,` for the totals. Then iterate through the top-five with `enumerate(..., start=1)` and print each as “N. WORD (COUNT occurrences)” — pad the word with `:<15` for alignment.

⑥ Understanding the design

Three habits worth noting. First, *clean once, count once*: building a fully cleaned list before counting means every later operation works on the same normalised data — no chance of subtle bugs from counting “Hello,” and “hello” separately. Second, *catch the specific error you expect*: `FileNotFoundError` is narrow enough that the recovery message can be specific and helpful. Third, *use Counter over manual counting*: a single import buys you a more reliable, more readable, and significantly faster counter — when the standard library has the right tool, take it.

⑦ The exercise twist

Process multiple files at once, output a combined report as CSV.

What new concept does this need? Two ideas. First, the `glob` module — `glob.glob("*.txt")` returns a list of paths matching a pattern. Second, the `csv` module's `csv.writer` — give it an open file and call `.writerow([col1, col2, ...])` for each row.

How does it extend what you built? Wrap your single-file logic in a function `analyse(path)` that returns a dict of metrics. Loop over the `glob` results, call `analyse` on each, and collect a list of result rows. Then open an output CSV and write a header row followed by each result.

Hint for the hardest part. Each file might fail to open. Wrap each `analyse` call in its own `try / except FileNotFoundError` so one missing file does not abort the whole batch.

⑧ What to remember from this chapter

- with `open(path, encoding="utf-8") as f:` is the safe, standard way to read a file — it closes automatically.
- `.read()` gives you the whole file as one string; `.readlines()` gives a list of lines. Pick based on whether line structure matters.
- `text.split()` with no argument breaks at any whitespace and never returns empty strings.
- `import string` gives you `string.punctuation`; pass it to `.strip()` to clean a single word.
- `Counter(my_list)` builds a frequency map in one call, and `.most_common(n)` gives the top `n` as tuples.

⑨ Coming up next

In the next chapter you meet **dictionaries** properly and learn to persist data to disk with **JSON**, building a contact book that survives between runs of the program.

Contact Book

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line contact book that lets you add, view, search, update, and delete contacts — and that *remembers* them between runs because it saves to a JSON file on disk. Each contact has a name (the key) and a small block of details: phone, email. You open the program, your contacts are still there from yesterday. By the end of this chapter you have your first piece of software with proper persistence — the same idea that powers every app on your phone.

② The concepts you need

- **Dictionaries.** A collection where you look things up by name, not by position.
- **Common dict operations.** `dict[key]`, `dict.get(key)`, `key in dict`, `del dict[key]`.
- **Nested dictionaries.** A dict whose values are themselves dicts.
- **Iterating a dict with `.items()`.** Walking through keys and values together.
- **JSON serialisation.** Converting Python objects to text for saving, and back again.
- **`json.dump()` and `json.load()`.** The two functions you will use to read and write the file.
- **The CRUD pattern.** Create, Read, Update, Delete — the four operations every data app needs.

③ Concept deep dives

Dictionaries — lookup by name

The big idea. A **dictionary** maps *keys* to *values*. You write it with curly braces and colons: `{"name": "Alice", "age": 30}`. You look something up by its key — `person["name"]` — instead of by a numeric position.

This is like when... a real dictionary maps a word to its definition. You do not look up the 4,217th word; you look up “ephemeral” by name.

The explanation.

You met dictionaries briefly in Chapter 2 as the dispatch table. Here they become the main data structure. Keys are usually strings (sometimes numbers); values can be anything — strings, numbers, lists, even other dictionaries. Order is preserved in modern Python (insertion order), but you do not normally rely on it.

To add or update, just assign: `contacts["Alice"] = {...}`. To remove, use `del contacts["Alice"]`.

The code example.

```
contacts = {}                                # empty dict
contacts["Alice"] = {"phone": "+250788123", "email": "alice@ex.com"}
contacts["Bob"] = {"phone": "+250788456", "email": "bob@ex.com"}
print(contacts["Alice"])                     # full nested dict
print(contacts["Alice"]["email"])            # one level deeper
```

Common mistakes.

- Many beginners write `contacts[Alice]` (no quotes) and get `NameError` — Python looks for a *variable* called `Alice`. Keys that are strings must be quoted.
- Many beginners ask for a missing key with `dict[key]` and crash with `KeyError`. Use `dict.get(key)` (returns `None`) or check `key in dict` first.

Quick Check. What is `len({"a": 1, "b": 2, "c": 3})`?

Answer: 3. `len` of a dictionary is the number of keys.

dict.get(), in, and del — safe access

The big idea. Three small operations cover most safe dict use: `dict.get(key)` returns the value or `None` if missing; `key in dict` tests for presence without raising; `del dict[key]` removes an entry.

This is like when... you check a library card catalogue. You can ask “do we have this book?” (`in`), “what is its info?” (`get`), or “remove it from the catalogue” (`del`).

The explanation.

`get` takes an optional default: `dict.get(key, "unknown")` returns `"unknown"` if the key is missing. `in` is the most idiomatic way to ask membership. `del` removes both the key and its value — and raises `KeyError` if the key is not there, so guard with `in` first when in doubt.

The code example.

```
contacts = {"Alice": {"phone": "+250788123"}}
print(contacts.get("Alice"))           # {'phone': '+250788123'}
print(contacts.get("Charlie"))         # None
print(contacts.get("Charlie", "no one")) # 'no one'
print("Alice" in contacts)             # True
if "Alice" in contacts:
    del contacts["Alice"]              # safe deletion
```

Common mistakes.

- Many beginners use `if dict[key]:` to test membership — this crashes on missing keys. Use `if key in dict:`.

Quick Check. What does `{"x": 0}.get("x")` return?

Answer: `0`. The key exists; the value happens to be `0`. Do not confuse “missing” with “falsy value.”

Iterating with .items()

The big idea. `for key, value in my_dict.items():` walks through the dictionary, giving you both the key and the value on each step. Perfect for displaying or processing every entry.

This is like when... you read every entry in an address book aloud — both the name and the details, in order.

The explanation.

There are three iteration helpers: `.keys()` (just the keys), `.values()` (just the values), and `.items()` (pairs). `.items()` is the most common. Inside the loop, `key` and `value` are just names — you can call them anything; I usually use `name`, `details` when the context is people.

The code example.

```
contacts = {
    "Alice": {"phone": "+250788123"},
    "Bob":   {"phone": "+250788456"},
}
for name, details in contacts.items():
    print(f"{name}: {details['phone']}")
```

Common mistakes.

- Many beginners write `for x in my_dict:` and expect pairs — but that iterates over *keys only*. Use `.items()` if you want both.
- Many beginners modify the dict (add/remove keys) inside the loop, which raises `RuntimeError`. Iterate over a copy: `for key in list(my_dict):` if you must mutate.

Quick Check. What does `for x in {"a": 1, "b": 2}:` give `x` on each step?

Answer: `"a"` then `"b"` — just the keys.

JSON — text format for structured data

The big idea. JSON (JavaScript Object Notation) is a plain-text format that looks almost exactly like a Python dictionary. It is the standard way to save structured data to a file or send it over a network.

This is like when... you write your phone contacts down on paper before backing up your phone, so you can read them later or copy them to a new device. JSON is that paper, but for programs.

The explanation.

JSON supports the same building blocks Python uses: objects (like dicts), arrays (like lists), strings, numbers, booleans, and null. Python's `json` module converts (serialises) Python objects to JSON text and parses JSON text back to Python objects. Two restrictions: JSON object keys must be strings, and JSON has no tuples (they become arrays).

A JSON contact file looks like this:

```
{
  "Alice": {"phone": "+250788123", "email": "alice@ex.com"},
  "Bob":   {"phone": "+250788456", "email": "bob@ex.com"}
}
```

You will recognise it as a near-identical mirror of the Python dict.

Common mistakes.

- Many beginners use single quotes in JSON — JSON requires double quotes around every string.
- Many beginners try to put a tuple in JSON. Tuples become lists when serialised, then come back as lists.

Quick Check. Why can you not save a tuple to JSON and get a tuple back?

Answer: JSON has no tuple type. Tuples are serialised as arrays, and arrays deserialise as Python lists.

`json.dump()` and `json.load()`

The big idea. `json.dump(obj, file)` writes a Python object as JSON to an open file. `json.load(file)` reads JSON from an open file and returns the matching Python object. They are exact mirrors.

This is like when... you photograph a scene and later look at the photo. `dump` is the snapshot; `load` reproduces what you saw.

The explanation.

Both functions need an open file object — combine with `with open(...)`. For writing, open with mode "w" (write) and pass `indent=2` for human-readable output. For reading, open with mode "r" (the default) and catch `FileNotFoundError` for the first-run case where the file does not exist yet.

The code example.

```
import json

contacts = {"Alice": {"phone": "+250788123"}}

# write
with open("contacts.json", "w", encoding="utf-8") as f:
    json.dump(contacts, f, indent=2)

# read
with open("contacts.json", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded["Alice"]["phone"])
```

Common mistakes.

- Many beginners use `json.dumps()` (with an s — “dump string”) when they meant `json.dump()` (to a file). Both exist; the s version returns the JSON as a string instead of writing it.
- Many beginners forget `encoding="utf-8"` and lose non-ASCII characters on Windows.

Quick Check. What does `json.load(open("missing.json"))` raise if the file does not exist?
Answer: `FileNotFoundError`, raised by `open()` itself before `json.load` ever sees the file.

The CRUD pattern

The big idea. **CRUD** stands for Create, Read, Update, Delete — the four operations every data application needs. Once you have these working on one data type (contacts), the same shape transfers to todos, notes, expenses, anything.

This is like when... a paper address book lets you write someone in, look them up, scribble out their old number and put in the new one, or rip the page out entirely.

The explanation.

In a CLI app, the four operations usually map to four menu choices. Inside each choice, you do *one* thing to your in-memory data and then save the whole file again so the change persists. Every operation needs input validation: a name that does not exist cannot be updated; a contact already in the book is either rejected or overwritten depending on your choice.

Common mistakes.

- Many beginners forget to save after a change — the program works in one session but loses everything when restarted.
- Many beginners save once at the end instead of after each change — a crash mid-session loses everything since the last save.

Quick Check. Which letter of CRUD corresponds to `del contacts[name]`?
Answer: **D** — Delete.

④ How the concepts connect

On startup, a `try / except FileNotFoundError` wraps a `json.load()` that fills the in-memory `contacts` dictionary — or starts empty if this is the first run. A `while True` loop shows a numbered menu. Each menu choice routes to a small helper function (`add()`, `view()`, `search()`, `update()`, `delete()`) that operates on the dictionary using `dict[key] = ...`, `dict.get(key)`, `for k, v in dict.items():`, `key in dict`, and `del dict[key]`. After every change, a single `save()` function dumps the dict to JSON. Quitting just ends the loop — the file is already up to date.

⑤ The build walkthrough

Step 1 · Load on start, save on every change. Write two helper functions. `load_contacts(path)` opens the file, returns the parsed dict, and returns an empty dict on `FileNotFoundError`. `save_contacts(contacts, path)` opens the file in write mode and `json.dumps` the dict with `indent=2`. Call `load_contacts` once at the top of the program.

Step 2 · The menu loop. Open a `while True:` loop. Print a numbered menu: add, view all, search, update, delete, quit. Read the user's choice with `input().strip()`. Use an `if / elif` chain to dispatch to the matching action. Quit **breaks** the loop.

Step 3 · Add. Prompt for name, phone, email. Strip every input. Validate — name must not be empty, and if the name is already a key, ask whether to overwrite. Assign `contacts[name] = {"phone": phone, "email": email}` and call `save_contacts`.

Step 4 · View all. If the dict is empty, print “No contacts yet” and return. Otherwise, walk `contacts.items()` and print one neat line per contact: name on the left padded with `:<20`, phone next, then email.

Step 5 · Search and update. Both take a name as input. Search: if `name in contacts`, print their details; else “Not found.” Update: if `name in contacts`, ask which field to change (phone or email) and the new value; modify the inner dict; save. Else “Not found.”

Step 6 · Delete. Take a name. If found, ask for confirmation (“are you sure? y/n”); on yes, `del contacts[name]` and save. Else “Not found.” Always print a confirmation message so the user knows it worked.

⑥ Understanding the design

Three habits to internalise. First, *save after every change*, not once at the end — a crash mid-session loses nothing. Second, *every read operation should guard with `in` or `.get()`* so missing keys never crash the program. Third, *separate the data (the dict) from the file (the JSON)* — your in-memory dict is the source of truth during a session; the file is just its written form. This is the same separation databases make between memory and disk, just simpler.

⑦ The exercise twist

Add fields (*address, birthday, notes*), categorise contacts into groups (*family, work, friends*), and export to CSV.

What new concept does this need? Two ideas. First, an extra dict field on each contact: `category`, with a fixed set of allowed values. Validate on add. Second, the `csv` module's `csv.DictWriter` — you give it a list of column names, then call `.writerow(dict)` for each row.

How does it extend what you built? Add the new fields to your add and update prompts. Add a new menu choice “filter by category” that walks the dict and prints only matching contacts. Add an “export” choice that opens a `.csv` file and writes one row per contact.

Hint for the hardest part. `DictWriter` needs every row dict to have the same keys. If older contacts in your JSON do not have a `category` yet, default missing fields to an empty string before writing — `row.get(field, "")` — so the CSV is well-formed.

⑧ What to remember from this chapter

- A dictionary maps keys to values: `d[key] = value`. Look up with `d[key]`, safely with `d.get(key)`, test with `key in d`, remove with `del d[key]`.
- `for k, v in d.items():` walks pairs; modifying the dict during iteration crashes — iterate over `list(d)` first.
- `json.dump(obj, f, indent=2)` writes; `json.load(f)` reads — both want an open file from `with open(...)`.
- Catch `FileNotFoundError` to handle the first-run case where the data file does not exist yet.
- CRUD = Create, Read, Update, Delete. Save after every change; reload only on start.

⑨ Coming up next

In the next chapter you build a todo list using a **list of dictionaries** — your first compound data structure — plus the `datetime` module for timestamps and a JSON-with-dates pattern that comes up everywhere in real software.

Todo List

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line todo manager. You add tasks, each with a title and a creation timestamp; you view them as a numbered list with a tick or empty box showing whether they are done; you mark tasks complete by number; you delete them; and you see live statistics — total, done, pending, and a completion percentage. It all persists to JSON, so your list survives between runs. By the end you will be comfortable with a **list of dictionaries** — the workhorse data shape behind almost every real application's data.

② The concepts you need

- **Lists of dictionaries.** A list where each element is a small dict — the most common compound structure in real code.
- **enumerate() for numbered display.** Reused from Chapter 4 — show tasks as 1, 2, 3.
- **The datetime module.** Getting the current date and time, and formatting it as text.
- **.strftime().** Turning a datetime into a human-readable string.
- **JSON with datetimes.** The trick: store dates as strings, because JSON has no date type.
- **list.pop() and in-place editing.** Removing a task by index, and flipping its status.
- **Computing a percentage.** Done divided by total, rounded.

③ Concept deep dives

Lists of dictionaries

The big idea. When you have *many things* and each thing has *several attributes*, the natural shape is a list (the many) of dictionaries (the attributes). [{"title": "Buy milk", "done": False}, {...}, {...}].

This is like when... a spreadsheet: each row is one record (a dict), and the whole sheet is the list of rows.

The explanation.

In Chapter 6 you used a dict-of-dicts keyed by name. That works when each record has a unique natural key. A todo list has no natural key — tasks are just an ordered sequence — so a *list* of dicts fits better. You access a task by position (`tasks[0]`), and each task's fields by key (`tasks[0]["title"]`).

This is the shape `json.load` gives you back when the file's top level is a JSON array, so it pairs naturally with persistence.

The code example.

```
tasks = [
    {"title": "Buy milk", "done": True},
    {"title": "Finish ch7", "done": False},
]
print(tasks[0]["title"])           # Buy milk
print(tasks[1]["done"])            # False
tasks.append({"title": "Sleep", "done": False})  # add a new task
```

Common mistakes.

- Many beginners mix up the two levels: `tasks["title"]` crashes because `tasks` is a list (index by number), not a dict. The dict is *inside*: `tasks[0]["title"]`.

Quick Check. How do you read the title of the third task?

Answer: `tasks[2]["title"]` — index 2 for the third element, then the "title" key inside it.

The `datetime` module

The big idea. `datetime` is the standard module for working with dates and times. `datetime.now()` gives you the current moment as a rich object you can format or do arithmetic on.

This is like when... you glance at a clock and a calendar at the same instant — `datetime.now()` captures both the date and the time in one object.

The explanation.

After `from datetime import datetime`, call `datetime.now()` to get a `datetime` object. It holds year, month, day, hour, minute, second. You rarely use those fields directly; instead you *format* the object into a string for display, or compare two datetimes to measure elapsed time (Chapter 9 and 22 lean on that).

The code example.

```
from datetime import datetime

now = datetime.now()
print(now)           # 2026-05-26 14:30:05.123456
print(now.year)      # 2026
print(now.hour)      # 14
```

Common mistakes.

- Many beginners write `import datetime` then call `datetime.now()` and crash — because `datetime` is then the *module*, and the *class* is `datetime.datetime`. Use `from datetime import datetime` to get the class directly.

Quick Check. With `from datetime import datetime`, how do you get the current moment?

Answer: `datetime.now()`.

`.strftime()` — formatting a `datetime` as text

The big idea. `.strftime(format)` turns a `datetime` into a string laid out exactly how you want, using percent-codes like `%Y` for year and `%m` for month.

This is like when... you fill out a date on a form in the exact boxes it asks for — DD/MM/YYYY here, Month Day, Year there. The underlying date is the same; only the layout changes.

The explanation.

The common codes: `%Y` (four-digit year), `%m` (month 01-12), `%d` (day 01-31), `%H` (hour 00-23), `%M` (minute). So `now.strftime("%Y-%m-%d")` gives "2026-05-26". The reverse — parsing a string *into* a `datetime` — is `.strptime()`, which you will meet in Chapter 9.

The code example.

```
from datetime import datetime

now = datetime.now()
stamp = now.strftime("%Y-%m-%d %H:%M")
print(stamp)           # 2026-05-26 14:30
date_only = now.strftime("%Y-%m-%d")
print(date_only)       # 2026-05-26
```

Common mistakes.

- Many beginners confuse `strftime` (format → string) with `strptime` (parse string → `datetime`). Memory aid: the *f* is for *format*, the *p* is for *parse*.

Quick Check. What does `datetime(2026, 1, 9).strftime("%d/%m/%Y")` produce?

Answer: "09/01/2026" — day, month, year, with zero-padding.

JSON with `datetimes` — store as strings

The big idea. JSON has no date type. If you try to `json.dump` a `datetime` object directly, it crashes. The fix: convert `datetimes` to strings before saving, and (if needed) parse them back after loading.

This is like when... you cannot post a physical clock through the mail, so you write the time on a postcard. The recipient reads the postcard to know the time.

The explanation.

When you create a task, store its timestamp as a string immediately: `"created": datetime.now().strftime("%Y-%m-%d %H:%M")`. Now the whole task dict contains only JSON-safe types (strings, booleans), so it dumps and loads cleanly. For a todo list you usually only *display* the created date, so keeping it as a string the whole time is simplest.

The code example.

```
from datetime import datetime
import json

task = {
    "title": "Finish ch7",
    "done": False,
    "created": datetime.now().strftime("%Y-%m-%d %H:%M"), # string, not object
}
print(json.dumps(task))      # works – every value is JSON-safe
```

Common mistakes.

- Many beginners store the raw `datetime.now()` object and then crash on `json.dump` with `TypeError: Object of type datetime is not JSON serializable`. Format to a string at creation time.

Quick Check. Why does `json.dump({"t": datetime.now()}, f)` crash?

Answer: JSON has no datetime type, so the encoder does not know how to represent the object. Convert it to a string first.

list.pop() and in-place editing

The big idea. `list.pop(index)` removes the item at that index and returns it. To edit a task in place — for example, mark it done — you reach into the list by index and change the dict's field.

This is like when... you cross an item off a written list (editing in place) or tear it off entirely (popping it).

The explanation.

`tasks.pop(0)` removes and returns the first task. With no argument, `pop()` removes the last. To *mark done* rather than delete, you do not pop — you mutate: `tasks[i]["done"] = True`. Because the dict inside the list is the same object you stored, changing its field changes the stored task directly.

The code example.

```
tasks = [{"title": "a", "done": False}, {"title": "b", "done": False}]
tasks[0]["done"] = True      # mark first task done (in place)
removed = tasks.pop(1)       # delete second task, capture it
print(removed["title"])      # b
print(tasks)                 # [{"title": "a", "done": True}]
```

Common mistakes.

- Many beginners `pop()` while iterating over the same list, which skips elements. Collect indices to remove first, or iterate over a copy.
- Many beginners forget that users count from 1 but lists index from 0 — subtract 1 when converting a user's task number to an index.

Quick Check. A user asks to delete “task 3.” What index do you pop?

Answer: Index 2 — `tasks.pop(3 - 1)`. Humans count from 1; lists from 0.

Computing a percentage

The big idea. A completion percentage is `done / total * 100`, rounded for display. The only trap is dividing by zero when the list is empty.

This is like when... you report “7 of 10 chores done, 70%.” The maths is trivial; the edge case (zero chores) is where bugs hide.

The explanation.

Count the done tasks (a sum over a condition), divide by the total, multiply by 100, and `round()`. Guard the empty case: if total is zero, report 0% rather than crashing. A neat way to count done tasks is `sum(1 for t in tasks if t["done"])` — a generator expression you will see again in Chapter 9.

The code example.

```
tasks = [{"done": True}, {"done": False}, {"done": True}]
total = len(tasks)
done = sum(1 for t in tasks if t["done"])    # count where done is True
pct = round(done / total * 100) if total else 0
print(f"{done}/{total} done ({pct}%")        # 2/3 done (67%)
```

Common mistakes.

- Many beginners divide by `len(tasks)` without checking it is non-zero, and crash with `ZeroDivisionError` on an empty list.

Quick Check. What does `round(2 / 3 * 100)` give?

Answer: 67. 66.66... rounds to the nearest whole number.

④ How the concepts connect

On start, `json.load` fills a list of task dicts (or an empty list on first run). A `while True` menu loop offers add, view, mark-done, delete, quit. **Add** appends a dict with a title, `"done": False`, and a `.strftime()` timestamp string. **View** uses `enumerate(tasks, start=1)` to print each task with a [✓] or [] box plus its created date, and a stats line computed with `sum(...)` and a guarded percentage. **Mark-done** converts the user’s 1-based number to a 0-based index and sets `tasks[i]["done"] = True`. **Delete** pops by the converted index. Every change is followed by a `json.dump` save.

⑤ The build walkthrough

Step 1 · Load and save helpers. As in Chapter 6: `load_tasks(path)` returns the parsed list or an empty list on `FileNotFoundError`; `save_tasks(tasks, path)` dumps the list with `indent=2`. The only difference from the contact book is that the top-level object is a *list*, not a dict.

Step 2 · The menu loop. A `while True`: with a numbered menu and an `if / elif` dispatch. Read the choice with `input().strip()`. Quit breaks.

Step 3 · Add a task. Prompt for a title and strip it; reject empty titles. Build a dict with the title, `"done": False`, and `"created"` set to `datetime.now().strftime("%Y-%m-%d %H:%M")`. Append it to the list and save.

Step 4 · View with stats. If empty, print “No tasks yet” and return. Compute `total`, `done` (with `sum(1 for t in tasks if t["done"])`), and the guarded percentage. Print the stats line. Then `enumerate(tasks, start=1)` and print each as `"N. [✓] title (created DATE)"`, choosing the box character based on `t["done"]`.

Step 5 · Mark done and delete. Both prompt for a task number. Convert to an index by subtracting 1, and validate it is in range (`0 <= i < len(tasks)`) inside a `try / except (ValueError, IndexError)`. Mark-done sets `tasks[i]["done"] = True`; delete calls `tasks.pop(i)`. Save after either.

⑥ Understanding the design

Three habits. First, *store dates as strings at creation time* — it sidesteps the entire JSON-datetime problem and is fine when you only display the date. Second, *validate the user’s number against the list length* before indexing — converting “task 9” to an index and popping blindly is how you get `IndexError` crashes. Third, *derive stats, never store them* — the completion percentage is computed fresh from the list every time you view it, so it can never drift out of sync with the actual tasks. Storing a number you could compute is a classic source of bugs.

⑦ The exercise twist

Add priority levels (*high, medium, low*) with sorting, and due dates with overdue alerts.

What new concept does this need? Two ideas. First, sorting a list of dicts by a field, using `sorted(tasks, key=lambda t: t["priority"])` — the `key` function tells `sorted` what to compare. Second, comparing dates: parse the stored due-date string back into a `datetime` with `datetime.strptime`, and compare it to `datetime.now()` to decide whether a task is overdue.

How does it extend what you built? Add `priority` and `due` fields at add-time. In the view, sort by priority before displaying, and flag any task whose due date is in the past with an `OVERDUE` marker.

Hint for the hardest part. Map your priority words to numbers for sorting — `{"high": 0, "medium": 1, "low": 2}` — so `sorted` puts high-priority tasks first. Sorting raw words alphabetically would put “high” after “low,” which is wrong.

⑧ What to remember from this chapter

- A list of dictionaries is the standard shape for “many records, each with several fields.” Access as `tasks[i]["field"]`.
- `from datetime import datetime`, then `datetime.now()` for the current moment and `.strftime("%Y-%m-%d %H:%M")` to format it.
- JSON has no date type — store timestamps as strings (format with `strftime` at creation).
- `list.pop(i)` removes and returns item `i`; to edit in place, mutate the dict: `tasks[i]["done"] = True`.
- Convert 1-based user numbers to 0-based indices, validate the range, and guard percentage maths against an empty list.

⑨ Coming up next

In the next chapter you build a password generator and meet **list comprehensions**, the `string` module’s character sets, and the security-focused `secrets` module — your first brush with writing code where safety matters.

Password Generator

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A password generator that builds strong, random passwords to a length and character-mix you choose — uppercase, lowercase, digits, symbols, in any combination. It evaluates the strength of what it produces and draws a little visual meter. Crucially, it uses the *right* random source for the job: the `secrets` module, which is designed for security, rather than the ordinary `random` module. By the end you will have met list comprehensions — one of Python’s most elegant features — and learned a real lesson about choosing tools where safety matters.

② The concepts you need

- **random vs secrets.** Two random sources, and why the choice matters for passwords.
- **The string module’s character sets.** Ready-made strings of letters, digits, punctuation.
- **Building a character pool.** Combining the chosen sets into one big pool to draw from.
- **List comprehensions.** A compact way to build a list — Python’s signature feature.
- **"".join().** Gluing a list of characters into a single string.
- **Strength evaluation.** Simple rules that score a password weak / medium / strong.
- **A text progress bar.** Drawing a meter with block characters.

③ Concept deep dives

random vs secrets — choosing the right randomness

The big idea. Both modules produce random values, but `random` is *predictable* if an attacker learns its internal state, while `secrets` is *cryptographically secure* — designed to resist prediction. For anything security-related — passwords, tokens, keys — use `secrets`.

This is like when... you shuffle cards for a friendly game (`random` is fine) versus shuffling cards in a casino where money is on the line (you need a tamper-proof shuffle). Same action, very different stakes.

The explanation.

`random` uses a fast algorithm seeded from the system clock. Great for games, simulations, picking a random greeting. But given enough outputs, its future values can be predicted — fatal for a password. `secrets`, added in Python 3.6, draws from the operating system’s secure randomness source. Its key function is `secrets.choice(sequence)`, which picks one random element, just like `random.choice` — but safely.

The code example.

```
import secrets

pool = "abcdefABCDEF0123456789"
ch = secrets.choice(pool)           # one cryptographically secure pick
print(ch)                          # e.g. 'C'
```

Common mistakes.

- Many beginners use `random.choice` for passwords because it is the one they learned first. For anything an attacker might target, use `secrets.choice`.

Quick Check. Which module should generate a password reset token?

Answer: `secrets`. Anything an attacker could exploit needs cryptographically secure randomness.

The string module's character sets

The big idea. After `import string`, you get ready-made constants holding common character groups: `string.ascii_lowercase`, `string.ascii_uppercase`, `string.digits`, and `string.punctuation`. No need to type out the alphabet by hand.

This is like when... you reach for a labelled box of LEGO bricks — “all the red ones,” “all the blue ones” — instead of hunting for each piece individually.

The explanation.

`string.ascii_lowercase` is “abcdefghijklmnopqrstuvwxyz”. `string.ascii_uppercase` is the capitals. `string.digits` is “0123456789”. `string.punctuation` is the symbol set. Each is just a string, so you can concatenate them with `+` to build exactly the pool you want.

The code example.

```
import string
print(string.ascii_lowercase) # abcdefghijklmnopqrstuvwxyz
print(string.digits)         # 0123456789
pool = string.ascii_uppercase + string.digits # caps + numbers
print(pool)                   # ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
```

Common mistakes.

- Many beginners type out “abcdef...xyz” by hand and make typos. Use the constants.

Quick Check. What is `string.digits`?

Answer: The string “0123456789”.

Building a character pool

The big idea. Based on which character types the user wants, you assemble one combined pool string by concatenating the chosen `string` constants. You then draw every password character from that pool.

This is like when... you decide which ingredients go in the salad bowl first, then toss — the bowl is your pool, and each forkful is a random draw.

The explanation.

Start with an empty pool string. For each character type the user said yes to, add the matching constant. After building, guard against the empty pool — if the user said no to everything, there is nothing to draw from, so you must reject that and ask again.

The code example.

```
import string

pool = ""
if True: pool += string.ascii_uppercase # user wanted uppercase
if True: pool += string.digits         # user wanted digits
if not pool:
    print("You must pick at least one character type.")
else:
    print(f"Pool has {len(pool)} characters.")
```

Common mistakes.

- Many beginners forget the all-no case and then call `secrets.choice("")`, which crashes on an empty sequence. Always check `if not pool:` first.

Quick Check. What goes wrong if the pool is the empty string?

Answer: `secrets.choice("")` raises `IndexError` — you cannot pick from nothing. Reject the empty pool before generating.

List comprehensions

The big idea. A **list comprehension** builds a list in one compact line: `[expression for item in iterable]`. It is Python's elegant replacement for the common “make an empty list, loop, append” pattern.

This is like when... a chef says “one skewer for each piece of chicken” — a single sweeping instruction that produces the whole tray, rather than threading them one slow append at a time.

The explanation.

The long form is: `result = []` then `for x in things: result.append(f(x))`. The comprehension form is `result = [f(x) for x in things]`. They produce the same list. You can add a filter: `[x for x in things if condition]`. For the generator, drawing N random characters is `[secrets.choice(pool) for _ in range(length)]` — the underscore is a throwaway loop variable, used when you do not need the value, only the count.

The code example.

```
import secrets

pool = "ABC123"
length = 5
chars = [secrets.choice(pool) for _ in range(length)] # list of 5 chars
print(chars) # e.g. ['B', '1', 'A', 'C', '3']
squares = [n * n for n in range(5)] # [0, 1, 4, 9, 16]
evens = [n for n in range(10) if n % 2 == 0] # [0, 2, 4, 6, 8]
```

Common mistakes.

- Many beginners try to cram heavy logic into a comprehension until it is unreadable. If it needs more than a simple expression and one condition, use a normal loop.
- Many beginners forget the comprehension *produces* a list — you still need to assign it to a variable.

Quick Check. What does `[c.upper() for c in "abc"]` produce?

Answer: `['A', 'B', 'C']` — a list, with each character uppercased.

"".join() — list of characters to one string

The big idea. `separator.join(list_of_strings)` glues the pieces into one string, putting the separator between them. With an empty separator `""`, it simply concatenates — perfect for turning a list of single characters into a password.

This is like when... you thread beads onto a string. `""` means the beads touch with no spacer between them.

The explanation.

The separator is the string you call `.join` on. `"".join(["a", "b", "c"])` gives `"abc"`. `"-".join(["2026", "05", "26"])` gives `"2026-05-26"`. Join only works on a list of *strings* — if the list has numbers, convert them first. For the password, you join the list of random characters from the comprehension into the final string.

The code example.

```
chars = ["K", "9", "$", "m"]
password = "".join(chars) # glue with nothing between
print(password) # K9$m
dashed = "-".join(["a", "b", "c"])
print(dashed) # a-b-c
```

Common mistakes.

- Many beginners write `chars.join("")` — backwards. The *separator* owns the method: `"".join(chars)`.
- Many beginners join a list containing numbers and get `TypeError`. Join needs all strings.

Quick Check. What does `" ".join(["Hello", "world"])` produce?

Answer: "Hello world" — the two words joined by a single space.

Strength evaluation and a text meter

The big idea. Score the password by simple rules — length and how many character types it uses — into weak, medium, or strong. Then draw a meter by repeating a filled block character proportional to the score.

This is like when... a phone signal indicator: a quick visual summary of an underlying measurement, easy to read at a glance.

The explanation.

A reasonable rule: weak if under 8 characters; strong if 12+ characters with 3+ character types; medium otherwise. To count character types used, check whether the password contains any lowercase, any uppercase, any digit, any symbol — each true adds one to the type count. The `any()` function with a generator does this neatly: `any(c.isdigit() for c in password)`.

For the meter, map the strength to a percentage, then build a bar: a filled block (█) repeated for the filled portion, a light block (░) for the rest, using string multiplication.

The code example.

```
pw = "K9$mPx2vNqLrTzF"
types = 0
types += any(c.islower() for c in pw)
types += any(c.isupper() for c in pw)
types += any(c.isdigit() for c in pw)
types += any(not c.isalnum() for c in pw) # a symbol
print(f"{len(pw)} chars, {types} types") # 15 chars, 4 types
```

Common mistakes.

- Many beginners count *occurrences* of each type instead of *whether each type is present at all*. For the type count you want presence (`any`), not frequency.

Quick Check. What does `any(c.isdigit() for c in "abc")` return?

Answer: `False` — there are no digits, so `any` finds nothing true.

④ How the concepts connect

The user gives a length and four yes/no answers for the character types. From those, an `if`-chain builds a `pool` string by concatenating the chosen `string` constants, and guards the empty-pool case. A list comprehension `[secrets.choice(pool) for _ in range(length)]` draws the characters securely, and `"".join(...)` glues them into the password string. A strength function counts length and (via `any(...)` generators) how many types appear, classifies the result, and a small bar built with string multiplication visualises it. A loop offers to generate again.

⑤ The build walkthrough

Step 1 · Gather the settings. Prompt for the length and validate it is a positive integer inside a `try / except ValueError`. Prompt four yes/no questions for uppercase, lowercase, digits, symbols — normalise each answer with `.strip().lower()` and treat `"y"` or `"yes"` as true.

Step 2 · Build the pool. Import `string`. Start with an empty pool string and concatenate the matching constant for each “yes”: `string.ascii_uppercase`, `string.ascii_lowercase`, `string.digits`, `string.punctuation`. If the pool is empty (all no), print an error and loop back to ask again.

Step 3 · Generate. Import `secrets`. Use a list comprehension `[secrets.choice(pool) for _ in range(length)]` to draw the characters, then `"".join(...)` them into the final password string.

Step 4 · Evaluate strength. Write a function that takes the password and returns a label and a percentage. Count types with four `any(...)` generator checks. Apply the rule: weak under 8 characters; strong at 12+ with 3+ types; medium otherwise. Map each label to a percentage for the meter.

Step 5 · Display and loop. Print the password, the strength label, and a meter built from "█" * filled + "░" * empty. Ask “generate another? (y/n)” and loop or stop.

⑥ Understanding the design

Three habits. First, *use secrets, not random, the moment security is involved* — it costs nothing extra and closes a real attack. Second, *guard the empty pool before generating* — a single `if not pool:` check turns a crash into a friendly re-prompt. Third, *separate generation from evaluation* into two functions — the generator only makes a password, the evaluator only judges one. That separation means you can reuse the evaluator later to score passwords the user typed themselves, not just ones you generated.

⑦ The exercise twist

Add a *passphrase mode*: 3–5 random words joined by dashes.

What new concept does this need? A word list to draw from — either a built-in list of common words you define, or words read from a file (you learned file reading in Chapter 5). The generation itself is the same idea: `secrets.choice` from the word list instead of from a character pool.

How does it extend what you built? Add a mode question at the start: characters or passphrase. For passphrase mode, draw N words with a comprehension and join them with `"-".join(...)` instead of `"".join(...)`. The strength logic shifts — passphrase strength comes from the number of words and the size of the word list, not character types.

Hint for the hardest part. A passphrase of 4 words drawn from a 2,000-word list is far stronger than it looks, because strength is about the *number of equally-likely possibilities*. Four words from 2,000 options is 2,000 to the 4th power combinations — astronomically large. Explain this to the user as part of the output; it teaches them why “correct-horse-battery-staple” beats “P@ss1”.

⑧ What to remember from this chapter

- Use `secrets` (not `random`) for anything security-related; `secrets.choice(pool)` picks one element securely.
- `import string` gives `ascii_lowercase`, `ascii_uppercase`, `digits`, `punctuation` — concatenate them to build a pool.
- A list comprehension `[expr for item in iterable]` builds a list in one line; add `if condition` to filter.
- `"".join(list_of_chars)` glues a list of strings into one; the separator owns the method.
- Score strength by length and type-count (`any(c.isdigit() for c in pw)`); guard the empty pool before generating.

⑨ Coming up next

In the next chapter you build an expense tracker, parsing and formatting dates with `strptime` and `strftime`, filtering lists with comprehensions, and aggregating money with `sum()` over generator expressions.

Expense Tracker

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line expense tracker that records what you spend, sorts it into categories, and produces a monthly report: total spent, a per-category breakdown with percentages, your daily average, and your single biggest expense. It persists to JSON, so your spending history accumulates over time. By the end you will be fluent at the three operations that power every analytics tool: **filtering** a list down to what you care about, **aggregating** numbers into totals, and **formatting** money so it reads like money.

② The concepts you need

- **`datetime.strptime()`**. Parsing a date string into a real datetime object (the reverse of `strftime`).
- **Filtering with comprehensions**. Selecting list items that match a condition.
- **`sum()` with generator expressions**. Adding up a computed value across a list in one line.
- **Dictionary aggregation**. Grouping and totalling by category.
- **Number formatting in f-strings**. Thousands separators and fixed decimals.
- **Guarding against empty data**. Avoiding divide-by-zero on a month with no expenses.

③ Concept deep dives

`datetime.strptime()` — parsing a string into a date

The big idea. `strptime` is the mirror of `strftime`. Where `strftime` turns a datetime *into* a string, `strptime` *parses* a string *into* a datetime — so you can do date maths and comparisons on it.

This is like when... you read a date written on a form (“2026-05-26”) and mentally place it on a calendar. The text becomes a real point in time you can reason about.

The explanation.

You met `strftime` in Chapter 7. `strptime(text, format)` takes the string and a format describing its layout, and returns a datetime. The format codes are the same: `%Y-%m-%d` matches “2026-05-26”. If the string does not match the format, it raises `ValueError` — so wrap user-entered dates in `try / except`.

Once parsed, you can read `.month` and `.year` to filter expenses to a given month, or compare two dates directly with `<` and `>`.

The code example.

```
from datetime import datetime

text = "2026-05-26"
d = datetime.strptime(text, "%Y-%m-%d")    # string -> datetime
print(d.month, d.year)                     # 5 2026
print(d > datetime.strptime("2026-01-01", "%Y-%m-%d")) # True
```

Common mistakes.

- Many beginners swap the argument order — the *string* comes first, then the *format*.
- Many beginners forget that a mismatched format raises `ValueError`; validate user-entered dates.

Quick Check. Which function turns “2026-05-26” into a datetime — `strftime` or `strptime`?
Answer: `strptime`. The **p** is for *parse* (string → datetime); the **f** is for *format* (datetime → string).

Filtering with comprehensions

The big idea. A list comprehension with an `if` clause keeps only the items that match a condition: `[x for x in items if condition]`. This is how you narrow a big list down to a relevant slice.

This is like when... you go through a stack of receipts and pull out only the ones from May — keeping the matches, discarding the rest.

The explanation.

You met comprehensions in Chapter 8. Adding `if condition` at the end filters. To get this month's expenses, you filter the full list to those whose parsed date matches the current month and year. The result is a smaller list you can then total or break down.

The code example.

```
expenses = [
    {"amount": 5000, "date": "2026-05-10", "category": "Food"},
    {"amount": 8000, "date": "2026-04-22", "category": "Bills"},
]
may = [e for e in expenses if e["date"].startswith("2026-05")]
print(len(may))      # 1
```

Common mistakes.

- Many beginners write a manual loop with `append` when a one-line comprehension is clearer. Use the comprehension for simple filters.
- Many beginners filter on the raw string when they need real date logic; for “this month” a `startswith("2026-05")` shortcut works, but for ranges, parse with `strptime` and compare.

Quick Check. What does `[n for n in [1,2,3,4] if n > 2]` produce?

Answer: `[3, 4]` — only the items where the condition is true.

`sum()` with generator expressions

The big idea. `sum(expr for item in iterable)` adds up a value computed from each item, in a single expression — no intermediate list needed. It is the cleanest way to total a column.

This is like when... you run a tape through an adding machine, feeding each receipt's amount in turn and reading the total at the end.

The explanation.

A *generator expression* looks like a list comprehension without the square brackets: `(e["amount"] for e in expenses)`. `sum()` consumes it and returns the total. You can combine it with a filter: `sum(e["amount"] for e in expenses if e["category"] == "Food")` totals just the food spending. Generators are memory-efficient because they produce one value at a time rather than building a whole list first.

The code example.

```
expenses = [{"amount": 5000}, {"amount": 8000}, {"amount": 3000}]
total = sum(e["amount"] for e in expenses)      # 16000
food = sum(e["amount"] for e in [
    {"amount": 5000, "cat": "Food"}, {"amount": 8000, "cat": "Bills"}
] if e["cat"] == "Food")                       # 5000
print(total, food)
```

Common mistakes.

- Many beginners build a list first (`sum([...])`) when the generator form (`sum(...)`) is leaner. Both work; drop the brackets.
- Many beginners call `sum()` on a list of dicts directly and crash — you must sum a specific numeric field with a generator.

Quick Check. What does `sum(n*n for n in [1,2,3])` give?

Answer: `14` — the sum of 1, 4, and 9.

Dictionary aggregation — totalling by category

The big idea. To break spending down by category, build a dictionary that maps each category to its running total. Walk the expenses once, adding each amount to its category's entry.

This is like when... you have labelled jars on a shelf — “Food,” “Transport,” “Bills” — and drop each coin into the matching jar. At the end, each jar's weight is that category's total.

The explanation.

Start with an empty dict. For each expense, use `dict.get(category, 0)` to fetch the current total (defaulting to 0 if the category is new), add the amount, and store it back. After the loop, the dict holds one total per category. To show percentages, divide each category total by the grand total.

The code example.

```
expenses = [
    {"amount": 5000, "category": "Food"},
    {"amount": 3000, "category": "Food"},
    {"amount": 8000, "category": "Bills"},
]
by_cat = {}
for e in expenses:
    cat = e["category"]
    by_cat[cat] = by_cat.get(cat, 0) + e["amount"]    # add to running total
print(by_cat)    # {'Food': 8000, 'Bills': 8000}
```

Common mistakes.

- Many beginners index `by_cat[cat] += amount` on a category that does not exist yet and crash with `KeyError`. Use `by_cat.get(cat, 0) + amount`, or `collections.defaultdict(int)`.

Quick Check. Why use `by_cat.get(cat, 0)` instead of `by_cat[cat]`?

Answer: The first time a category appears it has no entry yet; `.get(cat, 0)` returns 0 instead of raising `KeyError`.

Number formatting in f-strings

The big idea. Format specs make numbers read like money: `:`, adds thousands separators, `:.2f` fixes two decimals, and you can combine them as `:.2f`.

This is like when... a bank statement shows “1,250.00” not “1250.0” — the formatting makes the figure instantly readable.

The explanation.

You met `:.2f` in Chapter 2. The comma adds grouping: `{43000:,}` becomes `43,000`. Combine for currency: `{1234.5:,.2f}` becomes `1,234.50`. To right-align a number in a fixed width (useful for aligned report columns), prefix a width: `{x:>10,.2f}`.

The code example.

```
total = 43000
print(f"Total: {total:,.} RWF")    # Total: 43,000 RWF
price = 1234.5
print(f"Price: {price:,.2f}")    # Price: 1,234.50
print(f"{500:>10,.} RWF")    # right-aligned in width 10
```

Common mistakes.

- Many beginners write `:.2f` (missing the dot) — it must be `:.2f`.

Quick Check. What does `f"{1000000:,}"` produce?

Answer: `"1,000,000"` — commas grouping every three digits.

Guarding against empty data

The big idea. Reports do maths that breaks on empty input — dividing the total by the number of days, or by the grand total for percentages. Always check for zero before dividing.

This is like when... you split a bill among friends — but if nobody showed up, you cannot divide by zero people. You handle that case separately.

The explanation.

Before computing a daily average (`total / days`) or a percentage (`cat_total / grand_total * 100`), check that the denominator is non-zero. If it is zero — a month with no expenses — print “no data” rather than crashing. The conditional expression `value if condition else fallback` makes this a one-liner.

The code example.

```
total = 0
days = 26
avg = total / days if days else 0          # safe even if days were 0
grand = 0
pct = (5000 / grand * 100) if grand else 0 # safe when grand is 0
print(avg, pct)
```

Common mistakes.

- Many beginners compute percentages before checking the grand total is non-zero, and crash on an empty month.

Quick Check. What does `10 / x if x else 0` give when `x` is 0?

Answer: 0. The `else` branch runs because 0 is falsy, avoiding the division.

④ How the concepts connect

On start, `json.load` fills a list of expense dicts. To add an expense, you collect an amount, a category (validated against a fixed list), and a date (defaulting to today's `strftime`, or a user-entered string validated with `strftime`). For the monthly report, a comprehension filters the list to the current month; `sum()` with a generator gives the total; a loop builds a category-total dictionary with `.get(cat, 0)`; and f-strings with `:`, format every figure. Guards protect the daily-average and percentage maths from empty months.

⑤ The build walkthrough

Step 1 · Load, save, categories. Reuse the load/save JSON helpers from Chapter 6. Define a fixed tuple of allowed categories: Food, Transport, Entertainment, Bills, Other.

Step 2 · Add an expense. Prompt for amount (validate it is a positive number with `try / except ValueError`), category (validate it is in the allowed set, case-insensitively), and date (default to `datetime.now().strftime("%Y-%m-%d")` if blank, else validate the entered string with `strftime`). Append the dict and save.

Step 3 · View all. If empty, say so. Otherwise print each expense as an aligned row — date, category padded with `:<14`, amount formatted `:>10`,.

Step 4 · Filter by category. Prompt for a category, filter the list with a comprehension, and print the matches plus their subtotal from a `sum()` generator.

Step 5 · Monthly report. Filter to the current month with a comprehension. Compute the total with `sum()`. Build a category-total dict with the `.get(cat, 0)` pattern. Compute the daily average (guard the day count) and find the highest single expense with `max(month, key=lambda e: e["amount"])`. Print the total, daily average, and highest, then loop the category dict printing each total and its percentage of the month (guard the grand total).

⑥ Understanding the design

Three habits. First, *validate at the boundary* — amounts, categories, and dates are all checked the moment they are entered, so the rest of the program can trust every stored record. Second, *derive reports, never store them* — the monthly totals are computed fresh from the raw expenses each time, so they can never go stale. Third, *guard every division* — a report that crashes on an empty month is worse than useless; a single `if denominator else 0` makes it robust.

⑦ The exercise twist

Set a monthly budget per category, warn at 80%, and alert when exceeded.

What new concept does this need? A second data file — `budgets.json` — mapping each category to its monthly limit. The logic is comparison: for each category, compare this month's spending (which you already compute) against its budget.

How does it extend what you built? Add a “set budget” menu choice that writes to `budgets.json`. In the monthly report, after computing each category total, look up its budget and print a status: under budget, a warning at 80% or more, or an over-budget alert. A small text progress bar (you built one in Chapter 8) shows spending against the limit visually.

Hint for the hardest part. A category might have spending but no budget set, or a budget but no spending. Handle both with `.get(category, 0)` on each side, so a missing entry counts as zero rather than crashing.

⑧ What to remember from this chapter

- `datetime.strptime(text, fmt)` parses a string into a datetime — the mirror of `strftime`. String first, format second.
- Filter a list with `[x for x in items if condition]`; total a field with `sum(x["field"] for x in items)`.
- Aggregate by group with a dict and the `d[key] = d.get(key, 0) + value` pattern.
- Format money with `:,.2f` (thousands + two decimals); right-align in a column with `:>10`.
- Guard every division against a zero denominator with `value if denom else 0`.

⑨ Coming up next

In the next chapter you build a quiz game, shuffling questions with `random.shuffle`, normalising answers for comparison, and computing a final score as a percentage.

Quiz Game

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A multiple-choice quiz game. It holds a bank of questions across a few categories, shuffles them so no two playthroughs are identical, presents each with lettered options, checks the player's answer regardless of capitalisation, tracks the score, and reports a final percentage with a verdict. By the end you will have practised shuffling, normalising user input for fair comparison, and turning a raw score into a friendly result — the scoring loop behind every quiz, test, and survey.

② The concepts you need

- **A question bank as a list of dicts.** Reused shape from Chapters 7 and 9 — each question is a dict.
- **`random.shuffle()`.** Reordering a list in place so the quiz feels fresh each time.
- **Normalising input with `.strip().upper()`.** Making "a" and "A " count as the same answer.
- **`enumerate()` with `start`.** Numbering questions from 1 for display.
- **Score tracking.** A counter incremented on each correct answer.
- **Percentage and verdict.** Turning the score into a percentage and a message.

③ Concept deep dives

A question bank as a list of dicts

The big idea. Each question is a dictionary holding its text, its options, and the correct answer; the whole quiz is a list of those dictionaries. Separating data (the questions) from logic (the quiz loop) means you can add questions without touching the code.

This is like when... a teacher keeps a box of question cards. The quiz machinery — ask, check, score — stays the same no matter which cards are in the box.

The explanation.

You have used lists of dicts in Chapters 7 and 9. Here each dict has three keys: the question string, a list of option strings, and the correct option's letter (or index). Keeping the correct answer *in the dict* means the checking logic just compares the player's input against `question["answer"]` — no parallel lists to keep in sync.

The code example.

```
questions = [
    {
        "q": "What is the capital of Rwanda?",
        "options": ["Kigali", "Lagos", "Cairo", "Nairobi"],
        "answer": "A",
    },
    {
        "q": "What is 7 * 8?",
        "options": ["54", "56", "64", "49"],
        "answer": "B",
    },
]
print(questions[0]["q"])           # What is the capital of Rwanda?
print(questions[1]["answer"])     # B
```

Common mistakes.

- Many beginners keep options and answers in separate parallel lists, which drift out of sync. Keep them together in one dict.

Quick Check. Where is the correct answer for the first question stored?

Answer: In `questions[0]["answer"]` — inside that question's own dictionary.

`random.shuffle()` — reorder a list in place

The big idea. `random.shuffle(my_list)` randomly reorders the list *in place* — it changes the existing list and returns `None`. Use it to make the quiz present questions in a different order each play.

This is like when... you shuffle a deck of cards. The same cards, rearranged. The deck itself changes; you do not get a new deck.

The explanation.

Because `shuffle` works in place, you call it on the list and then use the list — you do *not* write `questions = random.shuffle(questions)`, which would set `questions` to `None`. If you need a shuffled copy while keeping the original order, use `random.sample(my_list, len(my_list))` instead, which returns a new shuffled list.

The code example.

```
import random

deck = [1, 2, 3, 4, 5]
random.shuffle(deck)           # reorders in place
print(deck)                   # e.g. [3, 1, 5, 2, 4]
# WRONG: deck = random.shuffle(deck) -> deck becomes None
```

Common mistakes.

- Many beginners write `x = random.shuffle(x)` and destroy their list (it becomes `None`). Call `shuffle(x)` on its own line.
- Many beginners shuffle but forget the change is permanent for that list; copy first if you need the original order.

Quick Check. What does `random.shuffle([1,2,3])` return?

Answer: `None`. It reorders in place; the return value is not the shuffled list.

Normalising input with `.strip().upper()`

The big idea. Players type answers inconsistently — "a", "A", " A ". Normalising both the input and the stored answer to the same form before comparing makes the check fair.

This is like when... a passport officer ignores whether you wrote your name in capitals or lowercase — they compare the normalised version against the record.

The explanation.

You met `.strip()` and case methods in Chapter 1. Here you chain `.strip().upper()` on the player's answer so leading spaces and lowercase letters do not cause a correct answer to be marked wrong. Store the correct answer in the same form (uppercase letters), and the comparison is a simple `==`.

The code example.

```
answer = " a "
clean = answer.strip().upper()    # "A"
correct = "A"
print(clean == correct)          # True
```

Common mistakes.

- Many beginners compare the raw input directly and mark "a" wrong when the answer is "A". Normalise both sides first.

Quick Check. What does `" b ".strip().upper()` produce?

Answer: "B" — spaces removed, letter capitalised.

enumerate() with start — lettered and numbered display

The big idea. `enumerate()` gives you a counter alongside each item. With `start=1` you number questions from 1; with a little arithmetic you can turn a 0-based index into option letters A, B, C, D.

This is like when... a printed exam numbers its questions and letters its choices — the structure that makes “answer 3B” unambiguous.

The explanation.

You met `enumerate` in Chapter 4. To label options A, B, C, D, enumerate the options list (default start 0) and convert each index to a letter with `chr(65 + i)` — 65 is the ASCII code for A, so `chr(65)` is "A", `chr(66)` is "B", and so on. To number questions for the player, enumerate the questions with `start=1`.

The code example.

```
options = ["Kigali", "Lagos", "Cairo", "Nairobi"]
for i, opt in enumerate(options):
    letter = chr(65 + i)          # 0->A, 1->B, 2->C, 3->D
    print(f" {letter}. {opt}")
```

Common mistakes.

- Many beginners hard-code the letters A, B, C, D in a parallel list. Generating them with `chr(65 + i)` adapts automatically to any number of options.

Quick Check. What does `chr(65 + 2)` return?

Answer: "C". 65 is A, so 65 + 2 is the code for C.

Score tracking and the final percentage

The big idea. Keep a `score` counter starting at 0, add 1 for each correct answer, and at the end convert it to a percentage with `round(score / total * 100)`. A few thresholds turn the percentage into a verdict.

This is like when... a teacher tallies correct answers, then converts the raw count into a grade out of 100 and writes an encouraging note.

The explanation.

You met counters in Chapter 3 and percentage maths in Chapter 7. Initialise `score = 0` before the question loop. After checking each answer, `score += 1` only when correct. After the loop, compute the percentage (the total is `len(questions)`, which is non-zero as long as the bank is not empty) and print a verdict using an `if / elif / else` on the percentage.

The code example.

```
score, total = 8, 10
pct = round(score / total * 100)      # 80
if pct == 100: verdict = "Perfect!"
elif pct >= 50: verdict = "Good job!"
else: verdict = "Keep practising!"
print(f"You scored {score}/{total} ({pct}%) – {verdict}")
```

Common mistakes.

- Many beginners increment the score outside the correctness check, inflating every result. `score += 1` belongs *inside* the “correct” branch only.

Quick Check. With 3 correct out of 4, what does `round(3/4*100)` give?

Answer: 75.

④ How the concepts connect

The question bank is a list of dicts. `random.shuffle` reorders it so each play differs. A `for i, question in enumerate(bank, start=1)` loop presents each one, printing its options A–D with `chr(65 + j)`. The player’s answer is normalised with `.strip().upper()` and compared to the stored answer; on a match, `score += 1`. After the loop, `round(score / len(bank) * 100)` gives the percentage, and an `if / elif / else` prints the verdict.

⑤ The build walkthrough

Step 1 · Build the bank. Define a list of at least ten question dicts across two or three categories, each with `q`, `options`, and `answer` (the correct letter). Keeping it in the file is fine for now; the twist moves it to JSON.

Step 2 · Optionally pick a category. Offer the player a category choice at the start. Filter the bank with a comprehension to the chosen category, or use all of them for “mixed.”

Step 3 · Shuffle and loop. Call `random.shuffle` on the chosen list. Initialise `score = 0`. Loop with `enumerate(bank, start=1)`, printing the question number, the question text, and the options lettered with `chr(65 + j)`.

Step 4 · Read and check. Prompt for the answer, normalise with `.strip().upper()`, and validate it is one of the valid letters (re-prompt on junk). Compare to the question’s stored answer; print “Correct!” and `score += 1`, or “Wrong — the answer was X.”

Step 5 · Final score. After the loop, compute the percentage (guarding against an empty bank) and print the score, percentage, and a verdict chosen by an `if / elif / else` on the percentage.

⑥ Understanding the design

Three habits. First, *data lives in dicts, logic lives in the loop* — adding a question never means touching the quiz machinery. Second, *generate option letters rather than hard-coding them* — `chr(65 + i)` works for two options or six, and never drifts out of sync with the options list. Third, *normalise both sides before comparing* — the fairness of the whole quiz rests on “a” and “A” being treated as equal, so the normalisation is not a nicety but the core of correct scoring.

⑦ The exercise twist

Load questions from an external JSON file, and save high scores per category.

What new concept does this need? Nothing new — it combines JSON (Chapter 6) with what you built here. The point is *separating data from code*: the questions move out of the `.py` file into a `questions.json` file you load at startup.

How does it extend what you built? Replace the in-file question list with a `json.load` of `questions.json`. After each game, load a `highscores.json`, compare the new score against the stored best for that category, update if it is higher, and save. Print the player’s best alongside their current score.

Hint for the hardest part. Design the JSON shape before coding. A natural layout is a top-level object keyed by category, each holding a list of question objects. Sketch two questions by hand in the file first, then write the loader to match — designing the data first makes the code obvious.

⑧ What to remember from this chapter

- Store each question as a dict with its text, options, and correct answer; the quiz is a list of those dicts.
- `random.shuffle(my_list)` reorders in place and returns `None` — never `x = random.shuffle(x)`.
- Normalise the player’s answer with `.strip().upper()` and compare against an answer stored the same way.
- Generate option letters with `chr(65 + i)` rather than hard-coding A, B, C, D.
- Track `score` with a counter incremented only inside the correct branch; convert to `round(score/total*100)` at the end.

⑨ Coming up next

In the next chapter you build Hangman, meeting **sets** for tracking guessed letters without duplicates, **random.choice** for picking a word, and multi-line strings for the ASCII art gallows.

Hangman

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

The classic Hangman word game. The computer picks a secret word and shows it as a row of blanks. You guess letters one at a time; correct guesses fill in the blanks, wrong ones add a piece to an ASCII gallows. Guess the word before the drawing is complete and you win; run out of attempts and you lose. By the end you will have met **sets** — the perfect tool for tracking which letters have been guessed — and learned to assemble a display from a secret string and a set of revealed letters.

② The concepts you need

- **Sets.** A collection with no duplicates and instant membership testing.
- **`random.choice()`.** Reused from earlier — pick a random word from a list.
- **Building the masked word.** Showing a letter if it has been guessed, a blank otherwise.
- **Membership testing with `in`.** Checking whether a letter is in the word or already guessed.
- **Multi-line strings and a frame list.** Storing the ASCII gallows stages.
- **The win/lose loop.** Ending when the word is complete or attempts run out.

③ Concept deep dives

Sets — no duplicates, instant lookup

The big idea. A **set** is a collection that automatically discards duplicates and tests membership (`x in my_set`) almost instantly. It is ideal for “have I seen this already?” questions.

This is like when... you keep a guest list where writing the same name twice changes nothing, and checking whether someone is on the list is a single glance, not a scan down every row.

The explanation.

You write a set with curly braces — `{"a", "b"}` — or build an empty one with `set()` (not `{}`, which makes an empty *dict*). Adding an element that is already present does nothing. Sets are unordered: you cannot index them like a list, but you can loop over them, test membership with `in`, and add with `.add()`.

For Hangman, the set of *guessed letters* is exactly right: guessing the same letter twice should not penalise the player, and checking “did they already guess this?” should be instant.

The code example.

```
guessed = set()                # empty set (NOT {})
guessed.add("e")
guessed.add("a")
guessed.add("e")               # duplicate ignored
print(guessed)                 # {'a', 'e'}
print("e" in guessed)         # True
print(len(guessed))           # 2
```

Common mistakes.

- Many beginners write `guessed = {}` for an empty set — that creates an empty *dictionary*. Use `set()`.
- Many beginners try `guessed[0]` to index a set — sets are unordered and have no index. Loop or test with `in`.

Quick Check. What does `len({"a", "a", "b"})` return?

Answer: 2. The duplicate "a" is collapsed; the set has two distinct elements.

`random.choice()` — pick a word

The big idea. `random.choice(my_list)` returns one randomly selected element from the list. Use it to choose the secret word from your word bank.

This is like when... you draw one ticket from a hat. Every ticket equally likely; one comes out.

The explanation.

You met `random.choice` in passing earlier; here it is the star. Give it a list of words and it returns one. (For Hangman, ordinary `random` is fine — there is nothing security-sensitive, so you do not need `secrets` from Chapter 8.) Lowercase the chosen word so all your letter comparisons can assume one case.

The code example.

```
import random

words = ["python", "rwanda", "rocket", "galaxy"]
secret = random.choice(words).lower()
print(secret)                # e.g. "rocket"
```

Common mistakes.

- Many beginners pass a string instead of a list and get a single random *character*, since `choice` works on any sequence. Pass the list of words.

Quick Check. What does `random.choice("hi")` return?

Answer: Either "h" or "i" — a string is a sequence of characters, so `choice` picks one character.

Building the masked word

The big idea. The display is computed fresh each turn: for every letter of the secret word, show the letter if it is in the guessed set, otherwise show an underscore. Join the result into a string.

This is like when... a game-show board reveals letters as they are guessed and keeps the rest hidden behind blanks — the board is redrawn after every guess.

The explanation.

A comprehension (Chapter 8) does this cleanly: for each character in the secret, choose the character or an underscore based on membership in the guessed set, then join with spaces. The conditional expression `value_if_true if condition else value_if_false` selects between the two.

The code example.

```
secret = "rocket"
guessed = {"r", "o", "t"}
shown = " ".join(c if c in guessed else "_" for c in secret)
print(shown)                # r o _ _ t
```

Common mistakes.

- Many beginners rebuild the word by mutating a list of characters and lose track of which positions are revealed. Recomputing from the secret + guessed set each turn is simpler and bug-free.

Quick Check. With secret "cat" and guessed {"a"}, what does the masked display show?

Answer: _ a _.

Detecting a win and a wrong guess

The big idea. The player wins when every letter of the secret has been guessed — that is, when the set of the word's letters is a subset of the guessed set. A guess is wrong when the letter is not in the secret at all.

This is like when... you tick off a checklist; you are done when every required box is ticked, regardless of order.

The explanation.

Convert the secret to a set of its unique letters with `set(secret)`. The player has won when `set(secret) <= guessed` (every letter of the word is in the guessed set — `<=` is the subset test for sets). For each guess, if the letter is not in the secret, decrement the remaining attempts; sets make both checks fast and clear.

The code example.

```
secret = "cat"
guessed = {"c", "a", "t"}
print(set(secret) <= guessed)    # True - all letters guessed: a win
print("z" in secret)             # False - a wrong guess
```

Common mistakes.

- Many beginners check the win by comparing the masked string to the secret, which works but is clumsier than the subset test on sets.

Quick Check. What does `set("ll") <= {"l"}` return?

Answer: True. `set("ll")` is `{"l"}`, which is a subset of `{"l"}`.

Multi-line strings and a frame list

The big idea. Store each stage of the gallows drawing as a multi-line string, and keep all the stages in a list indexed by the number of wrong guesses. Print the stage matching the current number of mistakes.

This is like when... a flip-book of drawings — each page is one frame, and you turn to the page that matches how far the game has progressed.

The explanation.

A triple-quoted string (Chapter 1) can span several lines, preserving the ASCII art exactly as you type it. Put one per stage in a list, ordered from empty gallows (0 wrong) to complete figure (6 wrong). Then `stages[wrong]` is the drawing for the current number of mistakes — a clean index lookup instead of a tangle of conditionals.

The code example.

```
stages = [
    """
    +---+
    |
    |
    =====
    """,
    """
    +---+
    0  |
    |
    =====
    # ... more stages ...
]
wrong = 1
print(stages[wrong])    # prints the one-mistake drawing
```

Common mistakes.

- Many beginners build the drawing with conditionals (`if wrong == 1: ...`). A list indexed by `wrong` is far cleaner and guarantees the stages stay in order.

Quick Check. If `stages` has 7 entries (indices 0–6), what is the maximum number of wrong guesses it supports?

Answer: 6. Index 6 is the seventh and final (full gallows) drawing.

④ How the concepts connect

`random.choice` picks the secret word. A set `guessed` starts empty. Each turn, `stages[wrong]` draws the gallows, and a comprehension builds the masked display from the secret and `guessed`. The player enters a letter; you validate it is a single new letter, add it to `guessed`, and if it is not in `secret`, increment `wrong`. The loop ends when `set(secret) <= guessed` (win) or `wrong` reaches the last stage (lose).

⑤ The build walkthrough

Step 1 · Word and stages. Define a list of at least twenty words and the list of ASCII gallows stages (0 through 6 wrong). Pick the secret with `random.choice(...).lower()`.

Step 2 · Game state. Create an empty `guessed = set()` and `wrong = 0`. The number of allowed mistakes is `len(stages) - 1`.

Step 3 · The turn loop. Open a `while True:`. Print the current gallows stage, the masked word (comprehension), the sorted guessed letters, and the attempts remaining.

Step 4 · Read and validate a guess. Prompt for a letter; normalise with `.strip().lower()`. Reject anything that is not exactly one alphabetic character, and warn (without penalty) if the letter was already guessed. Otherwise add it to `guessed`.

Step 5 · Apply the guess. If the letter is not in the secret, increment `wrong`. Then check the end conditions: if `set(secret) <= guessed`, print a win message and break; if `wrong` equals the maximum, print the final gallows, reveal the word, and break.

⑥ Understanding the design

Three habits. First, *recompute the display from state* — rebuilding the masked word each turn from the secret and the guessed set is simpler and less bug-prone than mutating a display string. Second, *use a set for guessed letters* — duplicates handled for free, membership instant, and the win check becomes a one-line subset test. Third, *index art by a counter* — `stages[wrong]` replaces a ladder of conditionals and keeps the frames provably in order.

⑦ The exercise twist

Add word categories, difficulty levels, and a hint system that reveals a letter at the cost of one attempt.

What new concept does this need? Nothing new — it recombines tools you have. Categories are a dict mapping a category name to a word list. Difficulty maps to word length (filter the list with a comprehension). The hint reveals an unguessed letter — pick one from `set(secret) - guessed` and add it to `guessed` while incrementing `wrong`.

How does it extend what you built? Add a category and difficulty prompt before choosing the word, and a special "hint" input inside the turn loop. The hint draws a letter the player has not found yet and pays for it with one attempt.

Hint for the hardest part. The set difference `set(secret) - guessed` gives exactly the letters still hidden. Pick one with `random.choice(list(...))` — convert to a list first, because `random.choice` needs a sequence, not a set.

⑧ What to remember from this chapter

- A set holds unique items with instant `in` testing; build an empty one with `set()`, not `{}`.
- `random.choice(my_list)` picks one random element; pass a *list* of words, not a string.
- Recompute the masked word each turn: `" ".join(c if c in guessed else "_" for c in secret)`.
- Win check is the subset test `set(secret) <= guessed`; a wrong guess is `letter not in secret`.
- Store ASCII art stages in a list and index it by the number of wrong guesses.

⑨ Coming up next

In the next chapter you build Rock Paper Scissors with running win/loss statistics, practising game-logic conditionals and a sentinel-controlled loop that ends when the player types “quit.”

Rock Paper Scissors

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

Rock Paper Scissors against the computer, with running statistics. Each round you choose rock, paper, or scissors; the computer picks randomly; the program applies the rules, announces the winner, and updates a tally of your wins, losses, and draws. Type “quit” and it prints your final record with win percentages. By the end you will have practised encoding game rules cleanly, keeping multiple counters in step, and ending a loop on a **sentinel value** — a special input that means “stop.”

② The concepts you need

- **random.choice() on a list.** Reused — the computer’s move.
- **Encoding the rules.** A clean way to decide who beats whom.
- **Counting with multiple variables (or a dict).** Tracking wins, losses, draws.
- **String normalisation.** Reused — accept “Rock”, “rock”, “ROCK”.
- **Sentinel-controlled loops.** Ending when the player types a special “quit” value.
- **Win percentage.** Reused percentage maths, guarded against zero rounds.

③ Concept deep dives

random.choice() on a list — the computer’s move

The big idea. `random.choice(["rock", "paper", "scissors"])` picks the computer’s move uniformly at random. Each option is equally likely every round.

This is like when... the computer rolls a three-sided die before each round — no strategy, no memory, just a fair pick.

The explanation.

You used `random.choice` in Chapter 11 to pick a word; here it picks one of three moves. Define the three options as a list once, and call `choice` on it each round. Ordinary `random` is appropriate — there is nothing to secure.

The code example.

```
import random

moves = ["rock", "paper", "scissors"]
computer = random.choice(moves)
print(computer)                # e.g. "paper"
```

Common mistakes.

- Many beginners rebuild the list inside the loop on every round — harmless, but define it once above the loop for clarity.

Quick Check. Over many rounds, roughly what fraction of the time should the computer pick “rock”?

Answer: About one third — each of the three moves is equally likely.

Encoding the rules cleanly

The big idea. Rather than a tangle of `if` conditions for every move pairing, encode “what beats what” in a single dictionary: each move maps to the move it defeats. One lookup decides the round.

This is like when... a referee carries a small rulebook: “rock beats scissors, scissors beats paper, paper beats rock.” Three lines settle every dispute.

The explanation.

Build `beats = {"rock": "scissors", "paper": "rock", "scissors": "paper"}`. If the player and computer chose the same move, it is a draw. Otherwise, the player wins exactly when `beats[player] == computer` — that is, the computer’s move is the one the player’s move defeats. Everything else is a loss. This replaces six or more `if` branches with one dictionary and two comparisons.

The code example.

```
beats = {"rock": "scissors", "paper": "rock", "scissors": "paper"}
player, computer = "rock", "scissors"
if player == computer:
    result = "draw"
elif beats[player] == computer:
    result = "win"
else:
    result = "loss"
print(result)                                # win
```

Common mistakes.

- Many beginners write out every pairing as a separate `if`, which is long and easy to get wrong. The `beats` dict captures all the logic in three entries.

Quick Check. With the `beats` dict, how do you tell if the player won?

Answer: The player won if `beats[player] == computer` (and it was not a draw).

Counting with multiple variables

The big idea. Keep three counters — wins, losses, draws — each starting at 0, and increment the right one after each round. A dictionary `{"win": 0, "loss": 0, "draw": 0}` keeps them tidy.

This is like when... a scorekeeper at a match marks a tally under the correct column after every point.

The explanation.

You met counters in Chapter 3. Here you have three of them. Using a dict keyed by result lets you increment with one line — `stats[result] += 1` — instead of an `if/elif/else` that picks which variable to bump. Since `result` is already “win”, “loss”, or “draw”, it doubles as the dict key.

The code example.

```
stats = {"win": 0, "loss": 0, "draw": 0}
for result in ["win", "win", "loss", "draw"]:
    stats[result] += 1
print(stats)                                # {'win': 2, 'loss': 1, 'draw': 1}
```

Common mistakes.

- Many beginners keep three separate variables and write a branch to choose which to increment. Keying a dict by the result string is shorter and less error-prone.

Quick Check. Why can `result` be used directly as the dict key?

Answer: Because `result` is already one of “win”, “loss”, “draw” — the exact keys the stats dict uses.

Sentinel-controlled loops

The big idea. A **sentinel** is a special input value that signals “stop.” The game loops forever reading moves until the player types the sentinel — here, “quit” — at which point the loop breaks.

This is like when... a turnstile keeps admitting people until someone inserts the special “closing” token, which shuts it down.

The explanation.

You have used `while True` with `break` since Chapter 3. The sentinel pattern is the same loop, where one specific input triggers the break. Read the player's move, normalise it, and if it equals the sentinel, print the final stats and break. Otherwise validate it is a real move and play the round.

The code example.

```
while True:
    move = input("rock/paper/scissors (or quit): ").strip().lower()
    if move == "quit":
        print("Final stats here")
        break
    if move not in ("rock", "paper", "scissors"):
        print("Not a valid move.")
        continue
    # ... play the round ...
```

Common mistakes.

- Many beginners check the sentinel after validating the move, so "quit" gets rejected as "not a valid move." Check the sentinel *first*.

Quick Check. Why check for "quit" before checking the move is valid?

Answer: Otherwise "quit" fails the valid-move test and is rejected as junk instead of ending the game.

Win percentage, guarded

The big idea. At the end, report each result as a percentage of total rounds: `round(wins / total * 100)`. Guard against a player who quits before playing a single round (total zero).

This is like when... a sports summary shows "won 60%, lost 30%, drew 10%" — but if no games were played, it shows "no games" rather than dividing by zero.

The explanation.

You met guarded percentages in Chapters 7 and 9. The total is the sum of the three counters. If it is zero, print "no rounds played." Otherwise compute each percentage and display the record. The same value if `total else 0` guard applies.

The code example.

```
stats = {"win": 1, "loss": 1, "draw": 0}
total = sum(stats.values()) # 2
if total == 0:
    print("No rounds played.")
else:
    print(f"Wins: {round(stats['win']/total*100)}%") # 50%
```

Common mistakes.

- Many beginners compute the total as only wins + losses, forgetting draws, so the percentages do not add to 100.

Quick Check. How do you total all three counters from the stats dict in one call?

Answer: `sum(stats.values())` — adds every value in the dictionary.

④ How the concepts connect

A `beats` dict encodes the rules and a `stats` dict holds the three counters. A `while True` sentinel loop reads the player's move, normalised with `.strip().lower()`. If it is "quit", the loop prints the final percentages (guarded against zero rounds) and breaks. Otherwise the move is validated, `random.choice` gives the computer's move, the draw/win/loss decision uses `beats`, the matching counter is incremented with `stats[result] += 1`, and a running tally is shown.

⑤ The build walkthrough

Step 1 · Setup. Define the moves list, the `beats` dict, and the `stats` dict initialised to three zeros.

Step 2 · The sentinel loop. Open `while True:`. Read the move, normalise with `.strip().lower()`. If it is "quit", break out (the final report comes after the loop). If it is not one of the three valid moves, print a warning and `continue`.

Step 3 · Play the round. Pick the computer's move with `random.choice(moves)`. Decide the result: draw if equal, win if `beats[player] == computer`, else loss. Print both moves and the outcome.

Step 4 · Update and show the tally. Increment `stats[result]`. Print the running record (e.g. "2W 1L 0D") so the player sees their standing each round.

Step 5 · Final report. After the loop, compute the total with `sum(stats.values())`. If zero, say no rounds were played. Otherwise print each result with its count and percentage, plus the total rounds.

⑥ Understanding the design

Three habits. First, *encode rules as data, not branches* — the `beats` dict turns the entire game logic into one lookup, which is shorter, clearer, and trivially extensible (the twist adds Lizard and Spock by adding entries). Second, *key counters by the result string* — because `result` is already a valid key, you avoid an `if/elif/else` just to pick which counter to bump. Third, *check the sentinel before validation* — order matters, or your quit command gets rejected as invalid input.

⑦ The exercise twist

Add an AI mode where the computer learns your patterns, and a Rock-Paper-Scissors-Lizard-Spock variant.

What new concept does this need? For the variant, only more data: extend the `beats` dict so each move maps to the *two* moves it defeats (a list), and adjust the win test to check membership. For the learning AI, track the frequency of the player's past moves in a dict and have the computer play the counter to the player's most frequent move.

How does it extend what you built? The Lizard-Spock variant is almost free thanks to the data-driven rules — you change the `beats` structure and the win check, not the loop. The learning AI adds a `Counter` of the player's moves (Chapter 5) and picks the computer's move to beat the most common one.

Hint for the hardest part. For the learning AI, decide what move beats the player's predicted move by inverting your `beats` mapping: if you predict the player will throw rock, you want the move whose `beats` value is rock — paper. Build a reverse lookup once, and the AI's choice is a single dictionary access.

⑧ What to remember from this chapter

- `random.choice(moves)` picks the computer's move from a list of options.
- Encode game rules as data: `beats = {"rock": "scissors", ...}`; the player wins when `beats[player] == computer`.
- Key counters by the result string and increment with `stats[result] += 1` — no branching needed.
- A sentinel loop runs until a special input ("quit"); check the sentinel *before* validating the move.
- Total the counters with `sum(stats.values())` and guard the percentage maths against zero rounds.

⑨ Coming up next

In the next chapter you take a big step: you build a student grade manager using **classes** — your first object-oriented program, where data and the functions that act on it live together.

Student Grade Manager

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A grade manager built with **classes** — your first object-oriented program. You define what a “Student” is: a name, an ID, and a list of grades, together with the things a student can do — add a grade, compute their average, work out their letter grade, and describe themselves. You then manage many Student objects in one program. By the end you will understand the single most important idea in modern programming: bundling data and the functions that act on it into one tidy unit called an object.

② The concepts you need

- **Classes and objects.** A class is a blueprint; an object is a thing built from it.
- **`__init__` and `self`.** The constructor that sets up a new object, and the name it uses for itself.
- **Attributes.** The data each object carries.
- **Methods.** Functions that belong to a class and act on its object.
- **`__str__`.** A special method controlling how an object prints.
- **`statistics.mean()`.** A standard-library helper, with the manual version for understanding.
- **Managing many objects.** Keeping objects in a dict keyed by ID.

③ Concept deep dives

Classes and objects — blueprint and thing

The big idea. A **class** is a blueprint that describes a kind of thing. An **object** (or *instance*) is a specific thing built from that blueprint. One **Student** class; many student objects, each with its own name and grades.

This is like when... an architect draws one house blueprint, and a builder uses it to put up a whole street of houses. The blueprint is the class; each actual house is an object. They share a design but have their own address and contents.

The explanation.

You define a class with the **class** keyword and a capitalised name. Inside, you describe what every object of that type will have (its data) and what it can do (its methods). To create an object, you call the class like a function: `alice = Student("Alice", "S001")`. Now `alice` is one object; you can make as many more as you like, each independent.

This is a shift from everything before. Until now your data (a dict of grades) and your functions (compute average) lived apart. A class brings them together: the grades and the “compute average” behaviour live *inside* each student.

The code example.

```
class Student:                # define the blueprint
    pass                       # empty for now

alice = Student()             # build one object from it
bob = Student()               # build another, independent object
print(type(alice))            # <class '__main__.Student'>
print(alice is bob)           # False – two distinct objects
```

Common mistakes.

- Many beginners forget the parentheses when creating an object: `alice = Student` stores the *class itself*, not an object. You need `Student()`.

Quick Check. What is the difference between the `Student` class and the `alice` object?

Answer: `Student` is the blueprint; `alice` is one specific thing built from it. One class can produce many objects.

`__init__` and `self` — setting up a new object

The big idea. `__init__` is a special method that runs automatically when you create an object. It sets up the object's starting data. `self` is the name the object uses to refer to itself inside its own methods.

This is like when... a new employee fills out their starter form on day one — name, ID, desk. `__init__` is that form; `self` is the employee referring to “my own” details.

The explanation.

You define `def __init__(self, ...):` inside the class. The first parameter is always `self` — Python passes the new object in automatically, so you never pass it yourself. Inside, you attach data to the object by assigning to `self.something`: `self.name = name`, `self.grades = []`. Those become the object's **attributes**, readable later as `alice.name`.

The double underscores around `init` mark it as a “dunder” (double-underscore) method — a name Python recognises specially. You do not call `__init__` yourself; Python calls it when you write `Student("Alice", "S001")`.

The code example.

```
class Student:
    def __init__(self, name, student_id): # runs on creation
        self.name = name                 # attach data to the object
        self.student_id = student_id
        self.grades = []                 # every student starts with none

alice = Student("Alice", "S001")         # __init__ runs now
print(alice.name)                       # Alice
print(alice.grades)                     # []
```

Common mistakes.

- Many beginners forget `self` as the first parameter, or forget the `self.` prefix when storing data — then the data vanishes when `__init__` ends.
- Many beginners try to call `__init__` directly. Never do; creating the object (`Student(...)`) calls it for you.

Quick Check. Why does every method's first parameter need to be `self`?

Answer: So the method knows *which object* it is working on — `self` is that specific object, passed in automatically.

Methods — functions that belong to a class

The big idea. A **method** is a function defined inside a class. It always takes `self` first, so it can read and change that object's attributes. Methods are how an object *does* things.

This is like when... a microwave has buttons. “Start,” “Stop,” “Add 30 seconds” — each button is a method that acts on that particular microwave's state.

The explanation.

You define methods exactly like functions, indented inside the class, with `self` as the first parameter. Inside, you use `self.attribute` to read or change the object's data. You call a method on an object with a dot: `alice.add_grade(85)`. Python passes `alice` in as `self` behind the scenes, so you supply only the other arguments.

A method that changes the object (like `add_grade`) usually returns nothing; a method that computes something (like `get_average`) returns a value.

The code example.

```

class Student:
    def __init__(self, name):
        self.name = name
        self.grades = []

    def add_grade(self, grade):           # a method
        self.grades.append(grade)       # change this object's data

    def get_average(self):               # a computing method
        if not self.grades:
            return 0
        return sum(self.grades) / len(self.grades)

alice = Student("Alice")
alice.add_grade(80)
alice.add_grade(90)
print(alice.get_average())              # 85.0

```

Common mistakes.

- Many beginners call a method without the object: `add_grade(85)` fails; it must be `alice.add_grade(85)`.
- Many beginners forget to guard `get_average` against an empty grades list and crash with `ZeroDivisionError`.

Quick Check. When you write `alice.add_grade(85)`, what becomes `self` inside the method?
Answer: `alice` — the object you called the method on.

`__str__` — how an object prints

The big idea. `__str__` is a dunder method that returns a string describing the object. Define it, and `print(alice)` shows your friendly description instead of a cryptic memory address.

This is like when... you give someone a name tag. Without it, strangers see only a faceless person; with it, they read “Alice, Student S001” at a glance.

The explanation.

By default, `print(alice)` shows something like `<__main__.Student object at 0x7f...>` — useless. Define `def __str__(self): return ...` to control it. Whatever string you return is what `print` and `str()` show. A close relative, `__repr__`, is the developer-facing version shown in the interactive shell; for now, `__str__` is the one you need.

The code example.

```

class Student:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self.grades = []

    def __str__(self):
        return f"{self.name} ({self.student_id}) - {len(self.grades)} grades"

alice = Student("Alice", "S001")
print(alice)                          # Alice (S001) - 0 grades

```

Common mistakes.

- Many beginners `print` inside `__str__` instead of `returning` a string. It must *return* the string; `print` returns `None` and breaks `str()`.

Quick Check. What must `__str__` do with its description — print it or return it?
Answer: Return it. Python’s `print` and `str()` use the returned string.

statistics.mean() and the manual version

The big idea. The `statistics` module provides `mean()`, `median()`, and more, so you do not reinvent them. But knowing the manual version — `sum(x) / len(x)` — keeps the concept clear.

This is like when... you own a calculator but still understand long division — the tool is faster, the understanding keeps you in control.

The explanation.

After `import statistics`, `statistics.mean(grades)` returns the average. It raises an error on an empty list, so guard it. The manual equivalent is `sum(grades) / len(grades)`. For this project either is fine; use `mean` for brevity once you understand what it does. Letter grades come from an `if/elif/else` on the average: 90+ is A, 80+ B, and so on.

The code example.

```
import statistics

grades = [85, 92, 78]
print(statistics.mean(grades))           # 85.0
print(sum(grades) / len(grades))         # 85.0 — same thing, by hand
```

Common mistakes.

- Many beginners call `statistics.mean([])` on an empty list and crash. Guard with `if not grades: return 0` (or “N/A”).

Quick Check. What is the manual equivalent of `statistics.mean(xs)`?

Answer: `sum(xs) / len(xs)` — total divided by count.

Managing many objects

The big idea. Keep your `Student` objects in a dictionary keyed by student ID. The dict gives instant lookup by ID; each value is a full `Student` object with its own data and methods.

This is like when... a school office files student records in cabinets indexed by ID number — pull the drawer, get the whole folder.

The explanation.

You met dicts in Chapter 6. Here the values are *objects* rather than plain dicts. `students[sid] = Student(name, sid)` files a new student; `students[sid].add_grade(85)` reaches into the stored object and calls its method. The class `average` loops over `students.values()`, collecting each student’s average. Objects-in-a-dict is one of the most common structures in real programs.

The code example.

```
students = {}
students["S001"] = Student("Alice", "S001")
students["S001"].add_grade(85)
print(students["S001"].get_average())    # 85.0
for s in students.values():               # loop over all students
    print(s)                              # uses __str__
```

Common mistakes.

- Many beginners store the data as a plain dict and lose the methods. Store the *object* so its behaviour travels with its data.

Quick Check. What does `students.values()` give you when the values are `Student` objects?

Answer: The `Student` objects themselves — each with its own attributes and methods.

④ How the concepts connect

A `Student` class defines `__init__` (storing name, ID, and an empty grades list), `add_grade` (validating and appending), `get_average` (guarded mean), `get_letter_grade` (an `if/elif/else` on the average),

and `__str__` (a readable summary). The program keeps a dict of students keyed by ID. Menu choices create students, call methods on the stored objects, and loop over `students.values()` for the class average. The data (grades) and the behaviour (averaging, grading) live together inside each object.

⑤ The build walkthrough

Step 1 · Define the class. Write `class Student:` with `__init__(self, name, student_id)` storing `self.name`, `self.student_id`, and `self.grades = []`.

Step 2 · Add the methods. `add_grade(self, grade)` validates the grade is 0–100 then appends it. `get_average(self)` returns 0 (or “N/A”) on no grades, else the mean. `get_letter_grade(self)` maps the average to A/B/C/D/F with `if/elif/else`. `__str__(self)` returns a summary line.

Step 3 · The student store. Create an empty `students = {}` dict. Write a menu loop offering: add student, add grade, view student, view all, class average, quit.

Step 4 · Wire the menu to methods. “Add student” reads name and ID and stores a new `Student` object. “Add grade” looks up the student by ID (handle missing) and calls `add_grade`. “View student” prints the object and its average and letter grade. “View all” loops `students.values()`.

Step 5 · Class average. Collect each student’s `get_average()` into a list (skip students with no grades), then report the mean of those — guarded against the empty case.

⑥ Understanding the design

Three habits. First, *data and behaviour belong together* — putting `get_average` inside `Student` means you can never have a student whose averaging logic lives somewhere else and drifts out of sync. Second, *guard inside the methods* — `get_average` handles the no-grades case itself, so every caller is safe without repeating the check. Third, *store objects, not raw dicts* — keeping `Student` objects in your dictionary means each record carries its own methods, which is the entire point of object orientation.

⑦ The exercise twist

Add GPA on a 4.0 scale, and a `GraduateStudent` subclass with a thesis grade.

What new concept does this need? GPA is just another method mapping letter grades to points (A=4.0, B=3.0, ...) and averaging them. The subclass is a preview of inheritance — the full topic of Chapter 14 — where `GraduateStudent` reuses everything in `Student` and adds a thesis grade.

How does it extend what you built? Add a `get_gpa()` method to `Student`. For the subclass, define `class GraduateStudent(Student):` and add the thesis handling; the next chapter explains exactly how that reuse works.

Hint for the hardest part. Map letter grades to points in a dict (`{"A": 4.0, "B": 3.0, ...}`) and compute the GPA as the mean of the points. Reusing your existing `get_letter_grade` per grade keeps the logic in one place.

⑧ What to remember from this chapter

- A class is a blueprint; an object is a thing built from it with `ClassName(...)`.
- `__init__(self, ...)` runs on creation and attaches data via `self.attribute = ....`
- Methods are functions inside a class; the first parameter is always `self`, the object itself.
- `__str__(self)` must *return* a string; it controls how the object prints.
- Store objects in a dict keyed by an ID so their data and methods travel together.

⑨ Coming up next

In the next chapter you extend classes with **inheritance**: a `LibraryItem` base class with `Book` and `Magazine` subclasses that reuse its code and override what they need.

Library Book System

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A library system that tracks different kinds of items — books and magazines — using **inheritance**. You define one base class, `LibraryItem`, holding everything every item shares (title, author, availability, checkout, return). Then `Book` and `Magazine` *inherit* from it, reusing all that shared code and adding only what makes them special. By the end you will understand the second pillar of object orientation: building specialised types on top of a shared base, without copying code.

② The concepts you need

- **Inheritance.** A subclass automatically gets the attributes and methods of its parent.
- **`super().__init__()`.** Calling the parent’s constructor from the child.
- **Method overriding.** Replacing a parent method with a more specific version in the child.
- **Polymorphism.** Treating different object types uniformly through a shared method name.
- **`isinstance()`.** Checking what type an object is.
- **A collection of mixed objects.** Storing books and magazines together in one list.

③ Concept deep dives

Inheritance — a child class reuses its parent

The big idea. **Inheritance** lets one class (the *child* or *subclass*) automatically receive all the attributes and methods of another (the *parent* or *base class*). The child can then add new things or change inherited ones — without recopying the shared code.

This is like when... a chef teaches a base recipe for bread, and apprentices learn variations — sourdough, focaccia — that reuse the whole base method and only change a step or two. They do not relearn how to knead.

The explanation.

You write `class Book(LibraryItem):` — the parent name in parentheses says “Book is a kind of `LibraryItem`.” Every method and attribute defined in `LibraryItem` is now available on `Book` objects, for free. If a book needs something extra (page count), you add it in `Book`. If most behaviour is shared (checkout, return), you write it once in the parent and every child gets it.

This is the cure for copy-paste. Without inheritance you would write `checkout` separately in `Book` and `Magazine`; with it, you write it once in `LibraryItem`.

The code example.

```
class LibraryItem:
    def __init__(self, title):
        self.title = title
        self.available = True

    def checkout(self):
        self.available = False

class Book(LibraryItem):           # Book IS A LibraryItem
    pass

b = Book("Python Crash Course")   # uses the parent's __init__
b.checkout()                     # uses the parent's checkout
print(b.available)               # False
```

Common mistakes.

- Many beginners recopy the parent’s methods into the child. The whole point of inheritance is *not* to.

Quick Check. If `Book(LibraryItem)` defines no `checkout` method of its own, can a `Book` still be checked out?

Answer: Yes — it inherits `checkout` from `LibraryItem`.

`super().__init__()` — calling the parent’s constructor

The big idea. When a child needs its own `__init__` (to store extra data), it should still run the parent’s `__init__` to set up the shared data. `super().__init__(...)` does exactly that.

This is like when... a new specialist employee fills out the standard starter form (the parent’s setup) *and* then an extra specialist form. `super().__init__` is filling out the standard form before adding the extra details.

The explanation.

Define `__init__` in the child, take the parent’s parameters plus the new ones, and call `super().__init__(parent_args)` first to let the parent set up the shared attributes. Then assign the child’s own attributes. `super()` means “the parent class,” so `super().__init__(title, author)` runs `LibraryItem.__init__` with `self` automatically.

The code example.

```
class LibraryItem:
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.available = True

class Book(LibraryItem):
    def __init__(self, title, author, pages):
        super().__init__(title, author)    # set up shared attributes
        self.pages = pages                 # then the book-specific one

b = Book("Fluent Python", "Ramalho", 792)
print(b.title, b.pages)                  # Fluent Python 792
```

Common mistakes.

- Many beginners forget `super().__init__(...)` and the shared attributes (`title`, `available`) never get set, causing `AttributeError` later.

Quick Check. Why call `super().__init__(...)` in a child’s constructor?

Answer: To run the parent’s setup so the inherited attributes are initialised before the child adds its own.

Method overriding — a more specific version

The big idea. A child can **override** a parent method by defining a method with the same name. When called on a child object, the child’s version runs instead of the parent’s.

This is like when... a general “greet the customer” script exists company-wide, but the VIP desk has its own version. A VIP customer gets the VIP greeting, not the generic one.

The explanation.

If `LibraryItem` has `display_info` and `Book` defines its own `display_info`, then `book.display_info()` runs `Book`’s version. The child’s version can even call the parent’s with `super().display_info()` and then add to it — reuse plus extension. Overriding is how each subclass shows its own extra fields (pages for books, issue number for magazines).

The code example.

```

class LibraryItem:
    def __init__(self, title):
        self.title = title
    def display_info(self):
        return f"{self.title}"

class Book(LibraryItem):
    def __init__(self, title, pages):
        super().__init__(title)
        self.pages = pages
    def display_info(self):
        # override
        return f"{super().display_info()}, {self.pages} pages"

print(Book("SICP", 657).display_info())
# SICP, 657 pages

```

Common mistakes.

- Many beginners override a method but forget they can reuse the parent's via `super().method()`, and end up duplicating the parent's logic.

Quick Check. If both parent and child define `display_info`, which runs for a child object?

Answer: The child's version — it overrides the parent's.

Polymorphism — one name, many behaviours

The big idea. **Polymorphism** means you can call the same method name on different object types and each does the right thing for its type. You loop over a mixed list and call `display_info()` on each, and books print like books, magazines like magazines.

This is like when... you tell everyone in a room “introduce yourself.” Each person responds in their own way, but you gave one instruction. The instruction is uniform; the responses vary by type.

The explanation.

Because both `Book` and `Magazine` have a `display_info` method (each overriding the parent's), you can put them in one list and loop, calling `item.display_info()` on every one. Python automatically runs the correct version based on each object's actual type. You write one loop; it handles every item type — and any new type you add later that also has `display_info`.

The code example.

```

items = [Book("SICP", 657), Magazine("Wired", 347)]
for item in items:
    print(item.display_info())
# each prints in its own format

```

Common mistakes.

- Many beginners write `if isinstance(item, Book): ... elif isinstance(item, Magazine): ...` to choose formatting. Polymorphism removes that branching — each object knows how to display itself.

Quick Check. Why can one loop calling `display_info()` handle both books and magazines?

Answer: Both types define `display_info`, so Python runs the matching version for each object — that is polymorphism.

`isinstance()` — checking an object's type

The big idea. `isinstance(obj, SomeClass)` returns `True` if `obj` is that class (or a subclass of it). Use it when you genuinely need to know the type — for counting how many books versus magazines you have, for instance.

This is like when... a sorting clerk glances at each parcel to tell letters from boxes before stacking them on the right shelf.

The explanation.

`isinstance(b, Book)` is `True` for a `Book`. Crucially, `isinstance(b, LibraryItem)` is *also* `True`, because a `Book` is a kind of `LibraryItem` — inheritance flows through. Prefer polymorphism (a shared method) over `isinstance` for behaviour; reserve `isinstance` for genuine type questions like counting or filtering by type.

The code example.

```
items = [Book("SICP", 657), Magazine("Wired", 347)]
book_count = sum(1 for it in items if isinstance(it, Book))
print(book_count)                # 1
print(isinstance(items[0], LibraryItem))  # True – a Book is a LibraryItem
```

Common mistakes.

- Many beginners reach for `isinstance` to pick behaviour when a shared (polymorphic) method would be cleaner. Use it for type questions, not for choosing what an object should *do*.

Quick Check. Is `isinstance(a_book, LibraryItem)` true if `Book` inherits from `LibraryItem`?

Answer: Yes — `isinstance` is true for the class and any of its ancestors.

A collection of mixed objects

The big idea. Because books and magazines share a base class, you can keep them all in one list and treat them uniformly — display them, search them, check them out — through the shared interface.

This is like when... a single library shelf holds books and magazines side by side; the checkout desk handles any of them with the same process.

The explanation.

Store every item in one list. Operations that use only shared methods (checkout, return, list-all via `display_info`) work on any item without caring about its exact type. Search by title loops the list comparing `item.title`. The list-of-objects pattern from Chapter 13 scales up here to a list of *different* object types unified by a common base.

The code example.

```
catalogue = []
catalogue.append(Book("SICP", 657))
catalogue.append(Magazine("Wired", 347))
for item in catalogue:
    print(item.display_info())  # uniform handling, varied output
```

Common mistakes.

- Many beginners keep separate lists per type, then duplicate every operation. One list plus a shared base keeps the code single-source.

Quick Check. What lets books and magazines live in the same list and be handled by one loop?

Answer: They share the `LibraryItem` base class and its interface (e.g. `display_info`), so common code treats them uniformly.

④ How the concepts connect

`LibraryItem` holds the shared state (title, author, available) and shared behaviour (checkout, return_item, a base `display_info`). `Book` and `Magazine` inherit from it, call `super().__init__` to set up the shared attributes, add their own (pages/genre, issue_number), and override `display_info` to show those extras. The catalogue is one list of mixed objects. A view loop calls `display_info()` polymorphically; a count uses `isinstance` to tell the types apart. Checkout and return work on any item through the inherited methods.

⑤ The build walkthrough

Step 1 · The base class. Define `LibraryItem` with `__init__(self, title, author, item_id)` setting those plus `self.available = True`. Add `checkout` (set unavailable, with a message if already out), `return_item` (set available), and a base `display_info` returning the shared fields and availability.

Step 2 · The subclasses. Define `Book(LibraryItem)` with `__init__` that calls `super().__init__(...)` then stores `pages` and `genre`, and overrides `display_info` to append those (reusing `super().display_info()`). Define `Magazine(LibraryItem)` similarly with `issue_number`.

Step 3 · The catalogue. Keep all items in one list. Write a menu: add book, add magazine, checkout, return, view all, search, quit.

Step 4 · Wire the menu. Add-book/add-magazine create the right object and append it. Checkout/return find the item by ID and call its inherited method. View-all loops the list calling `display_info()` polymorphically.

Step 5 · Search and counts. Search loops comparing `item.title` case-insensitively. Optionally show how many of each type with `isinstance` counts.

⑥ Understanding the design

Three habits. First, *put shared code in the base class once* — `checkout` lives only in `LibraryItem`, so a bug fix there fixes it for every item type. Second, *override to specialise, and reuse the parent with `super()`* — `Book.display_info` adds to the base rather than rewriting it. Third, *prefer polymorphism over type-checking* — a view loop that calls `display_info()` needs no `isinstance` branches, and it keeps working when you add a `DVD` class later. Reserve `isinstance` for genuine “what type is this?” questions.

⑦ The exercise twist

Add a `DVD` class with `runtime`, and a `Member` class tracking who has what checked out.

What new concept does this need? `DVD` is just another subclass — define it like `Book`, with a `runtime_minutes` attribute and an overridden `display_info`. `Member` introduces *composition*: a member object holds a list of the items they have borrowed (objects containing objects).

How does it extend what you built? Thanks to polymorphism, adding `DVD` requires no change to your view or checkout loops. The `Member` class is new structure: checkout now also appends the item to the member’s borrowed list, and return removes it.

Hint for the hardest part. Keep one source of truth for “is this checked out”: the item’s `available` flag. The member’s borrowed list should reference the same item objects, not copies — so checking an item out updates both views consistently because they point at the same object.

⑧ What to remember from this chapter

- `class Child(Parent):` makes `Child` inherit all of `Parent`’s attributes and methods.
- A child’s `__init__` should call `super().__init__(...)` to run the parent’s setup before adding its own data.
- Overriding means defining a method with the same name in the child; it can reuse the parent’s via `super().method()`.
- Polymorphism: call one method name on different types in a loop, and each runs its own version — no type branching.
- `isinstance(obj, Class)` checks type (true for the class and its ancestors); use it for type questions, not for choosing behaviour.

⑨ Coming up next

In the next chapter you leave classes for a while and work with the file system: a personal diary that names entry files by timestamp and uses the `os` module to list, read, and delete them.

Personal Diary

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A personal diary that stores each entry as its own timestamped text file inside a `diary/` folder. You write an entry and it is saved with a name like `2026-05-26_14-30.txt`; you list all entries, read one by date, search every entry for a keyword, and delete entries you no longer want. By the end you will be comfortable with the **os module** — the toolkit for working with folders and files on disk — and with building filenames safely from the current date and time.

② The concepts you need

- **The os module.** Tools for listing, joining, and removing paths.
- **os.makedirs().** Creating a folder, safely, if it does not exist.
- **os.listdir().** Getting the names of files in a folder.
- **os.path.join().** Building a path that works on any operating system.
- **File modes 'w', 'r', 'a'.** Writing, reading, and appending.
- **Timestamped filenames.** Using `strftime` to make a safe, sortable filename.
- **Searching with in.** Finding a keyword inside file contents.

③ Concept deep dives

The os module — talking to the file system

The big idea. The `os` module lets your program work with the file system itself — create folders, list what is inside them, build paths, delete files. Until now you opened files you named directly; `os` lets you *discover and manage* files.

This is like when... you stop just reading one book you were handed and start walking the library shelves yourself — seeing what is there, organising it, removing what you do not need.

The explanation.

After `import os`, you get a family of functions. The ones you need here: `os.makedirs(path)` to create folders, `os.listdir(path)` to see what is inside one, `os.path.join(a, b)` to build a path, `os.path.exists(path)` to test for existence, and `os.remove(path)` to delete a file. These are the same operations a file manager performs, available to your code.

The code example.

```
import os

print(os.path.exists("diary"))      # False (probably)
os.makedirs("diary", exist_ok=True) # create it
print(os.path.exists("diary"))      # True
print(os.listdir("diary"))          # [] - empty folder
```

Common mistakes.

- Many beginners confuse `os` (file system) with file *contents*. `os` manages files and folders; `open()` reads and writes what is *inside* a file.

Quick Check. Which function lists the files inside a folder?

Answer: `os.listdir(folder)`.

os.makedirs() — create a folder safely

The big idea. `os.makedirs(path, exist_ok=True)` creates a folder, including any missing parent folders, and the `exist_ok=True` flag means “do nothing if it already exists” instead of crashing.

This is like when... you tell a builder “make sure there is a room here” — if it already exists, fine; if not, build it. Either way you end up with the room.

The explanation.

On first run, the `diary/` folder does not exist, and trying to write a file into it would fail. Call `os.makedirs("diary", exist_ok=True)` at startup. Without `exist_ok=True`, a second run would crash because the folder already exists. With it, the call is safe to run every time.

The code example.

```
import os
os.makedirs("diary", exist_ok=True)    # safe every run
# now writing into diary/ will succeed
```

Common mistakes.

- Many beginners omit `exist_ok=True` and crash with `FileExistsError` on the second run. Always include it for setup folders.

Quick Check. What does `exist_ok=True` do?

Answer: It makes `makedirs` succeed silently if the folder already exists, instead of raising an error.

os.path.join() — build paths that work everywhere

The big idea. `os.path.join("diary", "entry.txt")` joins path pieces with the correct separator for the current operating system. It saves you from hard-coding slashes that break across platforms.

This is like when... you address an envelope and let the postal service handle the local formatting — you supply the parts, it assembles them correctly for the destination country.

The explanation.

Windows uses backslashes in paths, macOS and Linux use forward slashes. If you hard-code `"diary/" + filename`, your code is fragile. `os.path.join("diary", filename)` produces the right thing on every system. Always build paths this way rather than gluing strings with `+`.

The code example.

```
import os
path = os.path.join("diary", "2026-05-26_14-30.txt")
print(path)    # diary/2026-05-26_14-30.txt (or with \ on Windows)
```

Common mistakes.

- Many beginners write `"diary/" + name`, which works on their machine but can break elsewhere and mishandles edge cases. Use `os.path.join`.

Quick Check. Why prefer `os.path.join` over `"folder/" + name`?

Answer: It uses the correct path separator for the operating system and handles edge cases, so your code is portable.

File modes 'w', 'r', 'a'

The big idea. When you open a file, the *mode* says what you intend: `'w'` write (replace contents), `'r'` read (the default), `'a'` append (add to the end). Choosing the right mode is essential.

This is like when... you pick up a notebook with intent: to start a fresh page and overwrite (`'w'`), to read what is there (`'r'`), or to add below the last entry without erasing (`'a'`).

The explanation.

You met `open` in Chapter 5. The second argument is the mode. `'w'` *erases* any existing content before writing — powerful and dangerous. `'r'` is read-only (the default if you pass no mode). `'a'` keeps existing

content and adds at the end. For a diary where each entry is its own new file, 'w' is right (the file is brand new). For reading an entry back, 'r'.

The code example.

```
with open("note.txt", "w", encoding="utf-8") as f:    # create/overwrite
    f.write("Today I finished chapter 15.\n")

with open("note.txt", "r", encoding="utf-8") as f:    # read it back
    print(f.read())
```

Common mistakes.

- Many beginners open an existing file in 'w' mode expecting to add to it, and wipe it instead. Use 'a' to append.

Quick Check. Which mode adds to a file without erasing what is already there?

Answer: 'a' (append).

Timestamped filenames

The big idea. Use `datetime.now().strftime(...)` to build a filename from the current date and time — sortable, unique-per-minute, and free of characters that confuse the file system.

This is like when... a clerk stamps each document with the date and time received, so the files naturally sort in chronological order and none collide.

The explanation.

You met `strftime` in Chapter 7. For filenames, avoid colons (:) — they are illegal in Windows paths — so format the time with hyphens: `"%Y-%m-%d_%H-%M"`. This yields `2026-05-26_14-30`, which sorts correctly as text *because* the format is year-first. Append `.txt` and join it to the diary folder with `os.path.join`.

The code example.

```
from datetime import datetime
import os

name = datetime.now().strftime("%Y-%m-%d_%H-%M") + ".txt"
path = os.path.join("diary", name)
print(path)           # diary/2026-05-26_14-30.txt
```

Common mistakes.

- Many beginners use `"%H:%M"` (with a colon) in a filename, which fails on Windows. Use hyphens for the time part.

Quick Check. Why format the timestamp year-first (`%Y-%m-%d`)?

Answer: So filenames sort chronologically when sorted as plain text.

Searching file contents with in

The big idea. To find which entries mention a keyword, read each file's text and test `keyword in text`. Lowercase both sides for a case-insensitive search.

This is like when... you flip through a stack of letters looking for any that mention "Kigali," scanning each page for the word.

The explanation.

You used `in` for membership on lists and dicts; on strings it tests substring presence: `"project" in "my project"` is `True`. To search the diary, loop over `os.listdir("diary")`, read each file, and check `keyword.lower() in text.lower()`. Collect the matching filenames to report.

The code example.

```
import os
keyword = "project"
matches = []
for name in os.listdir("diary"):
    path = os.path.join("diary", name)
    with open(path, encoding="utf-8") as f:
        if keyword.lower() in f.read().lower():
            matches.append(name)
print(matches)
```

Common mistakes.

- Many beginners compare with different cases and miss matches. Lowercase both the keyword and the text.

Quick Check. What does "cat" in "concatenate" return?

Answer: True — in on strings tests for a substring, and "cat" appears inside "concatenate".

④ How the concepts connect

At startup, `os.makedirs("diary", exist_ok=True)` ensures the folder exists. To add an entry, you read the user's text, build a filename with `strftime`, join it to the folder with `os.path.join`, and write it in 'w' mode. To list, `os.listdir` gives the filenames (sorted chronologically thanks to the format). To read one, you join its name to the folder and open in 'r'. To search, you loop the folder reading each file and test `keyword in text`. To delete, `os.remove(path)`.

⑤ The build walkthrough

Step 1 · Setup. `import os` and `datetime`. At startup, `os.makedirs("diary", exist_ok=True)`. Define a constant for the folder name.

Step 2 · New entry. Prompt the user to type their entry (read lines until a blank line, joining them). Reject an empty entry. Build the filename with `strftime("%Y-%m-%d_%H-%M") + ".txt"`, join to the folder, and write in 'w' mode. Confirm the saved filename.

Step 3 · List and read. "View all" calls `sorted(os.listdir(folder))` and prints the entry names. "Read by date" takes a filename (or date prefix), joins it, opens in 'r', and prints the contents — handle a missing file gracefully.

Step 4 · Search. Prompt for a keyword. Loop the folder, read each file, and collect names where `keyword.lower() in text.lower()`. Report the matches (or "none found").

Step 5 · Delete. Take a filename, confirm with the user, and call `os.remove(os.path.join(folder, name))` inside a `try / except FileNotFoundError`.

⑥ Understanding the design

Three habits. First, *create the folder once at startup with `exist_ok=True`* — so the program is safe to run any number of times without crashing. Second, *build every path with `os.path.join`* — never glue strings, so the diary works on your Windows laptop and a Linux server alike. Third, *encode the timestamp year-first with no colons* — this single formatting choice gives you filenames that are both legal on every OS and naturally sorted in time order.

⑦ The exercise twist

Add mood tracking (rate each entry 1–10) and a monthly mood summary.

What new concept does this need? Nothing structurally new — it combines a number prompt with aggregation by month (Chapter 9). The mood can be stored as a first line in each entry file, or in the filename.

How does it extend what you built? Ask for a 1–10 mood when writing an entry and record it. For the summary, parse the month from each filename, collect that month's moods, and report the average per month using the guarded-mean pattern you already know.

Hint for the hardest part. Decide *where* the mood lives before coding. Storing it as the first line of the file ("mood: 7") keeps one file per entry and is easy to parse back — read the first line, split on the colon, convert to int. Sketch the file format first; the parsing code follows from it.

⑧ What to remember from this chapter

- `import os` gives file-system tools: `makedirs`, `listdir`, `path.join`, `path.exists`, `remove`.
- `os.makedirs(path, exist_ok=True)` creates a folder and does nothing if it already exists.
- `os.path.join(folder, name)` builds portable paths; never glue with + and slashes.
- File modes: 'w' overwrites, 'r' reads, 'a' appends.
- Build timestamped filenames with `strftime("%Y-%m-%d_%H-%M")` (hyphens, year-first) for legal, sortable names; search contents with `keyword in text`.

⑨ Coming up next

In the next chapter you analyse tabular data: a CSV analyzer that reads spreadsheet-style files with the `csv` module, computes statistics per column, and handles messy, missing values.

CSV Data Analyzer

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A tool that reads any CSV file — the format every spreadsheet exports — and produces a per-column analysis: for numeric columns, the count, mean, median, minimum, maximum, and standard deviation; for text columns, the number of unique values and the most common ones. It survives messy real-world data: missing cells, blank rows, columns that mix numbers and text. By the end you will know how to read structured tabular data with the **csv module**, compute real statistics with the **statistics module**, and detect a column's type from its contents.

② The concepts you need

- **CSV format.** Comma-separated rows — what spreadsheets export.
- **csv.reader** and **csv.DictReader.** Two ways to read rows; the second gives named columns.
- **The statistics module.** `mean`, `median`, `stdev` ready-made.
- **Type detection.** Deciding whether a column is numeric by trying to convert it.
- **Handling missing data.** Skipping blank cells without crashing.
- **Counter for most-common.** Reused from Chapter 5 — top values in a text column.

③ Concept deep dives

CSV format — the language of spreadsheets

The big idea. A **CSV** (comma-separated values) file stores a table as plain text: one row per line, columns separated by commas, usually with a header row of column names at the top.

This is like when... you write a table on paper using columns and rows; CSV is that table flattened into lines, with commas marking where one column ends and the next begins.

The explanation.

A CSV looks like this:

```
name,age,city
Alice,30,Kigali
Bob,25,Lagos
```

The first line is the header. Each later line is a record. You *could* split each line on commas yourself — but values containing commas, quotes, or newlines break that naive approach. The **csv** module handles all those edge cases correctly, which is why you use it rather than `.split(",")`.

Common mistakes.

- Many beginners parse CSV with `line.split(",")`, which breaks on quoted values like "Smith, John". Use the **csv** module.

Quick Check. Why not just use `line.split(",")` to read a CSV?

Answer: Values can contain commas inside quotes; `split(",")` breaks them. The **csv** module parses quoting correctly.

csv.reader and csv.DictReader

The big idea. **csv.reader** gives each row as a *list* of strings. **csv.DictReader** gives each row as a *dictionary* keyed by the header names — usually more convenient because you refer to columns by name, not position.

This is like when... you read a table either by column number (“the value in column 3”) or by column name (“the value in the ‘city’ column”). Names are easier to keep straight.

The explanation.

Both take an open file. `csv.reader(f)` yields rows as lists: `["Alice", "30", "Kigali"]`. `csv.DictReader(f)` reads the header automatically and yields rows as dicts: `{"name": "Alice", "age": "30", "city": "Kigali"}`. Note that every value comes in as a *string* — even “30” — so you convert to numbers yourself when needed. Wrap reading in `with open(...)` and pass `newline=""` as recommended for CSV files.

The code example.

```
import csv

with open("data.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["city"])  # access columns by name
```

Common mistakes.

- Many beginners forget that CSV values are strings and try arithmetic on `row["age"]` directly. Convert with `int()` or `float()` first.
- Many beginners omit `newline=""`, which can cause blank lines between rows on some systems.

Quick Check. What type is each row from `csv.DictReader`?

Answer: A dictionary keyed by the header column names.

The statistics module

The big idea. The `statistics` module gives you `mean`, `median`, `stdev`, and more, so you compute real summary statistics without writing the formulas yourself.

This is like when... a scientific calculator has a stats mode — you feed in the numbers and read off the average and spread, rather than computing variance by hand.

The explanation.

You met `statistics.mean` in Chapter 13. Here you also use `statistics.median` (the middle value) and `statistics.stdev` (the standard deviation, a measure of spread). Each takes a list of numbers. Two cautions: they raise on an empty list, and `stdev` needs at least two values. Guard both. `min()` and `max()` are built-ins, no import needed.

The code example.

```
import statistics

nums = [85, 92, 78, 90, 88]
print(statistics.mean(nums))      # 86.6
print(statistics.median(nums))    # 88
print(round(statistics.stdev(nums), 2))  # 5.41
print(min(nums), max(nums))       # 78 92
```

Common mistakes.

- Many beginners call `statistics.stdev` on a single value and crash — it needs at least two. Guard `if len(nums) >= 2`.

Quick Check. What does `statistics.median([3, 1, 2])` return?

Answer: 2 — the middle value once sorted.

Type detection — is this column numeric?

The big idea. Since every CSV value arrives as a string, you decide whether a column is numeric by *trying* to convert its values to `float`. If they convert, treat the column as numbers; if not, treat it as text.

This is like when... you test whether a key fits a lock by trying it. If it turns, it is the right type; if not, you know it is something else.

The explanation.

Write a small helper that attempts `float(value)` inside `try / except ValueError`, returning `True` or `False`. A column is numeric if (most of) its non-empty values convert. This “ask forgiveness, not permission” style — try the conversion and catch the failure — is idiomatic Python, cleaner than checking each character yourself.

The code example.

```
def is_number(s):
    try:
        float(s)
        return True
    except ValueError:
        return False

print(is_number("3.14"))    # True
print(is_number("hello"))   # False
print(is_number(""))        # False
```

Common mistakes.

- Many beginners try to detect numbers by checking `.isdigit()`, which fails on decimals and negatives. Trying `float()` handles all numeric forms.

Quick Check. Why use `try: float(s)` instead of `s.isdigit()` to detect numbers?

Answer: `.isdigit()` is `False` for `"3.14"` and `"-5"`; trying `float()` correctly accepts decimals and negatives.

Handling missing data

The big idea. Real CSVs have blank cells, partial rows, and stray whitespace. Before analysing a column, filter out the empty values so they do not crash your conversions or skew your statistics.

This is like when... you tally a survey and set aside the blank responses rather than counting them as zero — a blank is “no answer,” not “the answer is nothing.”

The explanation.

For each column, collect its values across all rows, strip whitespace, and drop the empties (if `value.strip()`). For numeric columns, convert the survivors to floats. This cleaning step is what separates a toy that works on perfect data from a tool that handles the messy files you actually get. Report how many values were missing so the user knows the data quality.

The code example.

```
raw = ["85", "", "92", " ", "78"]
clean = [v.strip() for v in raw if v.strip()]    # drop blanks
numbers = [float(v) for v in clean]              # convert survivors
print(numbers)                                  # [85.0, 92.0, 78.0]
```

Common mistakes.

- Many beginners convert a blank string with `float("")` and crash. Filter empties *before* converting.

Quick Check. What does `float("")` do?

Answer: Raises `ValueError`. Filter out empty strings before converting.

Counter for the most-common text values

The big idea. For a text column, the interesting summary is which values appear most. `Counter` (Chapter 5) gives the counts and `.most_common(n)` the top `n`.

This is like when... you tally which city appears most often in a guest list — a few quick ticks per name reveal the most frequent.

The explanation.

Build `Counter(clean_values)` from the column's non-empty values. `len(counter)` is the number of distinct values; `counter.most_common(3)` gives the three most frequent as `(value, count)` tuples. This is the right summary for non-numeric columns where mean and median make no sense (you cannot average city names).

The code example.

```
from collections import Counter
cities = ["Kigali", "Lagos", "Kigali", "Cairo", "Kigali"]
c = Counter(cities)
print(len(c))           # 3 unique
print(c.most_common(2)) # [('Kigali', 3), ('Lagos', 1)]
```

Common mistakes.

- Many beginners try to compute a mean of text values. For text columns, report unique count and most-common instead.

Quick Check. What summary suits a text column better than mean and median?

Answer: Unique count and most-common values (via `Counter`).

④ How the concepts connect

You read the CSV with `csv.DictReader`, collecting all rows into a list. For each column (the header keys), you gather its values, strip them, and drop the empties. A type-detection helper decides whether the cleaned column is numeric. Numeric columns convert to floats and report count, mean, median, min, max, and (guarded) `stdev`. Text columns build a `Counter` and report unique count and the most-common values. Missing-value counts are reported alongside.

⑤ The build walkthrough

Step 1 · Read the file. Prompt for a path. Inside `try / except FileNotFoundError`, open with `newline=""` and read all rows with `csv.DictReader` into a list. Capture the column names from `reader.fieldnames`. Handle an empty file.

Step 2 · Per-column gather and clean. For each column name, build a list of that column's values across all rows, stripped, with empties dropped. Record how many were missing (blank).

Step 3 · Detect type. Using the `is_number` helper, decide whether the column is numeric — for example, if all (or nearly all) non-empty values convert to float.

Step 4 · Numeric summary. Convert the cleaned values to floats. Report count, mean, median, min, max, and standard deviation (guard `stdev` for fewer than two values). Format figures with `:.2f`.

Step 5 · Text summary and report. For non-numeric columns, build a `Counter`, report the unique count and top-three most-common. Print a tidy per-column report, including the missing-value count for each.

⑥ Understanding the design

Three habits. First, *clean before you compute* — stripping and dropping empties up front means every later statistic runs on trustworthy data and never crashes on a blank cell. Second, *detect type by trying the conversion* — Python's "try it and catch the failure" style is more robust than inspecting characters, and it generalises to any numeric format. Third, *choose the summary that fits the data* — numbers get mean/median/spread, text gets unique-count and most-common; applying the wrong summary to a column produces nonsense, so the type check is what makes the report meaningful.

⑦ The exercise twist

Detect correlations between numeric columns and export a summary report.

What new concept does this need? `statistics.correlation` (Python 3.10+) measures how strongly two numeric columns move together, returning a value from -1 to 1. Exporting reuses file writing (Chapter 5) — or `csv.writer` to produce a summary CSV.

How does it extend what you built? After identifying the numeric columns, compute the correlation for each pair and report the strongest relationships. Add an “export” option that writes the full per-column analysis to a text or CSV file.

Hint for the hardest part. Correlation needs the two columns aligned row-by-row, with rows where *either* value is missing dropped from *both*. Build the aligned pairs first — walk the rows, keep only those where both columns have a value — then pass the two parallel lists to `statistics.correlation`.

⑧ What to remember from this chapter

- CSV is comma-separated rows with a header; read it with the `csv` module, never `split(",")`.
- `csv.DictReader` yields each row as a dict keyed by column name; all values are strings until you convert them.
- `statistics` gives `mean`, `median`, `stdev` (needs 2 or more values); `min`/`max` are built-ins.
- Detect a numeric column by trying `float(value)` in `try / except ValueError`.
- Strip and drop empty cells before computing; summarise numbers with stats and text with `Counter.most_common`.

⑨ Coming up next

In the next chapter you build a Markdown note-taking app that walks a folder tree with `os.walk`, sanitises filenames, and uses simple regular expressions to find and organise notes.

Markdown Note-Taking App

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line note-taking app where each note is a Markdown file inside a folder tree organised by topic. You create new notes (with a safe filename made from the title), browse the whole tree, search every note for a keyword, and find notes by `#tag` using a small piece of regular-expression matching. By the end you will be able to walk an entire folder tree with `os.walk`, scrub a user-provided title into a safe filename, and use the `re` module to extract patterns — three skills that come up in nearly every real-world data project.

② The concepts you need

- **`os.walk()`**. Walking every file in a folder and all its sub-folders.
- **Filename sanitization**. Turning a free-form title into a legal filename.
- **Regular expressions and `re.findall()`**. Patterns that find structured pieces of text.
- **Markdown basics**. A tiny bit of structure (`# heading`, `#tag`).
- **`os.path.relpath()`**. Showing a path relative to a folder, for tidy display.
- **with open for reading and searching**. Reused from Chapter 5.

③ Concept deep dives

`os.walk()` — visit every file in a folder tree

The big idea. `os.walk(root)` yields, for every folder in the tree, the folder path, its sub-folder names, and its file names. Loop over it and you visit every file in the tree, however deeply nested.

This is like when... you do a thorough inspection of a building, going floor by floor, listing every room on each floor — by the end you have visited everything inside.

The explanation.

You met `os.listdir` in Chapter 15, which only sees one folder. `os.walk` recurses. On each step it gives a 3-tuple: (`current_dir`, `subfolders`, `files`). You typically only use `current_dir` and `files`, joining them to build full paths. Filter for `.md` files with a comprehension or an `if` inside the loop.

The code example.

```
import os

for dirpath, _subdirs, files in os.walk("notes"):
    for name in files:
        if name.endswith(".md"):
            full = os.path.join(dirpath, name)
            print(full)
```

Common mistakes.

- Many beginners try to recurse manually with `os.listdir` and a stack of folders. `os.walk` does that for you.
- Many beginners forget to `os.path.join(dirpath, name)` and end up with bare filenames that no longer say which folder they came from.

Quick Check. Why join `dirpath` and `name` to build the full path?

Answer: `name` alone is just the filename; joining adds the folder so the path is usable for opening.

Filename sanitization

The big idea. A user’s note title can contain spaces, slashes, and punctuation that are illegal or awkward in filenames. Sanitising turns the title into a safe filename without losing readability.

This is like when... you write a label for a folder tab: long sentences with weird characters do not fit; you write a short, clean version.

The explanation.

A robust approach: lowercase the title, replace any whitespace with hyphens, and drop any character that is not a letter, digit, or hyphen. A list comprehension on the characters does the filter; `"-".join(s.split())` handles whitespace. This makes “My First Project!” become `my-first-project`. Truncate to a reasonable length (60 characters) so titles never produce monstrous filenames.

The code example.

```
title = "My First Project!"
safe = "-".join(title.lower().split())           # spaces -> hyphens
safe = "".join(c for c in safe if c.isalnum() or c == "-")
print(safe)                                     # my-first-project
```

Common mistakes.

- Many beginners allow user input straight into a filename and end up with crashes on Windows (illegal chars) or invisible files.

Quick Check. What goes wrong if you let “Notes / Drafts” be a filename?

Answer: The slash is interpreted as a folder separator, so the file lands in the wrong place — or the OS rejects it.

Regular expressions and `re.findall()`

The big idea. A **regular expression** is a short pattern that describes a *shape of text* — like “a hash followed by letters and digits.” `re.findall(pattern, text)` returns every substring matching the pattern.

This is like when... you give a librarian a description (“every book with a yellow spine”) instead of a list of titles. The pattern describes a shape; the librarian finds every match.

The explanation.

After `import re`, the function `re.findall(pattern, text)` returns a list of all matches. The pattern for “tags like `#python` or `#note-taking`” is `r"#\w[\w-]*"`: a hash, then one word character, then zero or more word characters or hyphens. The `r"..."` prefix is a *raw string* — it tells Python not to interpret the backslashes, leaving them for the regex engine.

You will not master regex in one chapter. Memorise just enough to extract simple patterns: `\w` (word character), `\d` (digit), `+` (one or more), `*` (zero or more), `[abc]` (any of these). Anything more complex, look up.

The code example.

```
import re

text = "Reading about #python and #web-dev today."
tags = re.findall(r"#\w[\w-]*", text)
print(tags)           # ['#python', '#web-dev']
```

Common mistakes.

- Many beginners omit the `r` prefix and find their backslashes silently swallowed by Python.
- Many beginners try to write one giant regex for everything. Keep patterns small; do further filtering in Python.

Quick Check. What does `re.findall(r"\d+", "abc 42 xyz 7")` return?

Answer: `['42', '7']` — every run of digits.

A tiny bit of Markdown

The big idea. Markdown is a plain-text format with light syntax: `#` **Heading** for a heading, `- item` for a list, and a few others. You do not need to render it — just read and search it as text — but a small structure helps. A note file's first line can be its title (`# Title`), with a free-form body below.

This is like when... a typed letter starts with the recipient's name on the first line. You always know where to look; the rest is freeform.

The explanation.

For this app you only need two conventions: the first line of every note starts with `#` and is the title; anywhere in the note, `#tag` patterns are tags. Read the first line to extract the title for display; use the regex above to extract tags. No parser needed — this is plain text with a couple of agreed shapes.

Common mistakes.

- Many beginners reach for a full Markdown library. For a note-finder, raw text + a regex is plenty.

Quick Check. How do you tell a Markdown heading line from a hashtag?

Answer: A heading starts the line with `#` followed by a space; a tag is a `#` followed directly by letters, anywhere in the text.

`os.path.relpath()` — tidy display paths

The big idea. `os.path.relpath(full_path, start)` returns the path of `full_path` *relative to* `start`. Use it to show “subfolder/note.md” instead of the full absolute path.

This is like when... directions from your current location are easier to follow than coordinates: “two streets east” beats “37.94°N, 30.06°E.”

The explanation.

`os.walk` gives you absolute-ish paths from the root you passed in; printing those in a list of notes is ugly. `os.path.relpath(full, "notes")` strips the leading `notes/` so the display reads cleanly. It is purely cosmetic — the full path is still what you open.

The code example.

```
import os
full = "notes/project/ideas.md"
rel = os.path.relpath(full, "notes")
print(rel)                # project/ideas.md
```

Common mistakes.

- Many beginners forget the second argument and get a path relative to the *current* directory, which is rarely what they want.

Quick Check. What does `os.path.relpath` need besides the full path?

Answer: The start point to be relative to.

Reading and searching every note

The big idea. Combine `os.walk`, with `open`, and substring `in` to search the entire tree for a keyword in note contents. Same shape as Chapter 15, scaled to a tree.

This is like when... you read every letter in every filing-cabinet drawer to find one mention of a name — slow on paper, instant for a computer.

The explanation.

Walk the tree, filter for `.md` files, open each, read the text, lowercase both keyword and text, and test `keyword in text`. Collect the matching paths to report. For tags, the same loop reads each file and runs the `re.findall` tag pattern, building a set of all tags or a list of files matching a specific tag.

The code example.

```
import os
matches = []
```

```

for dirpath, _, files in os.walk("notes"):
    for name in files:
        if not name.endswith(".md"):
            continue
        path = os.path.join(dirpath, name)
        with open(path, encoding="utf-8") as f:
            if "kigali" in f.read().lower():
                matches.append(path)
print(matches)

```

Common mistakes.

- Many beginners open a binary file by mistake (a stray image in the tree) and crash. Filter by extension first.

Quick Check. Why filter by `.md` extension before opening?

Answer: So you only read text notes — opening unexpected file types as text can produce errors or junk.

④ How the concepts connect

The notes folder is created at startup with `os.makedirs(..., exist_ok=True)`. To create a note, the user provides a topic and a title; the topic becomes a subfolder, and the sanitised title becomes the `.md` filename. The first line of every note is `# Title`. Browsing uses `os.walk` plus `os.path.relpath` to print a tidy tree. Search reads every note, testing the keyword with `in`. Tag listing runs `re.findall(r"#\w[\w-]*", ...)` over each note and collects unique tags; `filter-by-tag` returns the notes containing it.

⑤ The build walkthrough

Step 1 · Folder and helpers. Create the root notes folder at startup. Write a `sanitize(title)` helper that produces the filename. Write a tiny `extract_tags(text)` wrapper around `re.findall`.

Step 2 · Create a note. Prompt for a topic (used as a subfolder) and a title. Build the subfolder path with `os.path.join` and create it if needed. Build the filename from the sanitised title. Prompt the user for the body (multi-line until a blank line). Write the file in `'w'` mode with `# Title` as the first line and the body below.

Step 3 · List all notes. Walk the tree, collecting `.md` paths. For each, open and read the first line as the title (strip the leading `#`). Print as `relative_path – Title`.

Step 4 · Search by keyword. Walk the tree, read each file, test `keyword.lower() in text.lower()`, collect matches. Report paths with the matching line snippets if you like.

Step 5 · Tags. Walk the tree, collect all tags from every file into a set, and print the sorted list. `Filter-by-tag` is the same walk filtered to files whose tag set contains the queried tag.

⑥ Understanding the design

Three habits. First, *sanitise user input at the boundary* — once a title becomes a filename, every later operation can assume the filename is legal. Second, *recurse with `os.walk`, not by hand* — manual recursion with `listdir` is bug-prone; `os.walk` is the right tool. Third, *use regex for shapes, not for everything* — extract the tag pattern with `re.findall`, but do plain `in` checks for keyword search; mixing regex into every comparison makes code harder to read for no benefit.

⑦ The exercise twist

Add note linking (`[[other-note]]` references), backlinks, and a simple knowledge-graph view.

What new concept does this need? A second regex pattern for `[[name]]` references, and a second pass over the notes to *build a graph*: for each note, which notes link to it. Storing the graph as a `dict[note] -> set(notes that link to it)`.

How does it extend what you built? After the regular index, run a second pass that extracts `[[references]]` from every note, resolves each to a filename, and updates the backlink dict. Add a menu choice “show backlinks for note X.”

Hint for the hardest part. Resolution is fiddly: `[[my idea]]` should resolve to whichever note has a sanitised filename of `my-idea.md`, regardless of folder. Build a lookup `sanitized_title -> path` once, before processing references, so each lookup is a quick dict access.

⑧ What to remember from this chapter

- `for dirpath, _, files in os.walk(root):` visits every folder in the tree; join `dirpath` and each filename.
- Sanitise titles into filenames: lowercase, split-and-join with hyphens, keep only alphanumerics and hyphens.
- `re.findall(pattern, text)` with a raw-string pattern returns every match; `\w` is a word character, `\d` a digit, + “one or more.”
- `os.path.relpath(full, start)` gives a tidy display path; the file is still opened by its full path.
- Search by combining `os.walk` + `with open` + `keyword in text`, both sides lowercased.

⑨ Coming up next

In the next chapter you build a password manager and step into security: storing passwords as one-way hashes with `hashlib`, hiding input with `getpass`, and locking out brute-force attempts.

Password Manager

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line password manager protected by a master password. You set the master once; from then on, you must unlock it before you can store or read site passwords. The master is *never* stored in plain text — only a one-way **hash** of it. After a few wrong tries the program locks you out. By the end you will have your first real brush with security engineering: hashing, salting, hidden input, and logout. These are the foundations behind every login system on the internet.

② The concepts you need

- **Hash functions.** A one-way transformation: easy to compute, impossible to reverse.
- **hashlib.sha256().** Python's built-in SHA-256 hasher.
- **Salt.** A random value added to the password before hashing, so identical passwords hash differently.
- **secrets.token_hex().** Generating a random salt securely.
- **getpass.getpass().** Reading input without echoing it to the screen.
- **Lockout logic.** Counting wrong tries and refusing further attempts.

③ Concept deep dives

Hash functions — one-way fingerprints

The big idea. A **hash function** takes input of any size and produces a fixed-size string — the *hash*. Two crucial properties: the same input always hashes to the same output, and given a hash you cannot recover the original input.

This is like when... you press a fingerprint into wax. The print uniquely identifies the finger; but from the print alone, you cannot rebuild the finger.

The explanation.

Hashing lets you store proof that you know a password without storing the password itself. At setup, you hash the master and save the hash. At login, you hash whatever the user typed and compare the two hashes. If they match, the user knew the password; if not, they did not — and either way the original password was never written down. An attacker who steals your file finds only hashes, not passwords.

Hashing is *not* encryption. Encrypted text can be decrypted with the key; hashed text cannot be unhashed, by anyone, ever. That asymmetry is exactly what makes it the right tool for passwords.

The code example.

```
import hashlib

h1 = hashlib.sha256("hello".encode()).hexdigest()
h2 = hashlib.sha256("hello".encode()).hexdigest()
h3 = hashlib.sha256("Hello".encode()).hexdigest()
print(h1 == h2)           # True – same input, same hash
print(h1 == h3)           # False – one letter differs, totally different hash
```

Common mistakes.

- Many beginners use hashing for things that need to be reversible (encrypting messages). Hashes are one-way; use real encryption for that.
- Many beginners assume two different inputs cannot produce the same hash. In SHA-256 it is computationally infeasible, but in principle two inputs *could* — this is called a collision. SHA-256 is strong enough that you do not have to worry about it.

Quick Check. Why can you safely store a password's hash but not the password itself?

Answer: You can verify the user knows the password (by hashing what they type and comparing), but an attacker who reads your file cannot turn the hash back into the password.

hashlib.sha256() — Python's built-in hasher

The big idea. `hashlib.sha256(data).hexdigest()` returns the SHA-256 hash of `data` as a hex string. The wrinkle: `data` must be `bytes`, not `str`, so you call `.encode()` first.

This is like when... a copying machine that only takes documents in a specific tray — you slide the paper into the right tray (encode to bytes), then turn the handle (hashlib).

The explanation.

hashlib offers several algorithms; SHA-256 is the modern default for general use. Pass the bytes to `sha256()`, then call `.hexdigest()` to get a 64-character hex string you can store in a text file. To verify a password attempt, you encode and hash the attempt the same way, and compare the two hex strings with `==`.

The code example.

```
import hashlib

def hash_text(s):
    return hashlib.sha256(s.encode("utf-8")).hexdigest()

stored = hash_text("master123")
print(stored)                                # 64-char hex string
print(hash_text("master123") == stored)      # True
print(hash_text("Master123") == stored)      # False
```

Common mistakes.

- Many beginners forget `.encode()` and get `TypeError: Unicode-objects must be encoded before hashing`.
- Many beginners use the older `md5` or `sha1` they read about somewhere. Both are broken for security; use SHA-256.

Quick Check. Why is `.encode()` needed before hashing a string?

Answer: `hashlib` works on bytes, not strings — `.encode()` converts the string to its byte form.

Salt — defeating identical-password attacks

The big idea. A **salt** is a random string added to the password before hashing, so two users with the same password get different hashes. It also prevents an attacker from precomputing every common password's hash once and looking them up.

This is like when... a fingerprint reader that also senses the time of day. Even the same finger looks slightly different at each scan, so a copy of one reading does not help a forger.

The explanation.

At setup, generate a random salt — for example `secrets.token_hex(16)` from Chapter 8 — and store it alongside the hash. The stored hash is of `salt + password`, not the password alone. To verify a login attempt, fetch the stored salt, hash `salt + attempt`, and compare. Two users with the password "hello" now produce different stored hashes because their salts differ.

The code example.

```
import hashlib, secrets

salt = secrets.token_hex(16)                  # random, per password
stored = hashlib.sha256((salt + "master123").encode()).hexdigest()
# At login: fetch salt + stored, hash (salt + attempt), compare.
attempt = "master123"
ok = hashlib.sha256((salt + attempt).encode()).hexdigest() == stored
print(ok)                                    # True
```

Common mistakes.

- Many beginners hash without a salt, then wonder why two users with the same password show identical entries in their data — and why an attacker with a precomputed table can crack many passwords at once.

Quick Check. Why store a per-user salt rather than one fixed salt for everyone?

Answer: So two users with the same password get different stored hashes — and an attacker cannot exploit shared salts.

getpass.getpass() — reading without echo

The big idea. `getpass.getpass(prompt)` reads a line of input from the user without showing what they type. Use it for any password prompt, never plain `input()`.

This is like when... a teller hands you a small shielded keypad. You type your PIN; nobody around can see what your fingers pressed.

The explanation.

`getpass` is in the standard library. Call `getpass.getpass("Master: ")` and the user types blindly — the characters do not appear on screen. The function returns the typed string just like `input()`. This protects against shoulder-surfing and against the password appearing in terminal scrollbar.

A small caveat: in some IDEs the terminal does not support hidden input and `getpass` falls back to visible input with a warning. That is the IDE, not your code.

The code example.

```
from getpass import getpass

pw = getpass("Master password: ")      # typed invisibly
print("length:", len(pw))              # we never print the password itself
```

Common mistakes.

- Many beginners use plain `input()` for passwords, exposing them on screen and in terminal history.

Quick Check. What does `getpass.getpass()` return?

Answer: The string the user typed — just like `input()`, but the typing itself was hidden.

Lockout logic — slowing down brute force

The big idea. Limit how many wrong master-password attempts the user gets in a row. After, say, three wrong tries, refuse further attempts (for the session, or for a cooldown period). This prevents an attacker from trying millions of passwords automatically.

This is like when... an ATM swallows your card after three wrong PINs. The card itself stops the attack; the bank does not need to.

The explanation.

Keep a counter of consecutive wrong tries. Each wrong attempt increments it; a correct attempt resets it to zero. When the counter reaches the limit, refuse to accept any more attempts and exit (or sleep for a long cooldown). For a more permanent lockout, write the count to the data file so it survives the program quitting and re-opening.

The code example.

```
MAX_TRIES = 3
tries = 0
while tries < MAX_TRIES:
    attempt = "wrong"      # imagine getpass() here
    if attempt == "right":
        print("unlocked")
        break
```

```

    tries += 1
    print(f"Wrong ({MAX_TRIES - tries} left)")
else:
    print("Locked out.")    # runs when the while condition is False

```

Common mistakes.

- Many beginners track only one wrong try and lock out, frustrating legitimate users. Three to five is reasonable.
- Many beginners reset the counter every program run, so an attacker just restarts the program. Persist the counter to defeat that.

Quick Check. What does a successful attempt do to the wrong-try counter?

Answer: Resets it to zero — the next wrong streak starts fresh.

④ How the concepts connect

The data file holds the master's salt and salted hash plus a list of entries. At startup, `getpass` reads the master attempt invisibly; `hashlib.sha256` hashes `salt + attempt` and compares to the stored hash. A wrong-try counter (in memory, or on disk for persistence) enforces the lockout. On success, a menu offers add, view, search, delete. Each saved entry has a site, username, and password; the passwords inside the file could be stored as plain text *for this learning project*, but real systems would encrypt them with a key derived from the master.

⑤ The build walkthrough

Step 1 · First-run setup. If no data file exists, prompt the user to set a master with `getpass` (twice, to confirm a match). Generate a salt with `secrets.token_hex(16)`. Compute `sha256(salt + master)`. Save a JSON file containing the salt, the hash, and an empty entries list.

Step 2 · Unlock. On every later run, read the salt and stored hash from the file. Ask the master with `getpass`. Hash and compare. Use the lockout pattern (three tries, then exit).

Step 3 · The menu (unlocked). Add entry: prompt for site, username, password (`getpass` for the site password); append to the list; save. View all: list sites and usernames (do not display the passwords until the user explicitly asks). Reveal: prompt for a site, print its password — only after re-verifying the master via `getpass` for safety.

Step 4 · Search and delete. Search loops the entries comparing the site case-insensitively. Delete prompts for confirmation, removes the matching entry, and saves.

Step 5 · Persistence. Every change writes the full JSON back. Store nothing the user did not enter; keep the salt and hash unchanged unless they change the master.

⑥ Understanding the design

Three habits. First, *never store the master in plain text* — only its salted hash, so a stolen file cannot reveal the password. Second, *use `getpass`, never `input`, for any password prompt* — a one-line change that protects against shoulder-surfing and terminal scrollback. Third, *limit attempts* — three tries is the cheapest, most universal defence against brute force, and even a simple in-memory counter helps.

A note on scope: a real password manager encrypts the stored entries with a key derived from the master, so a stolen file is useless without the master. That step uses libraries you have not met yet (`cryptography`, `PBKDF2`). The version you build here teaches the foundations — hash, salt, hide, lock out — and is honest about its limits.

⑦ The exercise twist

Encrypt the stored entries with a key derived from the master.

What new concept does this need? Key derivation with `hashlib.pbkdf2_hmac` (built into `hashlib`): repeatedly hash the master with the salt to produce a fixed-length key. Symmetric encryption with the

`cryptography` library's Fernet, which gives you `encrypt(bytes) -> bytes` and `decrypt(bytes) -> bytes` using the derived key.

How does it extend what you built? At unlock, derive the key from the master + salt. Encrypt each saved password before writing; decrypt only when the user reveals one. A stolen file is now useless without the master.

Hint for the hardest part. Fernet keys must be URL-safe base64-encoded 32-byte strings. After PBKDF2 gives you 32 bytes, encode with `base64.urlsafe_b64encode` to get the format Fernet wants. Look up the recipe; do not invent the encoding yourself.

⑧ What to remember from this chapter

- A hash is a one-way fingerprint: `hashlib.sha256(s.encode()).hexdigest()` — store the hash, never the password.
- Salt with a per-password random value (`secrets.token_hex(16)`) so identical passwords hash differently.
- `getpass.getpass(prompt)` reads a line invisibly — always use it for password prompts.
- Lock out after a small number of consecutive wrong tries; reset the counter on success.
- This project teaches the foundations; real managers also encrypt the stored entries with a key derived from the master.

⑨ Coming up next

In the next chapter you reach out across the network for the first time: a stock-price fetcher that calls a JSON web API with the `requests` library, handles HTTP status codes, and keeps your API key safe in an environment variable.

Stock Price Fetcher

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line stock-price fetcher that calls a real JSON web API, parses the response, and prints the current price plus a short price history for a ticker symbol. You will keep your API key out of your code with an environment variable, handle network failures and bad status codes gracefully, and respect the API's rate limit with sensible pacing. By the end you will have written your first program that *reaches out across the internet* — the foundation of every modern application.

② The concepts you need

- **HTTP and APIs.** How a program talks to a web server.
- **The `requests` library.** The standard Python tool for making HTTP requests.
- **`.status_code` and `.json()`.** Reading what the server returned.
- **Common HTTP status codes.** 200, 401, 404, 429, 500 — and what to do about each.
- **Environment variables and `os.environ`.** Keeping secrets out of source code.
- **Network errors.** Catching the exceptions `requests` raises when things go wrong.

③ Concept deep dives

HTTP and APIs — programs talking to servers

The big idea. A web **API** is a URL you can fetch with a program to get structured data back. Most modern APIs return JSON (Chapter 6). Your program sends an HTTP **request**; the server sends a **response** containing a *status code* (was it OK?) and a *body* (the data).

This is like when... you phone a library and ask “is this book available?” The librarian’s reply has two parts: a quick yes/no (status code) and the details if yes (body).

The explanation.

You will not build the network plumbing — Python’s `requests` library does that. You give it a URL; it gives you back a response object with `.status_code`, `.text`, and (for JSON APIs) `.json()`. The shape of the URL and which parameters it accepts are defined by each API’s documentation. For this project, look up a free stock API such as Alpha Vantage or Finnhub; read its “GET quote” endpoint docs; that is the shape you target.

Common mistakes.

- Many beginners try to scrape a website’s price page with HTML parsing when the site has a documented JSON API. APIs are easier, more reliable, and what the site wants you to use.

Quick Check. What two things does an HTTP response contain?

Answer: A status code (was the request OK?) and a body (the data, often JSON).

The `requests` library

The big idea. `requests.get(url)` sends a GET request and returns a response object. Add a `params={}` argument for query-string values, and a `timeout=10` argument so a hung server cannot freeze your program forever.

This is like when... you call the librarian and you tell them “if you do not pick up in ten seconds, I will hang up and try later.” Timeouts are courtesy and self-defence.

The explanation.

`requests` is not in the standard library — install with `pip install requests`. Then `import requests`. The typical call is `r = requests.get(url, params={...}, timeout=10)`. The `params` argument lets

you supply query-string values cleanly without manually building the URL. Always set a timeout; APIs sometimes hang, and without a timeout your program will too.

The code example.

```
import requests

url = "https://api.example.com/quote"
r = requests.get(url, params={"symbol": "AAPL"}, timeout=10)
print(r.status_code)          # e.g. 200
print(r.json())               # parsed JSON as a Python dict
```

Common mistakes.

- Many beginners forget `timeout=...` and watch their program freeze when the server is slow.
- Many beginners build URLs with string concatenation and end up with bad escaping. Use `params=`.

Quick Check. What does `params={"symbol": "AAPL"}` do in a `requests.get` call?

Answer: Adds `?symbol=AAPL` to the URL, correctly escaped — the standard way to pass query values.

`.status_code` and `.json()`

The big idea. Always check the status code before trusting the response body. `.json()` parses the body assuming it is JSON — handy, but it raises if the body is not actually JSON.

This is like when... a courier shows up at the door. You check the delivery note (status) before unwrapping the package (body). A wrong delivery note means the contents are suspect.

The explanation.

`r.status_code` is an integer. 200 means OK; anything from 400 up is an error. Branch on it: if not 200, print a helpful message based on the code and return early. If 200, call `r.json()` to get a Python dict, then pluck the fields you need by name (look at the API docs for the response shape). For many APIs, even a 200 can include an `"error"` key in the JSON — always check.

The code example.

```
r = requests.get(url, params={"symbol": "AAPL"}, timeout=10)
if r.status_code != 200:
    print(f"Request failed with status {r.status_code}")
else:
    data = r.json()
    price = data["latest_price"]      # field names depend on the API
    print(f"AAPL: ${price}")
```

Common mistakes.

- Many beginners call `r.json()` on an error response and crash when the body is HTML or empty. Check the status first.

Quick Check. What does `r.json()` do?

Answer: Parses the response body as JSON and returns the matching Python object (usually a dict).

Common HTTP status codes

The big idea. Status codes follow a pattern: 2xx OK, 3xx redirect, 4xx your fault, 5xx server's fault. Knowing the common ones lets you write helpful error messages.

This is like when... a customer service script has a few standard responses (“your account is locked,” “the system is down”) that match common situations.

The explanation.

The codes you will see: **200** request OK; **400** bad request (your URL or params are malformed); **401** unauthorised (missing/wrong API key); **403** forbidden (the key is valid but not allowed); **404** not found

(the symbol does not exist); **429** too many requests (you hit the rate limit — wait); **500** internal server error (their problem, retry later). Branch on these to give the user a meaningful message instead of “request failed.”

Common mistakes.

- Many beginners lump all errors into one message. 401 vs 429 vs 500 each call for a different reaction — fix your key, slow down, try again later.

Quick Check. What does a 429 status code mean?

Answer: Too many requests — you exceeded the rate limit; wait and retry.

Environment variables and `os.environ`

The big idea. Never hard-code your API key into your source. Store it in an *environment variable* — a setting outside the program — and read it with `os.environ`. You can share the code without leaking the key.

This is like when... you leave a spare house key with a neighbour, not under the doormat. The location is private and changeable; the code that opens the door does not name where it is.

The explanation.

In your shell, run `export STOCK_API_KEY=xyz123` (on Linux/macOS) or `set STOCK_API_KEY=xyz123` (Windows). In Python, `os.environ.get("STOCK_API_KEY")` returns the value. Check for `None` and exit gracefully if the variable is unset — with a helpful message telling the user what to set. This way the key never appears in your code or in your git history.

The code example.

```
import os
api_key = os.environ.get("STOCK_API_KEY")
if not api_key:
    print("Please set the STOCK_API_KEY environment variable.")
    exit(1)
# use api_key in requests.get(params={..., "apikey": api_key})
```

Common mistakes.

- Many beginners commit their API key to git, expose it on GitHub, and watch it get abused within hours. Environment variables prevent that.

Quick Check. Why use an environment variable for an API key rather than a constant in the code?

Answer: So the key never appears in source code or version control; the same code runs with different keys on different machines.

Network errors and exception handling

The big idea. Networks fail. The DNS lookup might fail, the server might be unreachable, the request might time out. `requests` raises specific exceptions for each. Wrap the call in `try / except` to print a friendly message rather than a crash trace.

This is like when... you call a number and it goes to voicemail, or the line is busy. You do not give up forever; you note the problem and try again.

The explanation.

The exception to catch is `requests.exceptions.RequestException`, the parent of all `requests` errors — `Timeout`, `ConnectionError`, `HTTPError`, and others. Catch it broadly, print which API and which symbol you were fetching, and return. For more polish, catch `Timeout` separately to say “the server was slow” specifically.

The code example.

```
import requests
try:
    r = requests.get(url, params={"symbol": "AAPL"}, timeout=10)
```



```
except requests.exceptions.Timeout:
    print("The API timed out – try again in a moment.")
except requests.exceptions.RequestException as e:
    print(f"Network problem: {e}")
```

Common mistakes.

- Many beginners catch `Exception` everywhere and hide real bugs. `RequestException` is specific to network failures.

Quick Check. Which exception class is the parent of all `requests` network errors?

Answer: `requests.exceptions.RequestException`.

④ How the concepts connect

At startup, the program reads the API key from `os.environ.get("STOCK_API_KEY")` and exits with a helpful message if it is unset. To fetch a quote, it builds a `requests.get(url, params={..., "apikey": api_key}, timeout=10)` inside a `try / except requests.exceptions.RequestException`. The response's `.status_code` is checked first — 200 calls `.json()` and pulls the price field; other codes branch to specific messages (401 wrong key, 404 unknown symbol, 429 too many requests, 5xx server error). A small `time.sleep` between calls respects the rate limit when fetching several symbols.

⑤ The build walkthrough

Step 1 • Setup. Sign up for a free API (Alpha Vantage, Finnhub, or similar) and get a key. Read its quote-endpoint docs. Set the key as an environment variable. In Python, read it with `os.environ.get` and exit if missing.

Step 2 • The `get_quote(symbol)` function. Build the URL and params per the API docs. Call `requests.get(url, params=..., timeout=10)` inside a `try` block catching `RequestException`. On success, check `.status_code` — return `None` (or raise) on non-200, with a printed message per code; on 200, return the parsed JSON.

Step 3 • The main loop. Prompt the user for a ticker symbol. Call `get_quote`. If it returned data, pluck and print the latest price, change, and percent change with friendly formatting (`:.2f` and a `+` or `-` prefix). Loop until the user quits.

Step 4 • Watchlist mode. Let the user maintain a list of symbols (persist as JSON, Chapter 6). A “refresh all” command loops the list, calling `get_quote` for each. Add a `time.sleep(1)` between calls to stay below the free-tier rate limit.

Step 5 • Polish. Cache the most recent quote per symbol in memory with a timestamp; if the user re-asks within, say, 60 seconds, return the cached value instead of calling the API again. Print “cached” so they know what happened.

⑥ Understanding the design

Three habits. First, *the key lives outside the code* — environment variables make your code shareable and your key changeable without a re-deploy. Second, *check the status code before trusting the body* — different codes mean different things; a single “request failed” message hides what actually went wrong. Third, *be a polite client* — set a timeout, sleep between calls, handle 429 specifically, and cache when you can. APIs that are abused by bad clients get locked down for everyone.

⑦ The exercise twist

Add price-change alerts that watch in the background and notify you when a threshold is crossed.

What new concept does this need? A background loop that re-fetches periodically — usually with `time.sleep(interval)` between refreshes (Chapter 23 introduces this pattern more fully). Optional: desktop notifications via the `plyer` library.

How does it extend what you built? Add an “alert” command that records (`symbol`, `direction`, `price`). A new “watch” mode enters a loop: fetch each watched symbol, compare against its alert threshold, and notify if crossed. `Ctrl+C` exits the watch mode cleanly with a `try / except KeyboardInterrupt`.

Hint for the hardest part. Avoid double-alerts. Once an alert fires, mark it as “fired” so the watch loop does not re-trigger every refresh until the user resets it.

⑧ What to remember from this chapter

- `requests.get(url, params=..., timeout=10)` sends a GET; check `.status_code` before trusting `.json()`.
- Treat 401, 404, 429, and 5xx differently — each tells the user something useful.
- Read API keys from `os.environ.get("VAR")` and exit if missing; never commit them.
- Wrap network calls in `try / except requests.exceptions.RequestException` to handle timeouts and connection failures.
- Be polite: set a timeout, sleep between requests, cache when reasonable.

⑨ Coming up next

In the next chapter you build a web scraper for sites that do not offer an API — pulling structured data out of HTML with **BeautifulSoup**, while respecting `robots.txt` and rate limits.

Web Scraper

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A command-line web scraper that fetches a web page, parses the HTML, and extracts structured data — for example, all article titles and links from a news listing page. You will use **requests** to fetch the page and **BeautifulSoup** to find the pieces you want by tag, class, or attribute. The scraper waits politely between requests, respects the site’s **robots.txt**, and handles pages whose layout is not quite what you expected. By the end you will be able to pull data out of any reasonably well-structured website — and you will know the ethics that turn a scraper from a useful tool into a responsible one.

② The concepts you need

- **HTML structure.** Tags, attributes, and nesting — the building blocks of every web page.
- **The requests library.** Reused from Chapter 19 to fetch the raw HTML.
- **BeautifulSoup.** The library that parses HTML into a searchable Python object.
- **.find() and .find_all().** Locating a single tag or every tag matching a selector.
- **CSS selectors with .select().** A more powerful querying syntax for nested matches.
- **time.sleep().** Pacing requests so you do not hammer the site.
- **robots.txt and ethics.** What sites tell scrapers about what is allowed.

③ Concept deep dives

HTML structure — tags, attributes, nesting

The big idea. An HTML page is a tree of **tags**: `<h1>`, `<a>`, `<div>`. Each tag can have **attributes** like `class="title"` or `href="..."`. Tags nest inside one another, and you find data by describing where it lives in that tree.

This is like when... a house is a tree of rooms — living room contains a sofa contains a cushion. To find the cushion, you describe the path: “the cushion on the sofa in the living room.”

The explanation.

A scraper needs you to look at the page’s HTML and identify the *shape* around the data you want. Open the page in your browser, right-click the data, choose “Inspect.” You see something like `<a href="/article/123" class="headline">Title here</a>`. That tells you: the data lives in `<a>` tags with class `headline`. Your scraper asks BeautifulSoup for exactly those.

Common mistakes.

- Many beginners try to scrape without first inspecting the page. Always look at the structure in the browser first; the code follows from it.

Quick Check. What do you call `class="headline"` in `<a class="headline">...</a>`?

Answer: An attribute — a name-value pair attached to a tag.

BeautifulSoup — parse HTML into a searchable object

The big idea. `BeautifulSoup(html, "html.parser")` turns a string of HTML into an object you can search using Python. Once parsed, methods like `.find` and `.find_all` locate the tags you want.

This is like when... you scan a paper document into a tool that lets you click any sentence to extract it. The paper becomes searchable.

The explanation.

BeautifulSoup is not in the standard library — `pip install beautifulsoup4`. After `from bs4 import BeautifulSoup`, you create a soup from the fetched HTML: `soup = BeautifulSoup(r.text,`

"html.parser"). The soup represents the whole page. You navigate it with `.find` (first match) and `.find_all` (every match), passing the tag name and optionally a `class_` or other attribute filter. Note the underscore in `class_` — `class` alone is a Python keyword.

The code example.

```
import requests
from bs4 import BeautifulSoup

r = requests.get("https://example.com", timeout=10)
soup = BeautifulSoup(r.text, "html.parser")
title = soup.find("h1")           # first h1 on the page
print(title.text)                # the text inside
```

Common mistakes.

- Many beginners forget the parser argument and see a warning. Always pass "html.parser" (or "lxml" if installed).

Quick Check. Why is the keyword `class_` used in BeautifulSoup queries, with an underscore?

Answer: Because `class` alone is reserved in Python; the underscore avoids the clash.

.find() and .find_all()

The big idea. `.find(tag, ...)` returns the *first* matching element (or `None` if there is none). `.find_all(tag, ...)` returns a *list* of every match. Pass attribute filters with keyword arguments: `class_="headline", id="main"`.

This is like when... a librarian asks “any one book on this shelf” (`find`) versus “every book on this shelf” (`find_all`). Same shelf, different scope.

The explanation.

Most scraping is a single `.find_all` to get a list of items, then a loop pulling specific pieces out of each. Inside the loop, you call `.find` to dig into one item: `item.find("a")["href"]` returns the link from the first `<a>` in that item. `[attr]` reads an attribute value; `.text` reads the text inside a tag.

Always check for `None` — a missing element returns `None`, and reading `.text` or `[...]` on `None` crashes. A page layout changes more often than you would like; defensive code copes when an element is absent.

The code example.

```
articles = soup.find_all("article")
for art in articles:
    h2 = art.find("h2")
    link = art.find("a")
    if h2 and link:
        print(h2.text.strip(), "->", link.get("href"))
```

Common mistakes.

- Many beginners assume every match has every sub-element and crash on the first irregular item. Guard with `if x and y` before accessing.

Quick Check. What is the difference between `tag["href"]` and `tag.get("href")`?

Answer: `[]` raises `KeyError` if the attribute is missing; `.get` returns `None` instead. Prefer `.get` for safety.

CSS selectors with .select()

The big idea. `.select("css selector")` lets you query with the same syntax used in CSS stylesheets — concise and expressive for nested matches. `.select_one(...)` is the single-match version.

This is like when... you switch from picking through a folder one document at a time to using a search box that supports advanced filters — the right query is more powerful than a stack of separate searches.

The explanation.

Common selectors: `"div"` (every `div`), `".headline"` (every element with class `headline`), `"#main"` (the element with id `main`), `"div.item a.link"` (every `<a class="link">` inside any `<div class="item">`). For nested structures, `.select` is often shorter than chaining `.find_all` calls. Use whichever reads more clearly for the structure you face.

The code example.

```
links = soup.select("article h2 a")          # <a> inside <h2> inside <article>
for a in links:
    print(a.get_text(strip=True), a.get("href"))
```

Common mistakes.

- Many beginners reach for `.select` when a simple `.find_all("h1")` would do. Pick the simpler tool when it fits.

Quick Check. What does `.select_one("h1.title")` return?

Answer: The first `<h1>` whose class includes `title` (or `None` if none).

`time.sleep()` — polite pacing

The big idea. Between consecutive requests, call `time.sleep(seconds)` to pause. This keeps your scraper from hammering the site with hundreds of requests per second.

This is like when... you wait politely between knocks on a door. Pounding repeatedly is rude and gets you ignored.

The explanation.

After `import time`, `time.sleep(1.5)` pauses your program for 1.5 seconds. Inside a loop that fetches many pages, add `time.sleep` between iterations. One second is a reasonable starting pace; slower for older sites, faster only if the site explicitly allows it. This protects the site's server *and* protects you from being blocked.

The code example.

```
import time
for url in urls:
    fetch_and_process(url)
    time.sleep(1.5)          # pause between requests
```

Common mistakes.

- Many beginners scrape as fast as possible and get IP-banned within minutes. A slow scraper that finishes is better than a fast one that gets blocked.

Quick Check. What does `time.sleep(2)` do?

Answer: Pauses your program for 2 seconds before continuing.

`robots.txt` and ethics

The big idea. Most websites publish a file at `/robots.txt` saying which paths automated programs are allowed to fetch and which are off-limits. Reading it before scraping is the bare minimum of polite behaviour.

This is like when... a shop has a “Staff Only” sign on a door. Anyone can ignore it, but respectable visitors do not. `robots.txt` is the digital version.

The explanation.

Fetch `https://site.com/robots.txt` and read it. It looks like:

```
User-agent: *
Disallow: /private/
Crawl-delay: 5
```

That says all scrapers are forbidden from `/private/` and should wait at least five seconds between requests. The Python standard library has `urllib.robotparser` which parses this and tells you whether a URL is allowed.

Beyond `robots.txt`, the broader ethics: identify your scraper with a sensible **User-Agent** header so the site can contact you; cache pages you have already fetched; do not scrape private pages, paywalled content, or copyrighted material; and respect any rate-limit headers in the response.

Common mistakes.

- Many beginners ignore `robots.txt` and learn the lesson when the site blocks their IP — or sends a legal email.

Quick Check. What does `Disallow: /private/` in `robots.txt` mean?

Answer: Polite scrapers must not fetch URLs under `/private/`.

④ How the concepts connect

You fetch a page with `requests.get(url, timeout=10, headers={"User-Agent": "..."})`, check the status code (Chapter 19), and pass the HTML to `BeautifulSoup`. You inspect the page in your browser to learn which tags hold the data — then `.find_all` or `.select` extracts the list of items. A per-item loop pulls the title and link with `.find` and `.get`, guarding against missing fields. `time.sleep` paces follow-up requests. Before any of this, the program reads `robots.txt` and refuses URLs the site has disallowed.

⑤ The build walkthrough

Step 1 · Choose a target. Pick a site that publishes a clear listing page (a blog index, a news section). Open it in your browser; inspect the structure around the items you want.

Step 2 · Robots check. Build a `is_allowed(url)` helper that fetches `/robots.txt` (with a timeout), parses it with `urllib.robotparser`, and returns `True/False`. Refuse to fetch any URL it disallows.

Step 3 · Fetch one page. Write `fetch(url)` that calls `requests.get` with a timeout and a friendly **User-Agent** header. Inside `try / except RequestException`, check the status code; return the text on 200, `None` otherwise. Pause with `time.sleep` after each fetch.

Step 4 · Parse. `BeautifulSoup(html, "html.parser")`. Use `.find_all` (or `.select`) to get the item containers. Loop them, extracting the title, link, and any other fields you need with `.find` and `.get`. Guard each access; skip any item where required fields are absent.

Step 5 · Store and pretty-print. Save the results as JSON (Chapter 6) or a CSV (Chapter 16). Print a tidy table to the screen. Optionally support following pagination links to scrape several pages, sleeping between each.

⑥ Understanding the design

Three habits. First, *inspect before you code* — five minutes in DevTools learning the page’s structure saves an hour of guessing. Second, *guard every selector* — pages are messy and layouts change; `.find` returns `None` when an element is missing, so check before using. Third, *be a polite citizen* — read `robots.txt`, identify yourself, sleep between requests. Scraping is a privilege the web grants you; abuse it and the privilege disappears for everyone.

⑦ The exercise twist

Scrape a multi-page site (pagination), store results in a database, and turn the scraper into a re-runnable monitor that only reports new items.

What new concept does this need? Two ideas: detecting and following “next page” links until none remain, and SQLite (a built-in database, covered in Chapter 26) to store items uniquely keyed by URL so duplicates are skipped.

How does it extend what you built? Wrap the per-page logic in a loop that follows the “next” link. On each run, insert new items into the database; report only the rows that did not already exist. Now you have a true monitor: run it nightly and see what is new.

Hint for the hardest part. Defining “new” needs a stable unique key per item. URL is usually best — it is stable across re-fetches. Use it as the primary key in the database; insertion of a duplicate then becomes a quiet skip rather than a duplicate row.

⑧ What to remember from this chapter

- Inspect the page in your browser first; the structure tells you what to query.
- `BeautifulSoup(html, "html.parser")` makes a searchable soup; `.find` (one) and `.find_all` (many) locate tags, optionally filtered by `class_` or other attributes.
- `.select("css selector")` is concise for nested queries; `.select_one` for a single match.
- Read `robots.txt`, send a `User-Agent`, set a timeout, sleep between requests. Polite scrapers do not get blocked.
- Guard every selector with `if elem:` or `.get`. Layouts change; missing fields should skip an item, not crash the program.

⑨ Coming up next

In the next chapter you build a unit converter suite — a clean architecture that handles many conversion types through one menu — using function-dispatch dictionaries and the convert-through-a-base-unit pattern.

Unit Converter Suite

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A unit-conversion suite that handles five categories — length, weight, temperature, volume, time — all behind one tidy menu. Where the simple temperature converter of Chapter 2 had three units, this has dozens; instead of an `if/elif` tangle, you organise the conversions as **dispatch dictionaries** and convert through a **base unit** so each new unit costs only two small functions. By the end you will have built a real piece of architecture: a program whose structure makes it easy to extend without making the existing code more complicated.

② The concepts you need

- **Function-dispatch dictionaries.** A dict mapping a unit name to its conversion function.
- **Conversion through a base unit.** Every conversion goes via one canonical unit (metres, kilograms, Celsius).
- **The temperature special case.** Temperature uses offsets, not just multiplication.
- **A menu architecture.** Top-level menu chooses the category; each category has its own dispatch.
- **Input validation, reused.** Every prompt validated, every error caught.
- **Reusable code through helpers.** One generic `convert(value, src, dst, category)` powers every category.

③ Concept deep dives

Function-dispatch dictionaries

The big idea. Instead of an `if/elif` chain that picks which function to call, store the unit-to-function mapping in a dictionary. Looking the function up by name is one dict access; adding a new unit is one new entry.

This is like when... a phone directory replaces a person who remembers every number from memory. Looking up Alice is one lookup, not a recital of the whole address book.

The explanation.

You saw the seed of this in Chapter 2's dispatch table. Here you go further: for each category, two dicts — `to_base` (unit → function that converts that unit to the base) and `from_base` (base → unit). Each function takes a number and returns a number. Any conversion in the category becomes `from_base[dst](to_base[src](value))`.

The code example.

```
length_to_metres = {
    "m": lambda v: v,
    "km": lambda v: v * 1000,
    "ft": lambda v: v * 0.3048,
}
length_from_metres = {
    "m": lambda v: v,
    "km": lambda v: v / 1000,
    "ft": lambda v: v / 0.3048,
}
# 5 km in feet:
metres = length_to_metres["km"](5)
print(length_from_metres["ft"](metres))  # 16404.2
```

Common mistakes.

- Many beginners write an $N \times N$ table of every-pair-to-every-pair conversions. Going through a base unit needs only $2N$ functions, scales linearly, and is far harder to get inconsistent.

Quick Check. Why is the dispatch-dict approach better than a big `if/elif`?

Answer: Adding a new unit is one or two dict entries, not a new branch in every condition.

Conversion through a base unit

The big idea. Pick one **base unit** per category — metres, kilograms, Celsius, litres, seconds — and route every conversion through it: source $\rightarrow$ base $\rightarrow$ destination.

This is like when... every currency exchange goes through a major reserve currency. Lots of currencies, fewer pairs to remember.

The explanation.

If you tried to define every pairwise conversion directly (km $\rightarrow$ ft, km $\rightarrow$ in, km $\rightarrow$ m, ft $\rightarrow$ m, ft $\rightarrow$ in...) you would have to write N^2 formulas and keep them consistent. With a base unit, you only need *to-base* and *from-base* for each unit — $2N$ formulas. A conversion is the composition of two of them. This applies cleanly to length, weight, volume, and time. Temperature breaks the pattern slightly — covered next.

The code example.

```
def convert(value, src, dst, to_base, from_base):
    if src not in to_base or dst not in from_base:
        raise ValueError("Unknown unit")
    return from_base[dst](to_base[src](value))

print(convert(5, "km", "ft", length_to_metres, length_from_metres))
```

Common mistakes.

- Many beginners pick a tiny base unit (millimetres for length) and the maths gets ugly. Pick the natural base (metres, kilograms).

Quick Check. How many functions do you need for N units in one category, using a base unit?

Answer: $2N$ — one to-base and one from-base per unit.

The temperature special case

The big idea. Length, weight, and volume conversions are pure multiplications: metres $\times 1000$ = millimetres. Temperatures involve **offsets** (zero Celsius is not zero Kelvin), so the formulas are affine, not just linear. Keep the same architecture; just write more careful formulas.

This is like when... most measurements are like distances on a tape — start at the same zero. Temperature is like dates — both have offsets, so converting needs more than scaling.

The explanation.

$C_to_K(c) = c + 273.15$. $F_to_C(f) = (f - 32) * 5 / 9$. $K_to_C(k) = k - 273.15$. Each formula is a small `lambda`, slot it into the dispatch dict the same way. The general `convert` function works unchanged: source $\rightarrow$ base (Celsius) $\rightarrow$ destination. The architecture absorbs the irregularity at the per-function level.

The code example.

```
temp_to_c = {
    "C": lambda v: v,
    "F": lambda v: (v - 32) * 5/9,
    "K": lambda v: v - 273.15,
}
temp_from_c = {
    "C": lambda v: v,
    "F": lambda v: v * 9/5 + 32,
    "K": lambda v: v + 273.15,
}
```

Common mistakes.

- Many beginners try to bake the offset into a single multiplier. Affine formulas need both a multiply and an add.

Quick Check. Why does the convert function still work unchanged for temperature?

Answer: The dispatch dicts store *the formula*; the convert function just calls them. The maths complexity is hidden inside each lambda.

A menu architecture

The big idea. The top-level menu picks a category. Each category has its own dispatch dicts and runs through the same generic conversion shape. The menu is one place to extend; the conversion logic is shared.

This is like when... a restaurant menu has sections — starters, mains, desserts — each with its own choices but the same ordering process.

The explanation.

Group your per-category dicts in a single registry: `categories = {"length": (length_to_m, length_from_m), "weight": (...), ...}`. The main loop asks which category, then asks for value, source unit, destination unit, and calls the generic `convert` with the right dispatch pair. Adding “energy” later is one new pair of dicts plus one registry entry.

Common mistakes.

- Many beginners build a separate flow per category. Generic convert + registry keeps the loop singular.

Quick Check. What lets one main loop handle five (and later more) categories?

Answer: Each category exposes the same shape (to-base and from-base dicts), so one generic convert function works for all.

Input validation, reused

The big idea. Every numeric prompt is `float()` in `try / except ValueError`. Every unit prompt is `.strip().upper()` (or `.lower()`, consistently) and checked against the allowed set. Reject invalid input with a clear message; loop the prompt.

This is like when... every form field on a paper application has its own check before submission. The form is robust because each field gets its own validator.

The explanation.

Write small helpers: `prompt_number(msg)` reads a float, looping until valid; `prompt_unit(msg, allowed)` reads a unit, looping until in the allowed set. The main flow becomes a clean sequence of helper calls and a single convert. Validation cannot live “in the formulas” — it has to happen at the boundary.

Common mistakes.

- Many beginners spread validation across the whole program; pulling it into small helpers reduces both bugs and noise.

Quick Check. Why pull validation into small reusable helpers?

Answer: So every prompt validates the same way and the main flow stays readable.

Reusable code through helpers

The big idea. Architecture wins emerge when the *generic* code stays small and the *specific* code (the formulas) is concentrated in data. The convert function is short; the per-category dicts are long but mechanical.

This is like when... a chef’s recipes hold all the specifics, while their cooking technique is the same across every dish. Skill stays general; details live in the recipe.

The explanation.

This is the deepest takeaway. When you make formulas *data* (functions stored in dicts) rather than *code* (branches in a function), the program’s flexibility scales with the data, not with the code’s complexity. You will see this pattern again — registries, dispatch tables, strategy objects — in every framework you ever use.

Common mistakes.

- Many beginners hide complexity inside functions instead of moving it into data. Data-driven designs are easier to extend without re-reading the logic.

Quick Check. What gets bigger when you add a new unit — the convert function, or the data?

Answer: The data (one or two dict entries). The convert function stays the same.

④ How the concepts connect

A registry maps each category name to its (`to_base`, `from_base`) pair of dispatch dicts. The main loop asks for a category, then prompts for value and units through helpers that validate against the allowed set. A generic `convert(value, src, dst, to_base, from_base)` composes the two dispatch functions to do the conversion. Temperature plugs into exactly the same shape — its only quirk is in the lambdas themselves.

⑤ The build walkthrough

Step 1 · Define the dispatch dicts. For each category, write `to_base` and `from_base` lambdas. Use sensible bases (metres, kilograms, Celsius, litres, seconds).

Step 2 · Generic convert. Write `convert(value, src, dst, to_base, from_base)` that raises `ValueError` for unknown units and otherwise composes the two functions.

Step 3 · Validation helpers. Write `prompt_number(msg)` and `prompt_unit(msg, allowed)` that loop on bad input.

Step 4 · Registry and main loop. A dict mapping category name to the pair of dispatch dicts. The main loop: ask category, ask value, ask source unit (using the keys of the category's `to_base`), ask destination unit, run `convert`, print formatted result.

Step 5 · Polish. Show available units per category. Print results with sensible rounding (`:.4g` works well for converted values across scales). Offer “quit” and “back to categories” at appropriate places.

⑥ Understanding the design

Three habits. First, *base units cut N^2 to $2N$* — pick a base, write to-base and from-base for each unit, and every conversion composes the two. Second, *put formulas in data, not branches* — dispatch dicts let you extend by adding entries, not by editing logic. Third, *one validator per prompt* — pulling validation into small helpers makes the main flow readable and gives you one place to improve error messages.

⑦ The exercise twist

Add currency conversion using live exchange rates from an API.

What new concept does this need? A network call (Chapter 19) for rates, and caching so you do not re-fetch on every conversion.

How does it extend what you built? Currencies fit the architecture perfectly: USD as base, `to_usd[code]` and `from_usd[code]` are simple multipliers. Fetch the rates once per session; build the dispatch dicts from the response. The convert function and main loop are unchanged.

Hint for the hardest part. Cache the rates with a timestamp; refresh only if older than, say, an hour. That keeps you under the API's rate limit and the program responsive.

⑧ What to remember from this chapter

- Dispatch dictionaries map names to functions; one lookup replaces a chain of `if/elif`.
- A base unit cuts pairwise conversions from N^2 to $2N$ — write to-base and from-base, compose for any pair.
- Temperature uses affine formulas (multiply + offset); the architecture absorbs this inside each lambda.
- A registry of (`to_base`, `from_base`) pairs lets one main loop and one convert function serve every category.
- Move complexity into *data* (dicts of small functions) rather than *code* (long branches). New units are entries, not edits.

⑨ **Coming up next**

In the next chapter you build a flashcard app with **spaced repetition** — the date-arithmetic technique that schedules reviews using `timedelta`, JSON-stored dates, and date filtering.

Flashcard App with Spaced Repetition

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A flashcard study tool that schedules reviews using **Leitner spaced repetition**: cards you remember move to a higher box and reappear less often; cards you forget drop back to box 1 and return tomorrow. Each card has a question, an answer, a box number (1–5), and a “next review” date. The app shows only cards that are *due today*, asks them one at a time, updates the box and next-review date based on your answer, and saves everything to JSON. By the end you will be fluent at date arithmetic with `timedelta` and at filtering a collection by date — patterns that power every scheduler.

② The concepts you need

- **The Leitner system.** A simple, evidence-based spaced repetition algorithm using five boxes.
- **`timedelta`.** Date arithmetic — adding days to a date.
- **JSON with date strings.** The “store as `%Y-%m-%d`, parse on read” pattern (Chapter 7).
- **Date filtering.** Selecting cards whose review date is today or earlier.
- **Per-card mutation.** Updating each card’s box and next-review date based on the answer.
- **A study loop.** Asking each due card, scoring, updating, saving.

③ Concept deep dives

The Leitner system — boxes and intervals

The big idea. Five boxes. Every new card starts in box 1, reviewed daily. Get it right and the card moves up a box, where reviews come less often (every 2 days, then 4, then 7, then 14). Get it wrong and it drops straight back to box 1.

This is like when... a teacher reviews easy material rarely and hard material often. The schedule adapts to your real knowledge — you spend time on what you do not yet know.

The explanation.

The intervals are conventionally {1: 1, 2: 2, 3: 4, 4: 7, 5: 14} days. The algorithm is *that simple*. The magic is the schedule: cards you know fade into background reviews while cards you struggle with stay close. This single rule outperforms most study methods.

The next-review date is just `today + timedelta(days=interval)`. Compare the date when deciding what is due.

Common mistakes.

- Many beginners try to fancy it up with weighted scores and difficulty ratings. Leitner’s strength is its simplicity — keep it plain.

Quick Check. If you get a box-3 card right, which box does it go to and when does it next appear?
Answer: Box 4, with a 7-day interval — it reappears in 7 days.

`timedelta` — date arithmetic

The big idea. `timedelta(days=n)` is a duration. Adding it to a date gives a new date *n* days later. Subtracting two dates gives a `timedelta`.

This is like when... you flip a calendar forward by a known number of pages. Today plus seven days lands you a week ahead.

The explanation.

After `from datetime import datetime, timedelta, datetime.now() + timedelta(days=7)` is a week from now. For flashcards you usually want just the date part (no hour/minute), so `date-`

`time.now().date()` returns a **date** object — and `date + timedelta` is a date. Compare dates with `<`, `==`, `>` directly.

The code example.

```
from datetime import datetime, timedelta

today = datetime.now().date()
in_a_week = today + timedelta(days=7)
print(today, in_a_week)
print((in_a_week - today).days)      # 7
```

Common mistakes.

- Many beginners try to do date arithmetic by manipulating numbers (add 30 to the day) and run into month-boundary bugs. Use `timedelta`; it handles month and year rollovers correctly.

Quick Check. What does `date(2026,1,30) + timedelta(days=5)` give?

Answer: `date(2026, 2, 4)` — `timedelta` rolls over the month boundary correctly.

JSON with date strings — the persistence pattern

The big idea. Store dates in JSON as ISO strings ("2026-05-26"); parse them back to **date** objects on load. JSON has no native date type — strings are the bridge.

This is like when... you write a date on an envelope in a standard format the postal service understands, then re-read it when sorting mail back into chronological order.

The explanation.

You met this in Chapter 7. For dates only, the ISO format `%Y-%m-%d` sorts correctly as plain text — useful for older Python without full ISO parsing. Use `date.isoformat()` to serialise ("2026-05-26") and `date.fromisoformat(s)` to parse back. Both built into Python.

The code example.

```
from datetime import date

d = date(2026, 5, 26)
s = d.isoformat()          # "2026-05-26"
print(s)
print(date.fromisoformat(s)) # back to a date
```

Common mistakes.

- Many beginners forget to parse on load and try to compare a string to a date. Convert at the JSON boundary; work with date objects inside the program.

Quick Check. What format does `date.isoformat()` produce?

Answer: "YYYY-MM-DD" — the ISO-8601 date format.

Date filtering — what is due today

The big idea. A card is *due* when its next-review date is today or earlier. Filter the deck with a comprehension testing this condition.

This is like when... you go through a pile of bills and pull the ones whose due date is today or already past — those need attention now.

The explanation.

After loading, parse each card's next-review string into a date. The due set is `[c for c in deck if c.next_review <= today]`. Optionally sort due cards by box (review lower boxes first) or randomise the order. The same comprehension supports `n_due = len([c for c in deck if c.next_review <= today])` for the menu.

The code example.

```

from datetime import date, timedelta

deck = [
    {"q": "a", "next": "2026-05-20"},
    {"q": "b", "next": "2026-12-01"},
]
today = date(2026, 5, 26)
due = [c for c in deck if date.fromisoformat(c["next"]) <= today]
print([c["q"] for c in due])          # ['a']

```

Common mistakes.

- Many beginners use < instead of <= and skip cards that came due exactly today.

Quick Check. Why <= rather than < when checking due dates?

Answer: So cards due *today* are included, not only those overdue.

Per-card mutation — updating box and next-review

The big idea. After the user answers, mutate the card: if correct and below box 5, increment the box; if wrong, reset to box 1. Either way, set the next-review date to today plus the interval for the new box.

This is like when... you move a card from one tray to another after a review — the position records what to do next time.

The explanation.

Two-line update: `card["box"] = new_box, card["next"] = (today + timedelta(days=intervals[new_box])).isoformat()`

Save the deck after every card update so a crash mid-session does not lose progress. Mutating the card in place inside the loop keeps the data shape stable.

Common mistakes.

- Many beginners update the box but forget to update the next-review date, and the card keeps reappearing daily.

Quick Check. What does a wrong answer do to a card's box, regardless of which box it was in?

Answer: Sends it back to box 1.

The study loop

The big idea. Loop through due cards. Show the question, read the answer, score the user's response, update the card, save. End the loop when there are no more due cards or the user quits.

This is like when... a tutor goes through a stack of cue cards with a student until the stack is done, marking each one based on the response.

The explanation.

A simple `for card in due:` loop. For each card, print the question, read the user's answer (handle a "quit" sentinel to break early), and decide right/wrong. The "decide" step can be a strict match, or you can prompt the user themselves: show the right answer and ask "did you know it? (y/n)" — both are valid Leitner.

Common mistakes.

- Many beginners save only at the end and lose progress on a crash. Save after each card.

Quick Check. Why save after each card rather than once at the end?

Answer: A mid-session crash or close does not wipe the work you have already done.

④ How the concepts connect

The deck is a list of card dicts persisted as JSON, with each card's `next` field a date string and `box` an integer 1–5. On start, you load the deck, parse the dates, and filter to those due today. The study loop asks each, takes a yes/no, and updates the card's box (up by one on correct, capped at 5; reset to 1 on wrong) and `next` date (today plus the box's interval). Each update is saved.

⑤ The build walkthrough

Step 1 · Card and deck shape. Each card is `{"q": str, "a": str, "box": int, "next": "YYYY-MM-DD"}`. The deck is a list of cards. Helpers `load_deck(path)` and `save_deck(deck, path)` use JSON.

Step 2 · Menu. Add card, study due cards, view all (with their next-review dates), delete a card, quit. Adding a card defaults `box` to 1 and `next` to today's date.

Step 3 · The due filter. Define `INTERVALS = {1: 1, 2: 2, 3: 4, 4: 7, 5: 14}`. Implement `due_cards(deck, today)` as a comprehension comparing parsed dates.

Step 4 · The study loop. Compute the due list. If empty, congratulate the user. Otherwise loop, asking each card. Read the answer, score it (strict match or self-grade), update `box` and `next`, save.

Step 5 · Reports. “Stats” prints how many cards are in each box, how many are due today, how many are due tomorrow. These are all comprehensions over the deck.

⑥ Understanding the design

Three habits. First, *store dates as ISO strings; parse on load* — this keeps your JSON readable, sortable as text, and survives every Python version. Second, *mutate one card at a time and save* — the deck stays consistent on disk even if the program is interrupted. Third, *let the algorithm be simple* — Leitner's whole strength is that the rule is a sentence (“right moves up, wrong resets”); the elegance of your code should match.

⑦ The exercise twist

Add multiple decks, an importer for CSV vocabulary lists, and a “review history” log.

What new concept does this need? Multiple decks lift the top-level structure: a `decks/` folder with one JSON file per deck. CSV import (Chapter 16) reads pairs into card dicts. A history log appends a line per answer with the date, card ID, and outcome.

How does it extend what you built? A menu choice picks the active deck; the rest of the program is unchanged. CSV import is a small function that returns a list of cards to merge into the deck. The history file gives you the data to draw streaks and learning curves.

Hint for the hardest part. Give each card a stable unique ID (e.g. the result of `secrets.token_hex(4)`) so the history log can reference cards even after their question text changes.

⑧ What to remember from this chapter

- Leitner: five boxes, intervals `{1:1, 2:2, 3:4, 4:7, 5:14}`; right moves up, wrong resets to 1.
- `timedelta(days=n)` plus a date returns a new date; date arithmetic handles month boundaries correctly.
- Store dates as ISO strings (`date.isoformat()`); parse with `date.fromisoformat(s)` on load.
- Filter due cards with `[c for c in deck if next_date <= today]` — `<=` includes today.
- Save after every card update so a mid-session crash loses nothing.

⑨ Coming up next

In the next chapter you build a Pomodoro timer — the first program that *runs over time*, using `time.sleep`, live terminal updates with `\r`, and graceful `KeyboardInterrupt` handling.

Pomodoro Timer

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A Pomodoro timer: 25 minutes of work, a 5-minute short break, repeat. After every four work intervals, take a 15-minute long break. The terminal shows a live, updating countdown — the seconds tick down on a single line, not by spamming new lines. `Ctrl+C` stops the timer cleanly with a saved summary of completed sessions. By the end you will have built your first program that *runs over time*, using `time.sleep`, a single-line live update via `\r`, and `try / except KeyboardInterrupt` to exit gracefully.

② The concepts you need

- `time.sleep()` and `time.time()`. Pausing, and measuring elapsed time.
- A live single-line display. `\r` and `end=""` and `flush=True` together.
- `KeyboardInterrupt`. Catching `Ctrl+C` so quitting is graceful.
- The Pomodoro state machine. Work → short break → work → ... → long break.
- `datetime.now().strftime()`. Reused — timestamps for the session log.
- Appending to a log file. 'a' mode (Chapter 15) to record completed Pomodoros.

③ Concept deep dives

`time.sleep()` and `time.time()`

The big idea. `time.sleep(seconds)` pauses your program for that many seconds. `time.time()` returns the current time as a number of seconds. Use `sleep` to wait; use `time` to measure.

This is like when... you set a kitchen timer and wait (`sleep`), then check the clock to see how long the meal really took (`time`). Different tools, different jobs.

The explanation.

`time.sleep` blocks — nothing else in your program runs during the pause. For the countdown you sleep 1 second at a time, redrawing the display each tick. `time.time()` gives a float (seconds since the Unix epoch); the difference between two calls is an elapsed duration. For a clean countdown, you do not need `time.time()` — the loop counter is enough — but it is useful for “how long was the session, really?”

The code example.

```
import time
start = time.time()
time.sleep(2)           # pause two seconds
print(time.time() - start) # ~2.0
```

Common mistakes.

- Many beginners assume `time.sleep(0.1)` is exact. Real sleep can overshoot slightly; for a precise countdown, measure with `time.time()` rather than counting sleeps.

Quick Check. Which call pauses the program, and which reports the clock?

Answer: `sleep` pauses; `time` reports.

A live single-line display

The big idea. To update a line in place instead of printing a new line each tick, end your print with `\r` (carriage return — back to column 1) and no newline, and flush the output so it appears immediately.

This is like when... you erase the time on a chalkboard and write the new time on the same line, instead of starting a fresh line every second.

The explanation.

The three required pieces: `\r` at the start of the next message moves the cursor back to column 1 over the old text; `end=""` keeps `print` from adding a newline; `flush=True` forces the buffer to write immediately (without it, the line may sit invisible until the program decides to flush). When the countdown ends, print a final newline so the next message lands on its own line.

The code example.

```
import time
for s in range(5, 0, -1):
    print(f"\rTime left: {s} s", end="", flush=True)
    time.sleep(1)
print("\rDone!          ")      # extra spaces overwrite leftover text
```

Common mistakes.

- Many beginners omit `flush=True` and see nothing until the loop ends.
- Many beginners forget that the new line might be shorter than the old; pad with spaces or use a fixed width.

Quick Check. What three flags make `print` redraw on the same line?

Answer: Start with `\r`, set `end=""`, set `flush=True`.

KeyboardInterrupt — handling Ctrl+C gracefully

The big idea. Pressing `Ctrl+C` raises `KeyboardInterrupt`. Catch it to clean up — save the session log, restore the terminal, print a friendly goodbye — instead of crashing with a stack trace.

This is like when... you hear a fire alarm and walk calmly to the exit instead of leaving by jumping out of a window. Both leave the building; one keeps you in one piece.

The explanation.

Wrap the main loop in `try / except KeyboardInterrupt`. Inside the `except`, do whatever cleanup is appropriate — write the partial session to the log, print a summary, return to the menu. `KeyboardInterrupt` is not a regular exception; do not catch it with bare `except` — be explicit.

The code example.

```
try:
    for s in range(60, 0, -1):
        print(f"\r{s:3d} s", end="", flush=True)
        time.sleep(1)
    print("\nDone")
except KeyboardInterrupt:
    print("\nStopped early – saving progress.")
```

Common mistakes.

- Many beginners catch `Exception` and accidentally swallow `KeyboardInterrupt`. It is its own thing — catch it specifically.

Quick Check. What does `Ctrl+C` raise that you can catch?

Answer: `KeyboardInterrupt`.

The Pomodoro state machine

The big idea. The session cycles through states: work, short break, work, short break, ... after every four works, a long break. A counter tells you which break comes next.

This is like when... a traffic light cycles through fixed states in a fixed pattern. The state names and durations are different; the cycle pattern is the same.

The explanation.

Keep a counter `work_done` starting at 0. After each work block, increment. If `work_done % 4 == 0`, run a long break; otherwise a short one. Loop forever (or until a configured count). Wrap each phase in its own

countdown function, so the main loop reads like the pattern itself: `run("Work", 25*60); work_done += 1; run(break_name, break_length).`

Common mistakes.

- Many beginners write all four cases (first work, first short break, ...) inline and the structure is buried. Pulling each phase into a function (or just two — work and break) makes the state machine visible.

Quick Check. After how many work blocks does a long break replace the short break?

Answer: Every four — when `work_done % 4 == 0`.

Timestamps and an appended log

The big idea. Every completed work interval is a row in a log file: timestamp, duration, label. Use 'a' mode so each entry adds without erasing the previous.

This is like when... a runner records each completed lap as a new line on a tally sheet. The page grows; old entries stay.

The explanation.

You used 'a' mode in Chapter 15 only briefly. Here it shines: every time a Pomodoro completes, open the log in 'a' mode and write one line: `f"{datetime.now().isoformat()} {duration} {label}\n"`. Even a partial session (interrupted by `Ctrl+C`) can write a row tagged “partial.” Tomorrow’s “stats” command parses the log and prints how many Pomodoros you completed today.

Common mistakes.

- Many beginners open the log in 'w' and wipe their history every run.

Quick Check. Which file mode appends without erasing?

Answer: 'a'.

④ How the concepts connect

A `countdown(seconds, label)` function runs a `time.sleep(1)` loop, redrawing `f"\r{label}: M:SS"` with `end=""`, `flush=True`. The main loop alternates work and break phases, using a `work_done` counter to choose a long break every four works. The entire main loop is wrapped in `try / except KeyboardInterrupt` for graceful exit. Each completed phase appends a line to `pomodoro_log.txt` with a timestamp.

⑤ The build walkthrough

Step 1 · Configuration. Constants for the durations: `WORK = 25 * 60`, `SHORT = 5 * 60`, `LONG = 15 * 60`. The work-blocks-per-cycle is 4.

Step 2 · The countdown function. `countdown(seconds, label)` loops `seconds` down to 0, sleeping 1 each iteration. Inside, format remaining time as `M:SS` and print with `\r`, `end=""`, `flush=True`. After the loop, print a final newline plus a completion message. Optionally play a system bell with `print("\a")`.

Step 3 · Log helper. `log_session(label, seconds)` opens the log file in 'a' mode and writes a line with `datetime.now().isoformat()`, the duration in minutes, and the label.

Step 4 · Main loop. `work_done = 0` outside a `try` block. Inside: `while True`: running a work countdown, incrementing `work_done`, logging, then choosing a break length based on `work_done % 4 == 0`, running the break, looping. Wrap the whole `while` in `try / except KeyboardInterrupt`. On interrupt, log the partial phase and print a friendly summary.

Step 5 · Stats command. Outside the timer, a “stats” command parses the log file: count today’s completed Pomodoros, the total focused minutes today, and a list of past days with their counts.

⑥ Understanding the design

Three habits. First, *one place that redraws the line* — `countdown` is the only function that prints the live display, so any change to the format (showing minutes vs seconds, adding a bar) happens once. Second, *catch `KeyboardInterrupt` explicitly* — never with a bare `except`; the contract is “the user wants to stop,

save what you have, then exit.” Third, *append, never overwrite, the log* — ‘a’ mode is the difference between “a log” and “the latest entry.”

⑦ The exercise twist

Add per-task tracking (label your Pomodoros), a daily target, and a desktop notification at the end of each phase.

What new concept does this need? Notifications use the `plyer` library (cross-platform desktop notifications). Per-task tracking is just an extra column in the log. The daily target is a number compared to the parsed stats.

How does it extend what you built? Add a label prompt before each work block; pass it through to `countdown` and the log. After each phase, fire a notification with the next phase and its label. The stats command compares today’s count to the daily target and shows progress.

Hint for the hardest part. Make `plyer` optional. Wrap the import in `try / except ImportError`; if it failed, fall back to printing a banner. This keeps the program working when the library is unavailable.

⑧ What to remember from this chapter

- `time.sleep(seconds)` pauses; `time.time()` reports the clock. Sleep for waits; time for measurements.
- Live single-line updates: start with `\r`, `print(..., end="", flush=True)`; pad to fixed width to overwrite leftover characters.
- `Ctrl+C` raises `KeyboardInterrupt`; catch it explicitly to save progress and exit gracefully.
- For cycle logic, use a counter and `counter % n == 0` to trigger the rare branch.
- Append to log files with ‘a’ mode so history accumulates across runs.

⑨ Coming up next

In the next chapter you bring together everything from Chapter 9’s expense work and Chapter 22’s date arithmetic into a personal budget tracker with category limits and a simple savings projection.

Personal Budget Tracker

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A personal budget tracker that records your **income** and **expenses** by category, compares each category's spending against a monthly budget, and projects whether your current pace will hit your savings goals. It is a synthesis project: every concept from Chapter 9 (dates, filtering, sum-with-generators), Chapter 6 (JSON persistence), and Chapter 22 (date arithmetic) comes together. By the end you will have built a small piece of real personal-finance software — useful enough to use, simple enough to understand entirely.

② The concepts you need

- **A unified transactions list.** Income and expense as the same shape with a **type** field.
- **Nested dicts for budgets.** A per-category limit, persisted with the transactions.
- **Date filtering for “this month.”** Reused from Chapter 9.
- **Status calculation.** Compare spending to budget; classify under, warning, over.
- **Linear projection.** “If you keep this pace, you will save X by year-end.”
- **Savings goals.** A target amount and a target date — how much per month to reach it.

③ Concept deep dives

A unified transactions list

The big idea. Income and expenses are *the same kind of thing* — a dated, categorised amount — distinguished only by a **type** field. One list, one shape, simpler everything.

This is like when... a bank statement lists deposits and withdrawals as rows of the same form. The amount has a sign or a column; the structure is uniform.

The explanation.

A transaction is `{"date": "YYYY-MM-DD", "type": "income"|"expense", "category": str, "amount": float, "note": str}`. The full ledger is a list of these. Net income for a month is `sum(income amounts) - sum(expense amounts)`. Per-category spending filters by `type == "expense"` and the category. Keeping one shape avoids two parallel lists you have to keep in sync.

Common mistakes.

- Many beginners keep **income** and **expenses** as separate lists, then duplicate every date-filtering and summing operation.

Quick Check. What field distinguishes an income row from an expense row?

Answer: The **type** field — both are otherwise identical in shape.

Nested dicts for budgets

The big idea. Budgets are a dictionary mapping category to monthly limit: `{"Food": 60000, "Transport": 30000, ...}`. Stored alongside the transactions in the JSON file as a nested structure.

This is like when... a household budget pinned to the fridge has one row per category with the agreed monthly cap. The actual spending is tracked separately and compared.

The explanation.

The persisted JSON has two top-level keys: **"transactions"** (a list of transaction dicts) and **"budgets"** (a dict of category→limit). On load, both come back exactly as you saved them. Setting or updating a budget is one dict assignment plus save. Looking up a category's limit is one `.get` with a sensible default (treat unset as “no limit”).

The code example.

```
data = {
    "transactions": [],
    "budgets": {"Food": 60000, "Transport": 30000},
}
data["budgets"]["Bills"] = 50000      # add or update
print(data["budgets"].get("Other", 0)) # default for unset
```

Common mistakes.

- Many beginners store the budgets in a separate file. One file with both makes load/save symmetric and avoids two-file consistency bugs.

Quick Check. How do you safely look up a budget for a category that may not be set?

Answer: `data["budgets"].get(category, default)` — returns the default if missing instead of raising.

Date filtering for “this month”

The big idea. Reuse the Chapter 9 pattern: parse each transaction’s date string with `date.fromisoformat`, then filter the list by year and month equal to today’s.

This is like when... you flip through a stack of receipts and pull out the ones whose stamped date sits inside this calendar month.

The explanation.

`today = date.today()`. The this-month filter is `[t for t in txns if date.fromisoformat(t["date"]).year == today.year and ... .month == today.month]`. To keep the comprehension readable, write a small helper `in_month(t, year, month)` and use it: `[t for t in txns if in_month(t, today.year, today.month)]`.

Common mistakes.

- Many beginners filter on the string with `startswith("2026-05")` which works until the year ends. Parsing is more robust.

Quick Check. Why parse the date string rather than use `startswith`?

Answer: So the filter handles year boundaries and is comparable for general date arithmetic.

Status calculation — under / warning / over

The big idea. For each category with a budget, compute the ratio `spent / limit`. Under 0.8 is fine; 0.8–1.0 is a warning; above 1.0 is over budget. Return a status label so the report can show it.

This is like when... a fuel gauge has green, amber, and red bands. The same physical reading produces a categorical message that prompts action.

The explanation.

Write `def status(spent, limit):` returning "ok", "warning", or "over" based on the ratio. Guard the no-limit case (`limit == 0` or absent) by returning "unset". The monthly report walks the budgets, computes each category’s spent via a filter+sum, and decorates the line with the status label.

The code example.

```
def status(spent, limit):
    if not limit:          return "unset"
    if spent > limit:      return "over"
    if spent / limit >= 0.8: return "warning"
    return "ok"

print(status(50000, 60000))    # ok
print(status(55000, 60000))    # warning
print(status(70000, 60000))    # over
```

Common mistakes.

- Many beginners skip the `limit == 0` guard and crash with `ZeroDivisionError` on unset categories.

Quick Check. What threshold separates “warning” from “ok”?

Answer: 80% of the budget — `spent / limit >= 0.8`.

Linear projection — at this pace, you will...

The big idea. If you have spent `X` over `Y` days, project the full month as `X / Y * days_in_month`. Same for income. Subtract for projected savings. Crude but instantly useful.

This is like when... you check your fuel gauge a third of the way through a trip and double-check you have enough for the full distance. A simple multiplication tells you whether to slow down.

The explanation.

Use the `calendar.monthrange(year, month)[1]` to get the number of days in the current month. Today’s day-of-month is the elapsed-days count. Project: `monthly_estimate = current_total / day * days_in_month`. Guard against `day == 0` (first of the month). It is a linear extrapolation, no smoothing — perfect for “are we on track?” reports.

Common mistakes.

- Many beginners extrapolate from the first day of the month and produce absurd numbers. Show the projection only after, say, day 3, or warn that early-month projections are noisy.

Quick Check. On day 10 of a 30-day month, your spending is 20000. What is the linear projection for the full month?

Answer: 60000. `20000 / 10 * 30`.

Savings goals

The big idea. A goal is `{"name": str, "amount": float, "by": "YYYY-MM-DD"}`. Compute months remaining until the target date; divide the remaining amount by that many months to get the “needed monthly savings.” Compare to your current pace.

This is like when... you plan to save up for a phone in six months — you divide the price by six and check that your monthly savings match.

The explanation.

`months_left = (target.year - today.year) * 12 + (target.month - today.month)`. Guard `months_left <= 0` (the date is in the past). Needed-per-month is `(amount - current_saved) / months_left`. Show next to the user’s current projected monthly savings — if greater than needed, the goal is on track; otherwise show the gap.

Common mistakes.

- Many beginners compute months as `(target - today).days / 30` and get fractional months that confuse the user. Use the year/month difference for cleaner numbers.

Quick Check. How do you compute months between today (2026-05) and a target of 2026-11?

Answer: 6 months — `(2026 - 2026) * 12 + (11 - 5)`.

④ How the concepts connect

The JSON file holds a transactions list and a budgets dict. Adding a transaction validates type, category, amount, and date, then appends and saves. The monthly report filters the list to this month, computes per-category spending with `sum()` generators, looks up each budget, and labels each line with a status. The “projection” view multiplies the elapsed-days pace by days-in-month; the “goals” view compares that to each goal’s required monthly savings.

⑤ The build walkthrough

Step 1 · Data shape and persistence. Top-level dict with `transactions`, `budgets`, and `goals` lists/dicts. `load(path)` returns the parsed structure or a default empty version on `FileNotFoundError`; `save(data, path)` dumps with `indent=2`.

Step 2 · Add transaction. Prompt type (income/expense), category, amount, date (default today), note. Validate each (type from a fixed set, amount as positive float, date with `fromisoformat`). Append and save.

Step 3 · Set budget and goal. Two small menu choices: set a category budget (updates `data["budgets"]`) and add a goal (appends to `data["goals"]`).

Step 4 · Monthly report. Filter to this month. Compute total income, total expense, net. Per category in `budgets`: filter expenses to that category this month, sum, and print with the status label.

Step 5 · Projection and goals. Compute days elapsed in month and days-in-month. Project monthly income, expense, and net savings. For each goal, compute months-left and required-per-month; print on-track or short, with the gap if short.

⑥ Understanding the design

Three habits. First, *one shape for all transactions* — income and expense share the same form, so every filter and total works uniformly. Second, *every report is computed fresh* — none of the totals or projections are stored; they are derived from the raw transactions and budgets on each view, so they can never go stale. Third, *guards in the projections* — start-of-month, no budget set, target date in the past — each case has its own friendly message instead of a crash.

⑦ The exercise twist

Multi-currency, recurring transactions (monthly bills), and a chart export.

What new concept does this need? Currency adds an exchange-rate lookup (Chapter 19) and a per-transaction currency field. Recurring transactions need a small scheduler — on load, generate any missing recurrence rows up to today. Chart export uses `matplotlib` (not yet covered) or simpler ASCII bars.

How does it extend what you built? A “currencies” view fetches today’s rates and shows your totals converted to a base currency. A “recurring” menu defines monthly bills (amount, category, day-of-month); the loader inserts missing rows so reports always include them.

Hint for the hardest part. Idempotency for recurring rows: tag each generated row with a unique key like `f"{recurring_id}-{yyyy-mm}"` so re-running the generator never duplicates rows that already exist.

⑧ What to remember from this chapter

- One transactions list with a `type` field beats two parallel lists.
- Persist transactions and budgets in one JSON file; budgets are a `category → limit` dict.
- Status from a ratio: `<0.8` ok, `0.8–1.0` warning, `>1.0` over, with a `limit == 0` guard.
- Linear projection: `current / days_elapsed * days_in_month`; warn that early-month projections are noisy.
- Goals are amount + date; convert to months-left and required-per-month; compare to current pace.

⑨ Coming up next

In the final chapter of Part 1 you leave Python for a moment and pick up **Bash** — the shell language your terminal already speaks — to write a small collection of file-management scripts that complement everything you have built.

Bash Script Collection

Phase 1: Foundations · Tier: CLI & Python Core

① What you are building

A short collection of **Bash** scripts — small file-management tools you can run directly from your terminal. Bash is the language your shell already speaks, and writing scripts in it makes you ten times more powerful at the command line. You will build: a backup script that copies a folder with a timestamp, a **largest-files** script that lists the biggest files in a directory, a duplicate-finder, and a deploy script that ties them together. By the end of this last chapter of Part 1 you will know enough Bash to automate the everyday tasks Python is overkill for.

② The concepts you need

- **The shell vs the file.** Bash, your terminal, scripts, and the shebang line.
- **Variables and substitution.** `name="value"`, `$name`, `$(command)`.
- **Conditionals and loops.** `if`, `for`, `while` in Bash syntax.
- **Pipes and redirection.** `|`, `>`, `>>`, `2>&1`.
- **The classic tools.** `grep`, `awk`, `sed`, `find`, `du`, `sort`, `head`, `tail`, `wc`.
- **Making a script runnable.** `chmod +x`, the `PATH`, and the shebang.

③ Concept deep dives

The shell, scripts, and the shebang

The big idea. When you open a terminal you are typing into a *shell* — usually Bash or Zsh — which reads your commands and runs them. A **script** is just a text file full of those same commands. The first line, the **shebang**, tells the OS which shell to run it with.

This is like when... you talk to a person and you give them a written list of instructions to follow later. Same instructions; one is live, the other recorded.

The explanation.

The shebang line is `#!/bin/bash` (or `#!/usr/bin/env bash` for portability). It must be the very first line. Save the file as `script.sh`, make it executable with `chmod +x script.sh`, and run it with `./script.sh`. Lines starting with `#` (after the shebang) are comments — explain what you are doing.

The code example.

```
#!/bin/bash
# script.sh - say hello
echo "Hello from Bash!"
```

Save, run `chmod +x script.sh`, then `./script.sh`.

Common mistakes.

- Many beginners omit `chmod +x` and try to run the file. The OS refuses; the shebang alone is not enough — execute permission is required.

Quick Check. What two things make a file runnable as a Bash script?

Answer: A `#!/bin/bash` shebang on the first line, and execute permission via `chmod +x`.

Variables and substitution

The big idea. `name="value"` defines a variable (no spaces around `=`). `$name` or `${name}` reads it. `$(command)` runs a command and substitutes its output.

This is like when... you write “tomorrow’s date” in a letter and someone fills it in when the letter is delivered. The placeholder gets replaced with a real value when used.

The explanation.

Two crucial syntax rules. First, no spaces around `=` when assigning — `name="value"`, not `name = "value"` (the latter looks for a command called `name`). Second, always *quote* variable expansions: `"$name"` rather than bare `$name` — without quotes, values with spaces explode into multiple words. `$(...)` is **command substitution** — `today=$(date +%Y-%m-%d)` captures the date as a string.

The code example.

```
#!/bin/bash
name="Stephane"
today=$(date +%Y-%m-%d)
echo "Hello, $name. Today is $today."
```

Common mistakes.

- Many beginners put spaces around `=` and get a confusing “command not found” error.
- Many beginners forget the quotes and watch their script break the moment a filename contains a space.

Quick Check. What does `today=$(date +%Y-%m-%d)` store in `today`?

Answer: Today’s date as a string in YYYY-MM-DD form.

Conditionals and loops

The big idea. Bash has `if` and `for` and `while` like every language; the syntax is just different from Python. Conditions are tested with `[ ... ]` (or `[[ ... ]]`); loops iterate over space-separated lists.

This is like when... every language has nouns, verbs, and questions; the words and word order differ but the ideas translate.

The explanation.

`if [ -f "file.txt" ]; then ...; fi` — `-f` tests “is a regular file.” Other tests: `-d` directory, `-z` empty string, `=` string equality, `-eq` numeric equality. `for f in *.txt; do ...; done` loops every text file in the current directory. `while read line; do ...; done < file.txt` reads a file line by line.

The code example.

```
#!/bin/bash
if [ ! -d "backups" ]; then
    mkdir backups
fi

for f in *.txt; do
    echo "Found: $f"
done
```

Common mistakes.

- Many beginners write `if [name="value"]` with no spaces inside the brackets. Bash needs spaces: `if [ "$name" = "value" ]`.

Quick Check. What test would you use to check if a path exists as a directory?

Answer: `[ -d path ]`.

Pipes and redirection

The big idea. `command1 | command2` sends the output of the first into the second. `> file` writes output to a file (overwriting); `>> file` appends. `2>&1` sends error messages to the same place as output.

This is like when... an assembly line passes a part from one worker to the next. Each does its job; the part flows.

The explanation.

This is the original Unix philosophy: small tools each doing one thing well, composed via pipes into pipelines. `ls -l | grep "^d"` lists directories (lines from `ls -l` whose first character is `d`). `command > out.log 2>&1` captures both normal output and errors in one file. `find . -name "*.py" | wc -l` counts every Python file in the tree.

The code example.

```
# Count Python files in the tree, save the count to a log
find . -name "*.py" | wc -l > python_count.txt
```

Common mistakes.

- Many beginners use `>` when they meant `>>` and silently wipe their log file.

Quick Check. What does `cmd > file` do that `cmd >> file` does not?

Answer: `>` overwrites the file; `>>` appends to it.

The classic tools — `grep`, `find`, `awk`, `sed`, `du`, `sort`

The big idea. A handful of standard tools cover most file-and-text tasks. `grep` searches, `find` finds files, `awk` processes columnar text, `sed` substitutes, `du` measures disk usage, `sort` orders, `head/tail` slice.

This is like when... a workshop has a small set of well-loved tools that handle 95% of jobs. You do not need every gadget; you need to know these well.

The explanation.

A few flagship one-liners worth memorising:

```
# Search code for a TODO
grep -rn "TODO" .                # recursive, with line numbers

# Find Python files modified in the last 7 days
find . -name "*.py" -mtime -7

# Top 10 biggest folders here
du -sh */ | sort -rh | head -n 10

# Replace 'foo' with 'bar' in a file (in-place backup with .bak)
sed -i.bak 's/foo/bar/g' file.txt

# Print column 1 of a CSV
awk -F',' 'print $1' data.csv
```

You do not need to master each tool — you need to recognise what they do, so you reach for the right one and look up the exact flags.

Common mistakes.

- Many beginners try to write Python for tasks the shell does in one line. Pipelines are often faster and shorter.

Quick Check. Which tool would you use to find files modified in the last 7 days?

Answer: `find` with `-mtime -7`.

Making a script runnable from anywhere

The big idea. A script in your current folder runs as `./script.sh`. To run it from anywhere by name, put it in a folder on your `PATH` (such as `~/.local/bin`) and make sure that folder is in `PATH`.

This is like when... a tool kept in a workshop drawer is useful where you are. To use it in any room, you put a copy in your tool belt.

The explanation.

`echo $PATH` shows your colon-separated list of folders. If `~/.local/bin` is in `PATH` (most distributions add it), you can copy your script there as `backup` (no extension required), and `backup` will work from anywhere.

Adding a folder to PATH is a one-line edit in `~/.bashrc`: `export PATH="$HOME/.local/bin:$PATH"`. Re-open the terminal or `source ~/.bashrc` to apply.

Common mistakes.

- Many beginners do not realise PATH exists and copy-paste full file paths every time. Putting scripts on PATH is a one-time setup with huge payoff.

Quick Check. What environment variable controls which folders the shell searches for commands?

Answer: PATH.

④ How the concepts connect

Each script is a `.sh` file starting with a shebang, declaring its purpose in a comment block, then doing its job with variables, conditionals, loops, and the classic tools. Inputs come from positional arguments (`$1`, `$2`, ...). Outputs go to stdout or to a file via redirection. The scripts compose: your `deploy.sh` can call `backup.sh`, then run a Python program, then `tail` a log file — each piece small and clear, the whole more useful than the sum.

⑤ The build walkthrough

Step 1 · backup.sh — timestamped folder copy. Take a source folder as `$1`. Refuse if `$1` is empty or not a directory (`[ -z "$1" ] || [ ! -d "$1" ]`). Build the destination as `backups/${1}_${date +%Y-%m-%d_%H-%M}`. Make the parent folder. Copy with `cp -r`. Print a confirmation.

Step 2 · largest-files.sh — top 10 biggest files in a folder. Take a folder as `$1`, defaulting to `..`. Use `find "$1" -type f -printf "%s %p\n"` (or `du -ab "$1"` for portability), sort by size descending, take the top 10, and print each with `numfmt` for human-readable sizes.

Step 3 · find-dupes.sh — duplicate files by content hash. Walk the tree with `find`. Pipe to `xargs sha256sum`. Pipe to `sort`, then to `uniq -d -w 64` to keep lines whose first 64 characters (the hash) repeat. Print the duplicates.

Step 4 · clean.sh — remove temp files. Find files matching `*.tmp`, `*~`, `.DS_Store`, etc. Show what would be deleted first. Ask for confirmation with `read -p "Delete? (y/n) " ans`. On `y`, delete with `rm -i` (interactive — extra safety).

Step 5 · deploy.sh — the orchestrator. Run `backup.sh ./project`, then run your Python build, then `tail -n 20` the build log. Use `set -e` at the top so the script aborts on the first error. Wrap in a “are you on the right branch?” check using `git`.

⑥ Understanding the design

Three habits. First, *small scripts compose* — each does one thing; the orchestrator script chains them. This mirrors the Python “small functions, big composition” idea you have been practising. Second, *guard inputs at the top* — empty arguments, missing folders, the wrong branch — every script should refuse to run on bad inputs rather than half-do the job. Third, *use the classic tools rather than reinvent* — `find` | `sort` | `head` is shorter, faster, and more correct than a Python script for the same job; reach for the shell first for filesystem and text work.

⑦ The exercise twist

Turn the collection into a deploy pipeline: `backup` → `tests` → `build` → `smoke check` → `notify`.

What new concept does this need? `set -e` to fail fast, exit codes (`$?`) to test whether a step succeeded, and conditional execution with `&&` and `||`. Notifications via a webhook (`curl`) or a desktop tool.

How does it extend what you built? A `pipeline.sh` runs each phase; on any failure (`set -e` triggers, or an `||` branch), it logs the failure, rolls back to the last backup, and notifies you. Treat the pipeline as a real piece of infrastructure — comments, dated logs, exit codes mean something.

Hint for the hardest part. A failing pipeline must leave the system in a known state. Either succeed entirely, or roll back. Define rollback explicitly — copy the latest backup into place — and call it from a `trap` so it runs even if the script crashes.

⑧ What to remember from this chapter

- Bash scripts start with `#!/bin/bash`, get `chmod +x`, and run with `./script.sh` (or from anywhere if on `PATH`).
- Variables: `name="value"` (no spaces), read as `"$name"` (always quote); `$(cmd)` substitutes a command's output.
- Tests use `[ ... ]` with spaces; common flags: `-f` file, `-d` directory, `-z` empty string, `=` string equality.
- Pipes (`|`) compose tools: `find | sort | head` is shorter and faster than equivalent Python for file/text work.
- The five tools to know: `grep` (search text), `find` (search files), `du` (sizes), `sort` (order), `awk/sed` (column and substitution).

⑨ Coming up next — and Part 1 closing

That is the end of Part 1. You have built 25 projects covering every core Python tool — values, loops, functions, classes, files, JSON, HTTP, hashing, scraping — and learned enough Bash to be effective at the terminal. From here, Part 2 (Projects 26–50) moves into mathematics and algorithms — first encounters with NumPy, plotting, sorting and searching, simple machine-learning loops — and Part 3 (51–70) reaches into the web with Flask, Django, and small full-stack apps. Each is a continuation; the foundation you have built here is what makes them possible.

Take a break, look back at how much you can now do, and — when you are ready — bring me the next chapter.